

The background features a large, light blue watermark logo of the Technical University of Cluj-Napoca. The logo consists of a shield with a stylized 'T' and 'U' inside, with the text 'TECHNICAL UNIVERSITY' at the top, 'OF CLUJ-NAPOCA' in the middle, and 'Computer Science' at the bottom.

Logic Programming

Lecture #1, 2021
Cluj-Napoca

Computer Science

Agenda

- **Administrative stuff**
- **What this course is/is NOT about**
- **Logic in Programming**
- **Main LP elements**
- **LP program**
 - **Syntax**
 - **Semantic**
 - **Execution tree**

Administrative stuff

- English track + Romanian track A
 - Rodica Potolea
 - Professor, Computer Science Department
 - Room C09 ☹ hope soon (Teams & Moodle for now)
 - Rodica.Potolea@cs.utcluj.ro
- Romanian B track
 - Camelia Lemnaru
 - Assoc. Professor, Computer Science Department
 - Room M03 (Teams, Skype, etc.)
 - Camelia.Lemnaru@cs.utcluj.ro

Structure of the course

- **Lectures (MS Teams + Moodle)**

- <https://moodle.cs.utcluj.ro/course/view.php?id=373>
- Slides + code + analysis + discussions
- Open course with Q&A sessions
- Stop us and ask questions whenever you have
- **If you have a question, most probably other students have the same question!**

- **Seminars (MS Teams + Moodle)**

- Problem solving – analysis and design, evaluation, comparisons
- Almost always asked: WHY? WHAT IF? With INTERPRETATION (answer WITHOUT running your code).

- **Labs (MS Teams + Moodle)**

- same content, every group a different faculty member, or a (graduate) PhD student or master student
- Each group – separate MS Teams channel + Moodle
- HANDS ON Problem solving (again: for ANY solution: WHY? WHAT IF?)
- Prolog

Lab sessions info

Group	Enrollment key (moodle)	TA
30231	2021_Group@30231	<i>Raluca Portase</i>
30232	2021_Group@30232	<i>Mihai Feier</i>
30233	2021_Group@30233	<i>Robert Vacareanu</i>
30234	2021_Group@30234	<i>Andrei Rad</i>
30235	2021_Group@30235	<i>Ramona Tolas</i>
30236	2021_Group@30236	<i>Roxana Pop</i>
30237	2021_Group@30237	<i>Paul Helmer</i>
30238	2021_Group@30238	<i>Andrei Moldovan</i>
30239	2021_Group@30239	<i>Richard Ardelean</i>
302310	2021_Group@302310	<i>Eliza Olariu</i>
30431	2021_Group@30431	<i>Roxana Pop</i>
30432	2021_Group@30432	<i>Anda Stoica</i>
30433	2021_Group@30433	<i>Anda Stoica</i>
30434	2021_Group@30434	<i>Vasile Suciu</i>

References

- **No bible this time**
- **Instead, several references**
 - **Check the Moodle course page**

Evaluation (FG formula)

- **Course quizzes (20% in the FG = Final Grade)**
 - Between 3 and 7 quizzes, during the course, **un-announced**
 - Target info in the current lecture and ALL the things discussed up to that point
 - Delivered on Moodle
 - CANNOT retake the quizzes
- **Hands on evaluation (laboratory work: 30% in the FG)**
 - Stay in your group
 - 12 lab works, must work during the lab sessions, must upload work and discuss it with the TA
 - Activity during the semester will be part of your lab grade!
 - Final evaluation, at the end of the semester (written + oral)
 - Need a lab grade of 5 or more to be allowed to take the final exam
- **FE (50% in the FG)**
 - 2h examination (Moodle): problem solving only (NO THEORY)
 - This is for FG up to 8
 - Oral examination just for FG > 8
 - Based on prior enrollment
 - Permission granted based on lab and FE performance (TBD)

FG formula

- **$FG = 20\% QAG + 30\% LG + 50\% FEG + \text{Bonus}$**
- **Meaning:**
 - **FG=Final Grade**
 - **QAG=Quizzes Average Grade**
 - **LG=Lab Grade**
 - **FEG=Final Exam Grade**
- **Conditions to pass:**
 - **Preconditions:**
 - **$LG \geq 5$**
 - **$FEG \geq 5$**
 - **$\text{Averaged}(QAG + FEG) \geq 5$**
 - **$FG \geq 5$ (applicable only AFTER preconditions met)**
- **Bonus (Eligible based on Seminar Activity & Lectures Interaction with meaning)**
 - **Apply a +[0.5 to 1] point**

What this course is about?

- **Course on Logic Programming**
 - Define the LP **syntax** and **semantics**
 - Understand the **execution mechanism**
 - Learn the main **elements of the LP paradigm**
 - *Language as executable specifications*
 - Practice in a **LP language** (Prolog and “dialects”)
- **General CS practice**
 - Basic **problem solving**
 - *Revise* **Data Structures and Algorithms** from the LP perspective

In a nutshell...

- Programming so far:
 - Imperative (procedural languages)
 - assumes the presence of an assignment ($:=$) operator
 - specific control structures used
 - presence of side effects
- Declarative languages (symbolic languages)
 - Functional and *logic* languages
 - assignment operation not present
 - control removed from programmer (*partially*)
 - no side effects (*ideally*)

In a nutshell...

- Program = Algorithm + **Data Structures**
 - Wirth (book title)
- Algorithm = ***Logic*** + **Control**
 - Kowalsky's statement

LP philosophy

- Program:
 - In conventional programming languages is describing the algorithm using the language's syntax
 - In LP is describing formal relationships occurring among objects (here objects is NOT the OO meaning)
 - Focus on the descriptive perspective rather than the prescriptive one
 - A LP program = collection of known facts and relationships between components, rather than a sequence of steps to be taken
 - Computation is carried out by the specification (with logic embedded in the declarative semantic + new facts inferred) and just *partially* by explicit control information supplied by the program
- LP main characteristics:
 - Logic variables
 - Shared structures
 - Unification mechanism (matching)
 - Search for alternative solutions (when writing nondeterministic solutions)

LP syntax

- We'll use PROLOG to generically denote Logic Languages
 - PROLOG = PROgramming LOGic
 - For a brief history check references
- [*almost*] NO keywords (just a very few; we'll learn them on the fly)
- NO [*predefined*] statements
- The program is just what you write
- Case sensitive language
 - **Variables** – UPPER FIRST case (Variable, VARIABLE, VaRiAbLe are all variables; variable is constant); also starting with _ (_var is a variable)
 - **Constants** – lower first case (constant, cONSTANT, cOnStAnT are all constants; Constant is variable)
- **Matching** basic operation (evaluation rare, just by **explicit request**)
 - Ex: **A=a** means A unifies (matches) with a and NOT that A takes the value of a
- **Unification** = the attempt to make the two sides the same
 - Successful matching (TRUE) – they are the SAME from now on
 - Unsuccessful matching (FALSE or "fail" in Prolog jargon) – backtracking mechanism initiated (see later)

LP syntax

- Program = collection of predicates
- Predicate = a description (in Prolog syntax) of theorems
 - a collection of clauses
 - name of a predicate = a constant (starts with lower case)
- The general form of a clause is represented by:

$p(X) : -q_1(Y), q_2(X, Y), \dots, q_n(X, Z) .$

$p, q_i, i=1\dots n$ represent names of predicates (first letter lower case)

X, Y, Z arguments (variables = start with upper case)

X = parameter

Y, Z = local variables (hidden variables; explained later)

$p(X)$ = head of the clause

$q_i, i=1\dots n$ = body of the clause (sub-query)

$:-$ = if

$,$ = and (conjunction)

$.$ = full stop (end of clause). Dot is mandatory.

Prolog PREDICATES (clauses)

- The general form of a clause is represented by:

$$p(X) : -q_1(Y), q_2(X, Y), \dots, q_n(X, Z).$$

Left hand side (lhs) if right hand side (rhs) .

head:-body.

- Clause
 - a **theorem** = a representation in Prolog syntax of the relationships (body) among the objects (variables) occurring in the clause; it has a standalone meaning
 - Can be viewed as a procedure (function, method) from imperative (procedural, OO) languages
 - Meaning: the head is true (p; lhs) if the body is true (q_i, i=1...n; rhs)
 - The conclusion of the theorem (head, lhs) is true if all the hypotheses of the theorem (rhs) are true
 - p = head
 - q_i, i=1...n = body
- Predicate = a set of one or several clauses
- Prolog program = a set of one or several predicates

Prolog PREDICATES

(rules, facts, queries)

- The general form of a clause is represented by:

$$p(X) : -q_1(Y), q_2(X, Y), \dots, q_n(X, Z).$$

- Particular form of clauses:
 - General form – above (**rule**)
 - Particular forms: **facts** and **queries**

- **Fact**

= a clause without body

= a theorem WITHOUT hypothesis = AXIOM (no need for proof)

$p(a)$. // meaning *p of a is true, without condition.*

- **Query**

= a clause without head

= a theorem WITHOUT conclusion = only hypotheses which need proof = if true, will be a new axiom

?- $q_2(a, b)$. // meaning *check if q2 of a and b is true.*

This defines the **declarative semantics** of Prolog = interpretation of the statements

The first LP program

```
man(john) .
```

```
•
```

```
•
```

```
•
```

```
parent(john,david) .
```

```
parent(john,edward) .
```

```
•
```

```
•
```

```
•
```

```
father(X,Y) :-
```

```
    man(X) ,
```

```
    parent(X,Y) .
```

```
?-father(john,Z) .
```

The first LP program

```
man(john) .           //john, lower case start is a man.
.
.
.
parent(john,david) .  //john is david's parent
parent(john,edward) . //john is edward's parent
.                     //could be the other way around
.                     // david is john's parent
.                     //is up to us; depends on how WE define it!
father(X,Y) :-        //X is Y's father IF (by :- sign)
    man(X) ,          //X is man AND (by ,)
    parent(X,Y) .     //X is Y's parent. That's all.
?-father(john,Z) .    // whose father is john? Return me the result in Z.
```

Execution of the first LP program: Questions to answer

- How it works...
 - **Matching** mechanism
 - Match the query against the head of the clause of the predicate with the same name (`father` in our case) = **query/head unification**
- Some questions need to be answered:
 - **Q1: what if query/head unification have several alternatives** (translates into: predicate has several clauses)
 - **Q2: what if query/head unification succeeds and we have some body** (translates into: we have a complete clause)
 - **Q3: what if query/head unification fails**
- The answers define the **procedural semantics** of PROLOG = interpreting the statements as actions to take (call and run) (also named **operational semantics**)

Execution of the first LP program answers to questions

- **Q1:** what if query/head **unification** have several alternatives (translates into: predicate has several clauses)
 - A1: first cause first = **top-down**
- **Q2:** what if query/head unification succeeds and we have some body (translates into: we have a complete clause)
 - A2: first subgoal (sub-body) first = **left-right**
- **Q3:** what if query/head unification fails
 - A3: unification fails = **backtracking**
 - Backtracking is a built-in mechanism = it is automatically launched in case of failure

Computer Science

LP operational semantics

- Defined by the answers from before:
 - **Selection** (computation) rule: the top/down approach (A1)
 - **Search** rule: the left/right approach (A2)
 - **Backtracking** by default

Execution of the first LP program

```
m1 man(john) .
.
.
.
p1 parent(john,david) .
p2 parent(john,edward) .
.
.
.
f1 father(X,Y) :-
    man(X),           f11
    parent(X,Y) .     f12
q ?-father(john,Z) .
```

A1: **(1) q/f1** father(john,Z) / father(X,Y) => X=john, Z=Y

It results the CONCLUSION could be true in case all conjunction in the body is true as well the body of the clause becomes the NEW goal (or query) to execute.

So, the NEW goal is an instance of f11,f12, that is:

f11,f12 = man(X) , parent(X,Y) .

Where X and Y got instantiated already (X and Y are shared variable; shared as they are THE SAME as in the head of the rule)

A2 tells us f11 gets executed before f12; moreover, f12 is executed **iff** f11 is TRUE.

Execution of the first LP program – contd.

f11, f12 = `man(X), parent(X, Y)` .

A2 tells us to SOLVE f11 and PUT on hold f12 (what happens: the entire query is put on stack with f12 bottom and f11 top) and the execution pops the top of the stack

f11 = `man(X)` .

Is the new query to execute => Q1 => A1 =>

(2) f12/m1 `man(X) /man(john)` => X=john succeeds .

Is a fact => unconditionally true => f11 IS TRUE.

HOW DO WE CONTINUE? We have 3 conditions to check (for a final successful unification):

- **C1: current goal succeeds?**
- **C2: is a fact?**
- **C3: is the execution stack empty?**

Continuation depends on how the conditions are (not) met.

(the execution stack contains the goals to execute)

Execution mechanism – conditions to check

- C1: current goal succeeds? If yes, go C2, else, backtrack.
- C2: is it a fact? If yes, go C3, else, go take and execute the entire body.
- C3: is the stack empty? If yes, over (the whole execution ends successfully), else pop the top of the execution stack and continue (from C1)
- Backtrack means WHAT?
 - UNDO the latest unification (that is REMOVE latest matching) and
 - for the given goal try a different alternative (next one, if any)
 - If again fail (or no other alternative), the backtracking mechanism propagates in REVERSE order of the execution.

Computer Science

Execution of the first LP program – contd.

f11, f12 = `man (X) , parent (X, Y) .`

Solved f11 so?

- C1: current goal succeeds?
in the execution tree YES => unification succeeds => successful node
- C2: is a fact? YES => node is a leaf in the tree
- C3: is the stack empty?
sibling of the latest node NO => pop the top and it becomes the right

In our case, pop -> **f12** = `parent (X, Y) .`

Is the new query to execute => Q1 => A1 =>

(3) f12/p1 `parent (X, Y) / parent (john, david) .`

BUT, from the previous query we had => X=john, and X is shared (among the body and head), thus here we actually have:

john=john (T) and Y=david (T)

So, successful matching. Following the deduction tree, we have the whole unification chain.

Here goes the deduction tree

1 $2/f_1 : \{ X_1 = \text{john} \checkmark \}$
 $Y_1 = Z \checkmark$
 $?- \text{man}(\text{john}), \text{parent}(\text{john}, Z)$
 2 $f_{1.1}/m_1 : \{ \text{john} = \text{john} \checkmark \}$
 3 $f_{1.2}/p_1 : \{ \text{john} = \text{john} \checkmark \}$
 $\text{david} = Z \checkmark$
 $?- \text{yes}, Z = \text{david.} ; (n)$
 $f_{1.2}/p_2 : \{ \text{john} = \text{john} \checkmark \}$
 $\text{edward} = Z \checkmark$
 $?- \text{yes}, Z = \text{edward.} ; i$

```
m1 man(john) .
.
p1 parent(john,david) .
p2 parent(john,edward) .
.
f1 father(X,Y) :-
    f11 man(X) ,
    f12 parent(X,Y) .
?-father(john,Z) . . . .
```

Diagram showing a deduction tree with nodes $2/f_1$, $f_{1.1}/m_1$, and $f_{1.2}/p_1$. A red circle around $2/f_1$ is labeled "fail". A red circle around $f_{1.1}/m_1$ is labeled "fail". A red circle around $f_{1.2}/p_1$ is labeled "fail".

1 $2/f_1$ fail
 2 $f_{1.1}/m_1$ fail
 3 $f_{1.2}/p_2$ fail

Execution of the first LP program – contd.

f12 = parent (X, Y) .

Solved f11 so?

- C1: current goal succeeds? YES => unification succeeds => successful node in the execution tree
- C2: is a fact? YES => node is a leaf in the tree
- C3: is the stack empty? YES => the whole execution ends successfully

Where is the answer?

So, successful matching. Following the deduction tree we have the whole unification chain

Let's follow the execution tree:

Deduction Tree

- The final successful shape of an execution tree forms the **deduction** tree
- It stores the query/head matchings along with the unifications (formal vs actual arguments) made during the execution
- **Nondeterministic** behavior of Prolog = ability to find alternative solutions (if any) of the problem at hand
- It is performed via launching the **backtracking** mechanism (built-in)
- it refers to **repeating** the query; repeating the query is NOT asking the SAME query (if so, the SAME answer is provided)
- repeating the query is: for the SAME query AND in the context of the ALREADY built deduction tree, what OTHER possible answers are there?
- Syntactically, it is ":" (at command prompt) (or ";", "n" depending on the language), and it says: in the tree you already have, take the nodes in reverse order and try to find an alternative matching
- So, the LAST built node attempts to backtrack and only after it has no other choice it asks its predecessor node to backtrack.

Backtracking

- Evolution backwards in a tree already built
- The last built, the first to backtrack
- We had: (1)

(1) q/f1

(2) f12/m1

(3) f12/p1 backtrack on this => ERASE THE NODE and try **(3') f12/p2**

`parent(X,Y) / parent(john,edward) .`

BUT, from the previous query (2) we had => `X=john`, and `X` is shared (among the body and head), thus here we actually have:

`john=john (T) and Y=edward (T)`

So, successful matching.

Following the new deduction tree we have the whole unification chain:

Backtracking – the tree

```
m1 man(john) .  
.  
p1 parent(john,david) .  
p2 parent(john,edward) .  
.  
f1 father(X,Y) :-  
    f11      man(X) ,  
    f12      parent(X,Y) .  
q ?-father(john,Z) .
```

On slide 26...

Computer Science

Backtracking until final failure

- If we need ANOTHER answer, the evolution backwards in the new shape of the tree (1, 2 and 3')
- The last built, the first to backtrack
- We had:

(1) q/f1

(2) f12/m1

(3') f12/p2 backtrack on this => ERASE THE NODE and try **(3'') f12/p3**

Assume p3 has as first argument something different from j_{ohn}

(3'') fails to build, (3'') is erased and the backtracking propagates on to 2:

(2') f12/m2 fails (as m2 has something different from j_{ohn} , and 2 constants unify iff they represent the same constant)

(2') fails to build, (2') is erased and the backtracking propagates on to 1:

(1') q/? No other clause exists => FINAL FAILURE

Meaning: NO other solution exists

Action: the whole tree is removed from memory (*go to the prev page to see*)

Execution mechanism – review

- Rule 1: **Search** rule:
the left to right evaluation of goals in conjunction
- Rule 2: **Selection** (computation) rule:
the top down selection of clauses from the program
- Rule 3: **Backtracking** by default:
(see later mechanisms for avoiding it)

Computer Science

List representation and decomposing lists

Lists: $[1,2,3]$ – comma separated elements

- NO type required (heterogeneous lists by default)
- $[a,b,c]$ how is it different from $[A,B,C]$?
- $[]$ empty list
- $[H|T]$ Head/Tail decomposition pattern with meaning:
 - H = the first element of the list
 - T = the rest of the list (= the list with the first element removed)
- $[H|T] = [a,b,c]$ unification/matching
 - Before, H, T free variables
 - Now, bound to specific constants: $H=a, T=[b,c]$
- $[H|T] = [a]$
 - $H=a, T=[]$
- $[H|T] = []$
 - Fails, as H CANNOT unify the first element of the list (as there is NO element at all)
- In **LP basic operator is matching**
(as opposed to FP where evaluator is the basic operation)

Example 2 – concatenating lists

```
append(L1, L2, L3) :-
```

```
    L1 = [],           //L2 empty
```

```
    L3 = L2.           //L3 gets L2
```

```
append(L1, L2, L3) :-
```

```
    L1 = [H1 | T],      //decomposition pattern; dec L1 into head and tail
```

```
    append(T, L2, R),    //recurse on tail to get the partial result
```

```
    H3 = H1,
```

```
    L3 = [H3 | R].       //recomposition pattern; rec L3 from head and tail
```

Example 2 contd

```
append(L1, L2, L3) :-
```

```
    L1 = [],           //L2 empty
```

```
    L3 = L2.           //L3 gets L2
```

```
append(L1, L2, L3) :-
```

```
    L1 = [H1 | T],      //decomposition pattern; dec L1 into head and tail
```

```
    append(T, L2, R),    //recurse on tail to get the partial result
```

```
    H3 = H1,
```

```
    L3 = [H3 | R].       //recomposition pattern; rec L3 from head and tail
```

Here we have explicit unifications.

By replacing them with default unifications, we get:

```
append([], L, L).
```

```
append([H | T], L, [H | R]) :-
```

```
    append(T, L, R).
```

Example 2 execution

a1: append([], L, L) .

a2: append([H|T], L, [H|R]) :-

a21: append(T, L, R) .

q: ?-append([1, 2], [3, 4], Result) .

Here goes equations and tree.

(1) **q/a1** [1, 2] = [] fails, default backtracking

(1') **q/a2** [H1|T1] = [1, 2] $\Rightarrow$ H1=1, T1=[2]

L1=[3, 4]

Result=[H1|R1] $\Rightarrow$ Result=[1|R1]

a21: append(T1, L1, R1) (why?) and translates into

append([2], [3, 4], R1)

(2) **a21/a1** fails

(2') **a21/a2** [H2|T2] = [2] $\Rightarrow$ H2=2, T2=[]

L2=[3, 4]

R1=[H2|R2] $\Rightarrow$ R1=[2|R2]

a21: append(T2, L2, R2)

append([], [3, 4], R2)

(3) **a21/a1** [] = [] L3=[3, 4] R2=L3

Result=[1|R1]=[1|[2|R2]]=[1, 2|R2]=[1, 2|L3]=[1, 2|[3, 4]]=[1, 2, 3, 4]

Lists patterns rephrasing

$[H1, H2|T] = [H1|[H2|T]]$ just rewriting

First conclusions:

- a Prolog **program** is a set of **theorems** (complete clauses) and axioms (facts)
- Executing a Prolog program means **proving a new theorem** (the query/goal) from the existing ones (program)
- The execution relies on **solving** a set of **systems of linear equations**
- The number of systems = number of nodes in the deduction tree
- Each system has a number of equations equal to the number of arguments of the goal executed in the corresponding node

append/3 (Seminar B)

a1 append(A,B,Res):- A=[H|T], a11
append(T,B,RI), a12
Res=[H|RI]. a13

a2 append([],L,L).

q ?- append([1,2], [a,b,c], R).

1: q/a1:

A1 = [1,2]

B1 = [a,b,c]

Res1 = R

?- [1,2] = [H2|T2],

2: a11(=):

[1,2] = [H2|T2] => H2=1, T2=[2]

?- ... append([2], [a,b,c], RI3), ...

3: a12/a1:

A3 = [2]

B3 = [a,b,c]

Res3 = RI3

?- [2] = [H4|T4], ...

4: a11(=):

[2] = [H4|T4] => H4=2, T4=[]

?- ..., append([], [a,b,c], RI5), ...

5: a12/a1 – will fail!

a12/a2:

[] = []

L5 = [a,b,c]

L5 = RI5 => RI5=[a,b,c]

?- ..., ..., Res3=[H4|RI5].

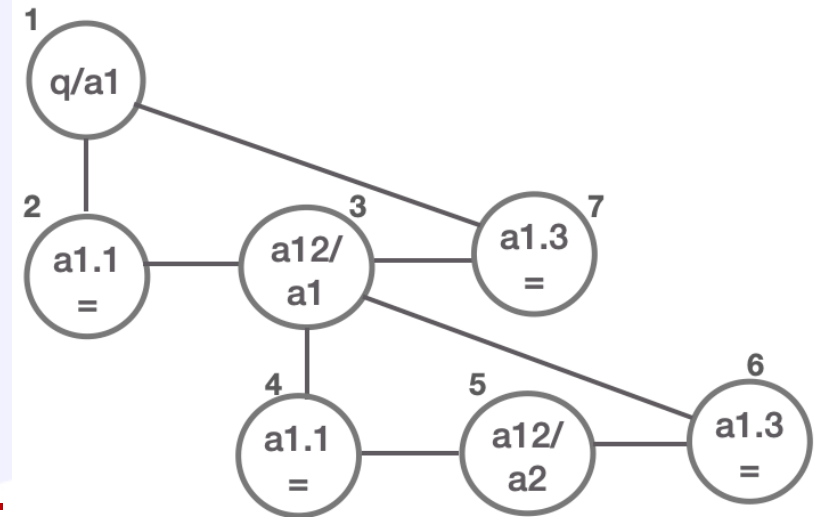
6: a13 (=)

Res3=[H4|RI5]=[2,a,b,c]

?- ..., ..., Res1=[H2|RI3].

7: a13 (=)

Res1=[H2|RI3]=[1,2,a,b,c]



Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #2, 2021
Cluj-Napoca

Computer Science

Agenda

- **LP paradigms – review**
- **Operational Semantic - review**
- **Execution tree**
 - **Representation**
 - **Mechanism**

Prolog PREDICATES

(rules, facts, queries)

- **Clause** general form:

$$p(X) : -q_1(Y), q_2(X, Y), \dots, q_n(X, Z) .$$

= a **TEOREM** in form "conclusion if hypotheses"

meaning $q_1 \wedge q_2 \wedge \dots \wedge q_n \rightarrow p$

Horn clause with at most one negated literal

$$q_1 \wedge q_2 \wedge \dots \wedge q_n \wedge \neg p$$

- **Fact**

$$p(a) .$$

= a clause without body

= a theorem WITHOUT any hypothesis = **AXIOM** (no need for proof)

- **Query**

$$?- q_2(a, b) .$$

= a clause without head

= a theorem WITHOUT conclusion

They define the **declarative semantics** of Prolog

= interpretation of the statements

Execution of LP programs

Unification mechanism – the core of LP

- **Q1:** For the query/head unification take
 - A1: first clause first = **top-down**
- **Q2:** In the query/head successful unification the body becomes the new goal. In a conjunction of goals
 - A2: first subgoal (sub-body) first = **left-right**
- **Q3:** In the query/head failed unification
 - A3: unification fails = **backtracking**

Operational semantics in action

When building the tree, we start from the initial query in the matching process. For each (sub)goal (one at a time):

- C1: current goal succeeds? If *yes*, current node successful built & go C2, *else*, backtrack.
- C2: does it match a fact? If *yes*, current node is a leaf & go C3, *else*, go take and execute the entire body as goal (push it all on the execution stack, and pop stack's top).
- C3: is the stack empty? If *yes*, over (the whole execution ends successfully, the execution tree becomes at this very moment the deduction tree), *else* pop the top of the execution stack & go C1

First conclusions

- a Prolog **program** is a set of **theorems** (complete clauses) and **axioms** (facts)
- Executing a Prolog program means **proving a new theorem** (the query/goal) from the existing ones (program)
- The execution relies on **solving** a set of **systems of linear equations**
- The number of systems = number of nodes in the deduction tree
- Each system has a number of equations equal to the number of arguments of the goal executed in the corresponding node

Main elements of Prolog

Built-in predicates = ready to use

var(X) if X unbound then T else F

nonvar(X) if X bound then T else F

atom(X) if X constant then T else F

integer(X) self-explanatory

atomic(X) if X atom or int then T else F

call(X) executes X, where X is the name of a predicate

used in metaprogramming, where X gets instantiated at runtime

Main elements of Prolog – contd.

- =** infix operator (*equality*)
Prolog attempts to match (unify) the lhs with rhs
if successful, they are bound together from this point on
an un-instantiated var will become equal to ANYTHING as the unification succeeds
- ==** infix operator (*identity*)
if $X == Y$ then $X = Y$
if $X = Y$ then $X == Y$ NOT MANDATORY!!!!
an uninstantiated var will become identical to another uninstantiated variable ONLY if they are already sharing same location, otherwise FAILS

Examples

?-X==Y no (Fails; even if both X and Y are free variables)

?-X==X yes, X=_some_number

?-X=Y, X==Y yes, X=_some_number; Y=_some_number (SAME)

?-X=[a,b], Y=[a,b], X==Y

no (although same list [a,b] is JUST same content, they are NOT shared in memory, NOT same location)

?-X=[a,b], Y=X, X==Y

yes

is

infix operator

rhs gets evaluated and the result instantiates the lhs (if successful) or else fails. ONLY lhs may get instantiated

X **is** some_expression

X is 2+3

yes, X=5

X=2+3 **yes, X=2+3**

2+3 **is** X no

Matching rules

Goal(q)	Rule head (r)	Outcome
a	b	fails
a	a	succeeds
X	a	succeeds, and X gets instantiated to a
a	X	succeeds, and X gets instantiated to a
Q	R	succeeds, and Q and R become the same

Execution tree representation

Tree built top-down and left-right (DFS)

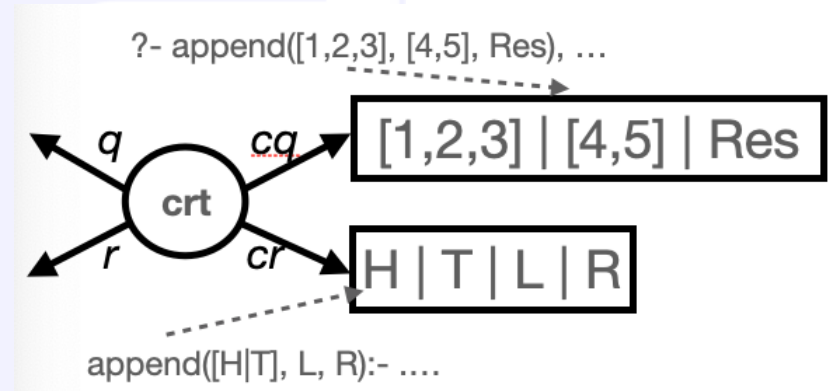
Is a multiway tree (represented as binary)

- Each node in the exe tree would contain **pointers** to several structures
 - Data: Program and memory locations of the variables **(1)**
 - Linking: connections in the tree (to navigate during execution, including backtracking) **(2)**
- Several **actions** to build the tree **(3)**

Computer Science

Execution tree: pointers to program and variables (1)

- A set of **4 pointers** to the:
query (goal) **q**
head of the rule **r**
context of the query (goal) **cq**
context of the rule **cr**



- The **query context** inherited from the parent node = rule context of the parent node becomes query context of the child node (it may be enhanced by the query context of a brother to the left due to hidden variables, or enhanced with new variables here)
- The **rule context** is allocated at the level of the current node = ALL formal variables are indexed with the number of the node (to keep track of the variable occurrence)

Execution tree: Linking pointers (2)

- A set of **5 pointers** to other nodes in the tree (they provide the ability to make the sound actions according to the operational semantics):

parent rule

(parent node; for root, null)

previous query

(sibling node to the left; for first child, null)

next rule

(first child; if any; for leaves, null)

next query

(sibling node to the right; for last child, null)

next try

(the next alternative to try in case of failure/backtracking; points to the rightmost child, to provide fast reachability to the last node in the tree)

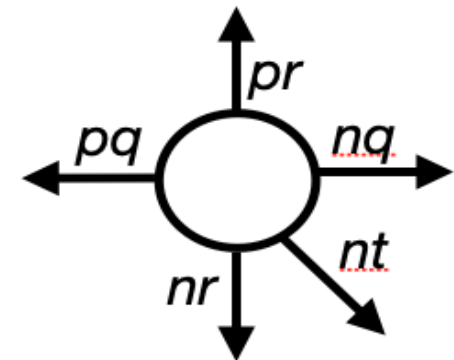
pr

pq

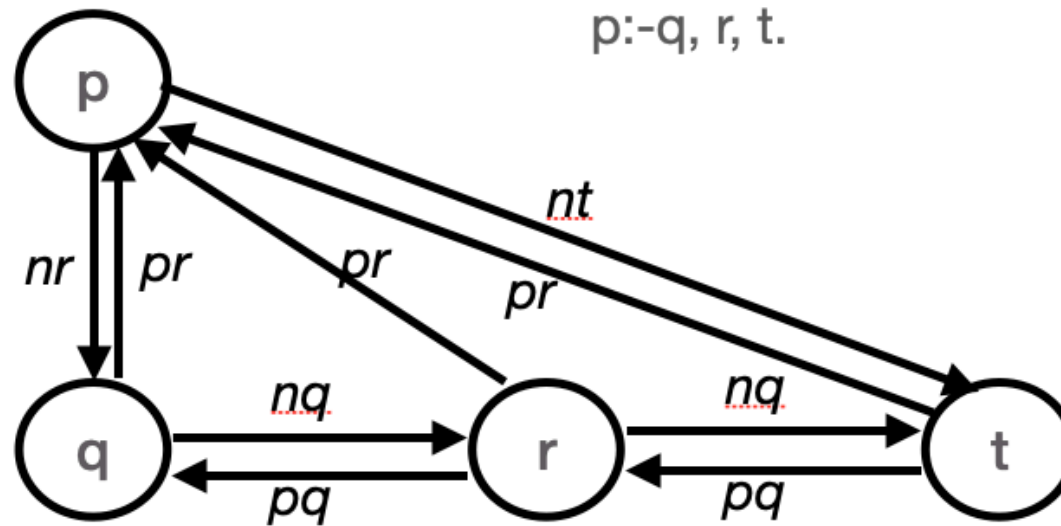
nr

nq

nt

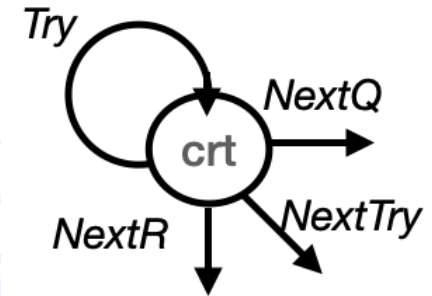


Execution tree: Linking pointers (2)



Computer Science

Execution tree: Actions (3)



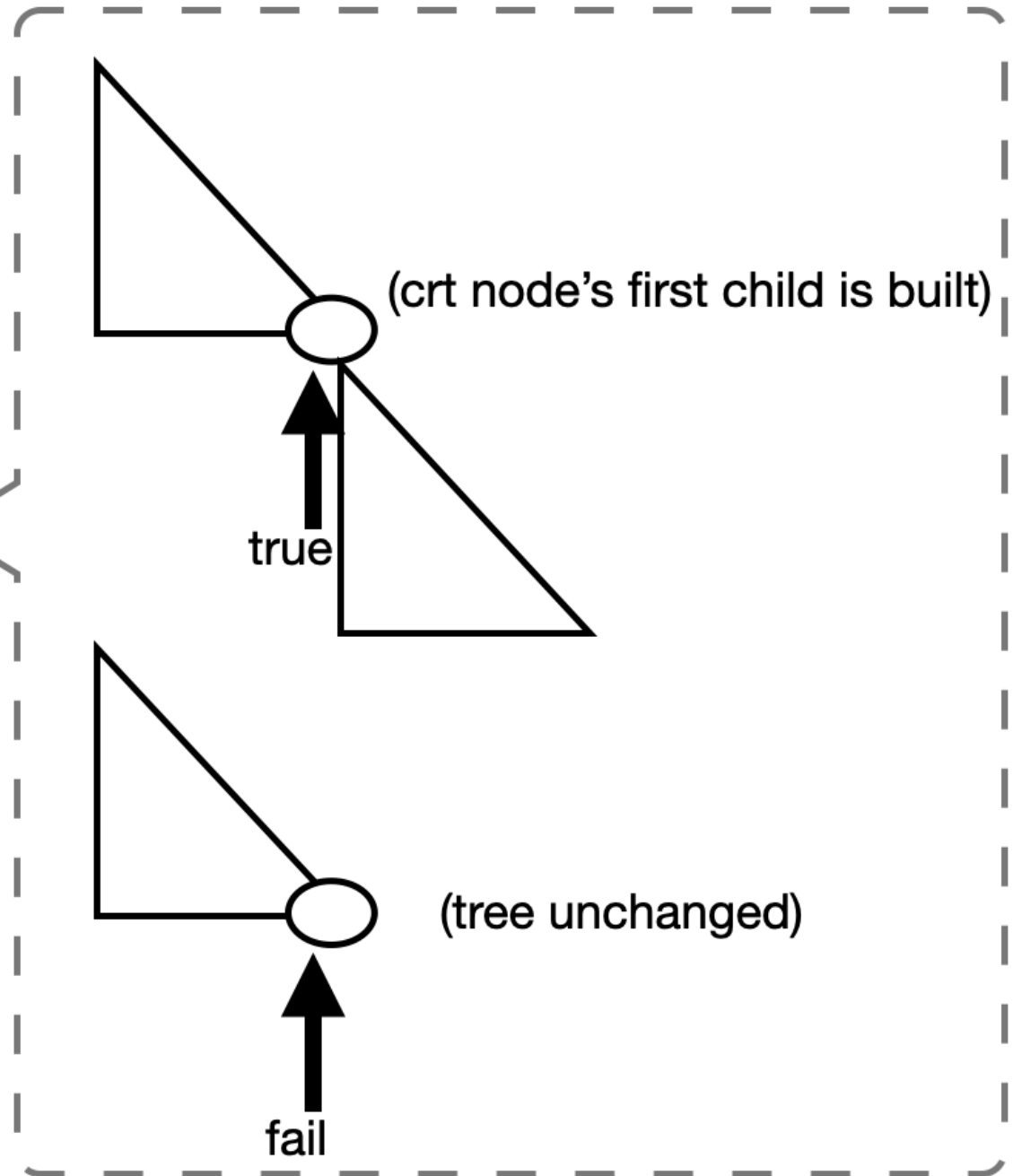
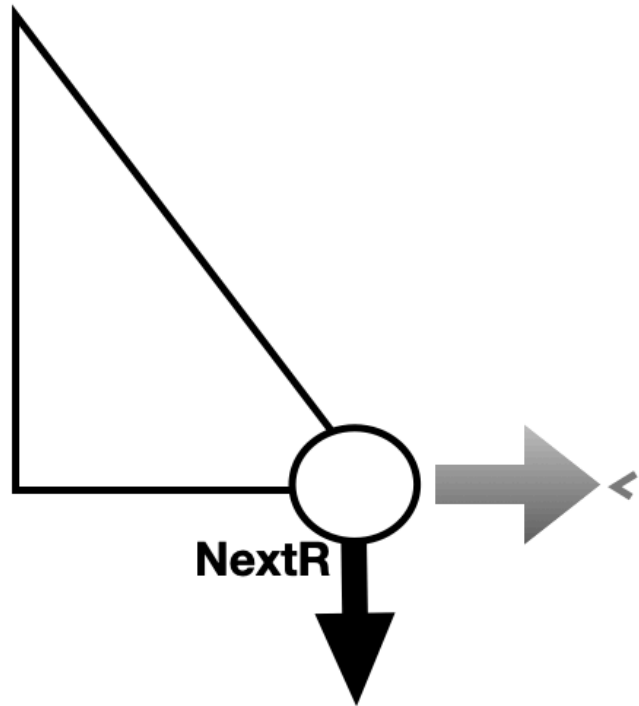
- **Try** - tries the definition of the predicate with the same name as the query (represents the matching between a query and the head of the rule)
the action BUILDS the **current node**
- **NextR**- enters the body of the clause at the first subgoal in the body, passing the conditional (:-)
the action BUILDS the **first child** of the current node
- **NextQ**- continues the body of the clause at the next subgoal in the body, passing a conjunction (,)
the action BUILDS the **first sibling to the right** of the current node
- **NextTry** – determines the first node to backtrack and launches the backtracking mechanism
the action takes place in a tree already built, by trying an alternative solution for the node identified as responsible to backtrack (examples during seminars)

Execution tree: Pointers and Actions – NextR

NextR

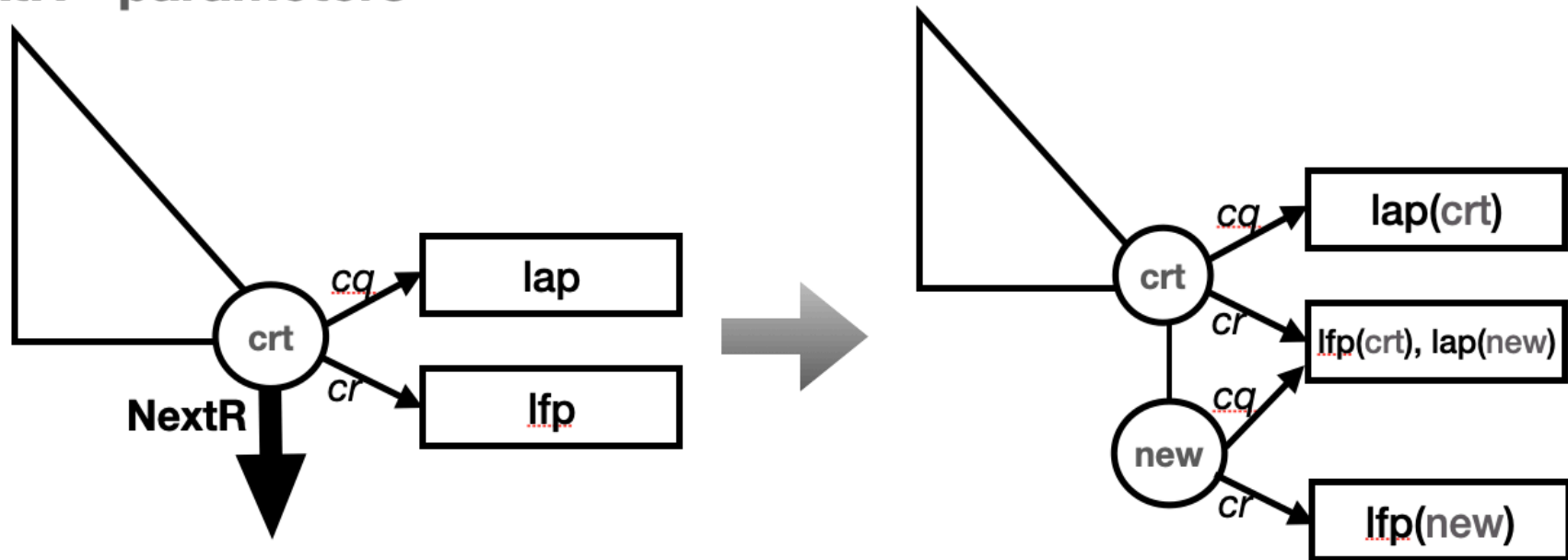
- At least one *new* node is generated, the left (first) child of the current node
- If it (*new* node; first child of current) matches a fact, it is a leaf
- If matches a complete rule, the entire tree rooted by it (*new* node; first child of current) is generated
- the list of **actual parameters** of the new node is **inherited** from the list of actual parameters of the current node
- The list of **formal parameters** of the new node is **allocated now** (thus, the corresponding variables are indexed with the number of the node)

NextR



NextR – how arguments evolve

NextR - parameters



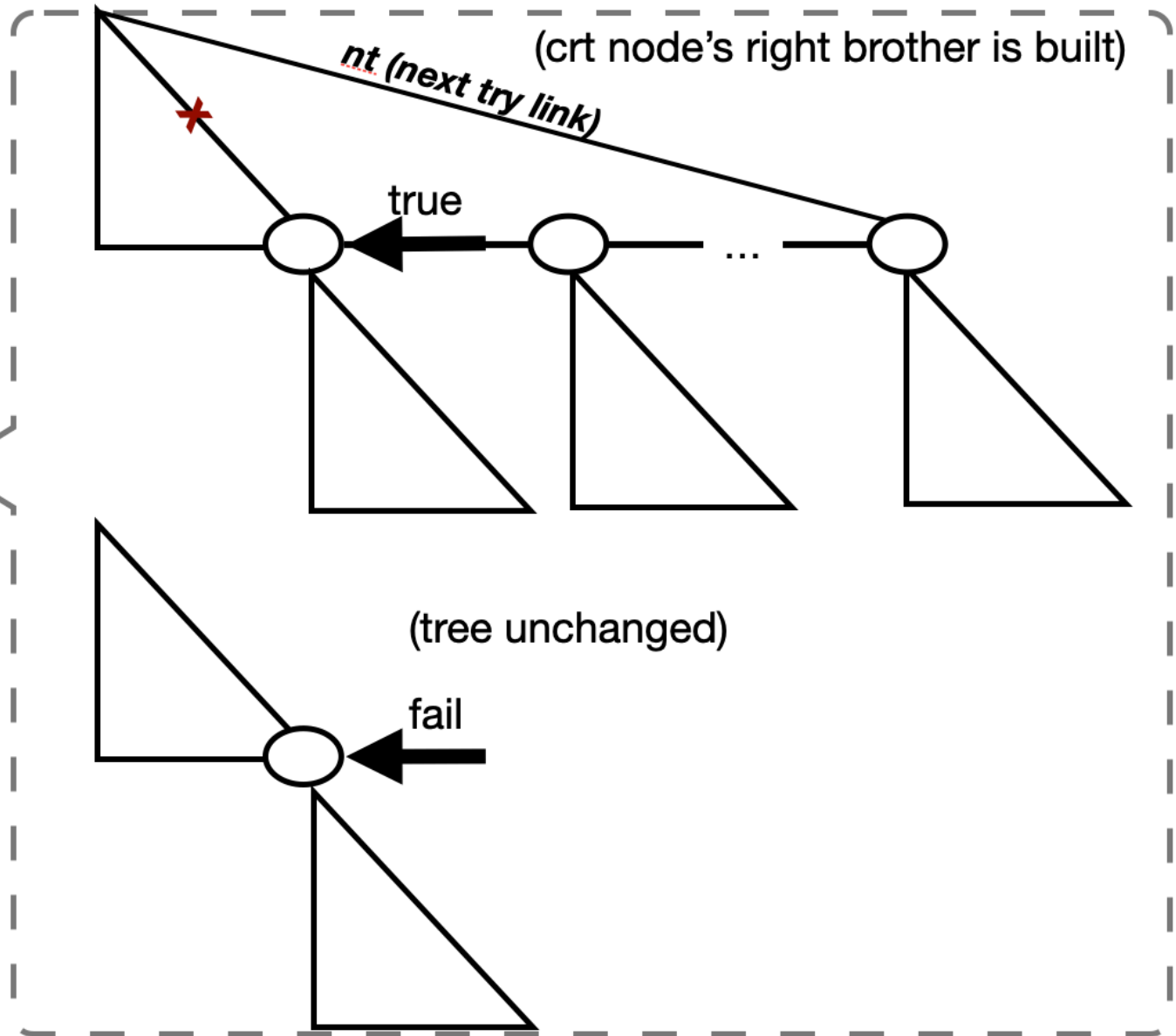
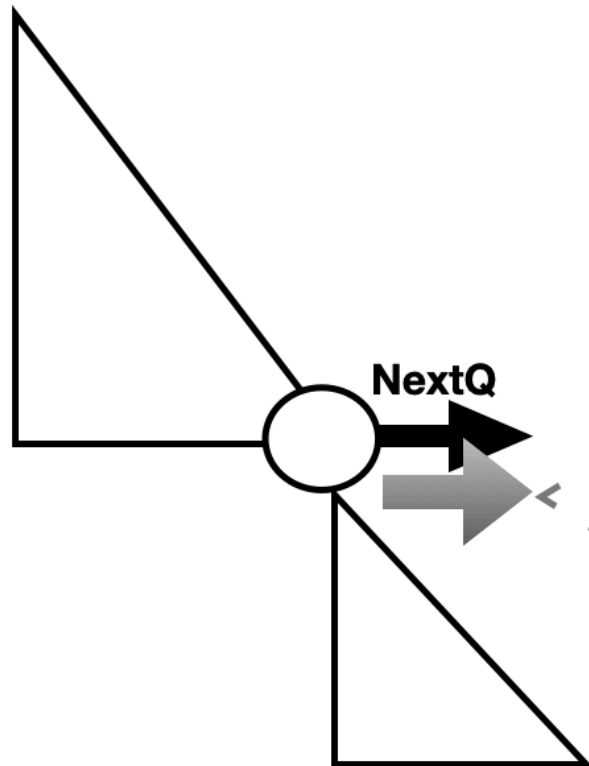
- the list of **actual parameters** of the new node is **inherited** from the list of **formal parameters** of the current node
- the list of **formal parameters** of the new node is **allocated** now (thus, the corresponding variables are indexed with the number of the node)

Execution tree: Pointers and Actions – NextQ

NextQ

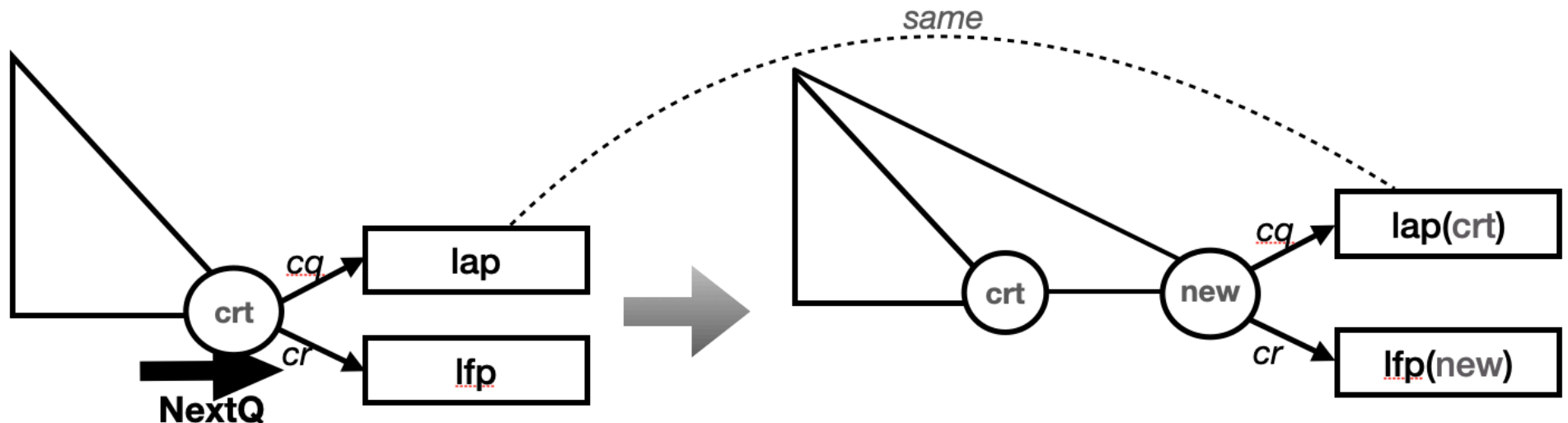
- At least one *new* node is generated, the right brother of the current node
- If it (*new* node; right brother of current) matches a fact, it is a leaf; else the entire tree rooted by is generated
- If it is NOT the rightmost sibling (NOT the last in conjunction, NOT before.), all its right siblings (with subtrees) are generated when completed
- the list of **actual parameters** of the new node is **inherited** from the list of actual parameters of the current node
- The list of **formal parameters** of the new node is **allocated now** (thus, the corresponding variables are indexed with the number of the node)

NextQ



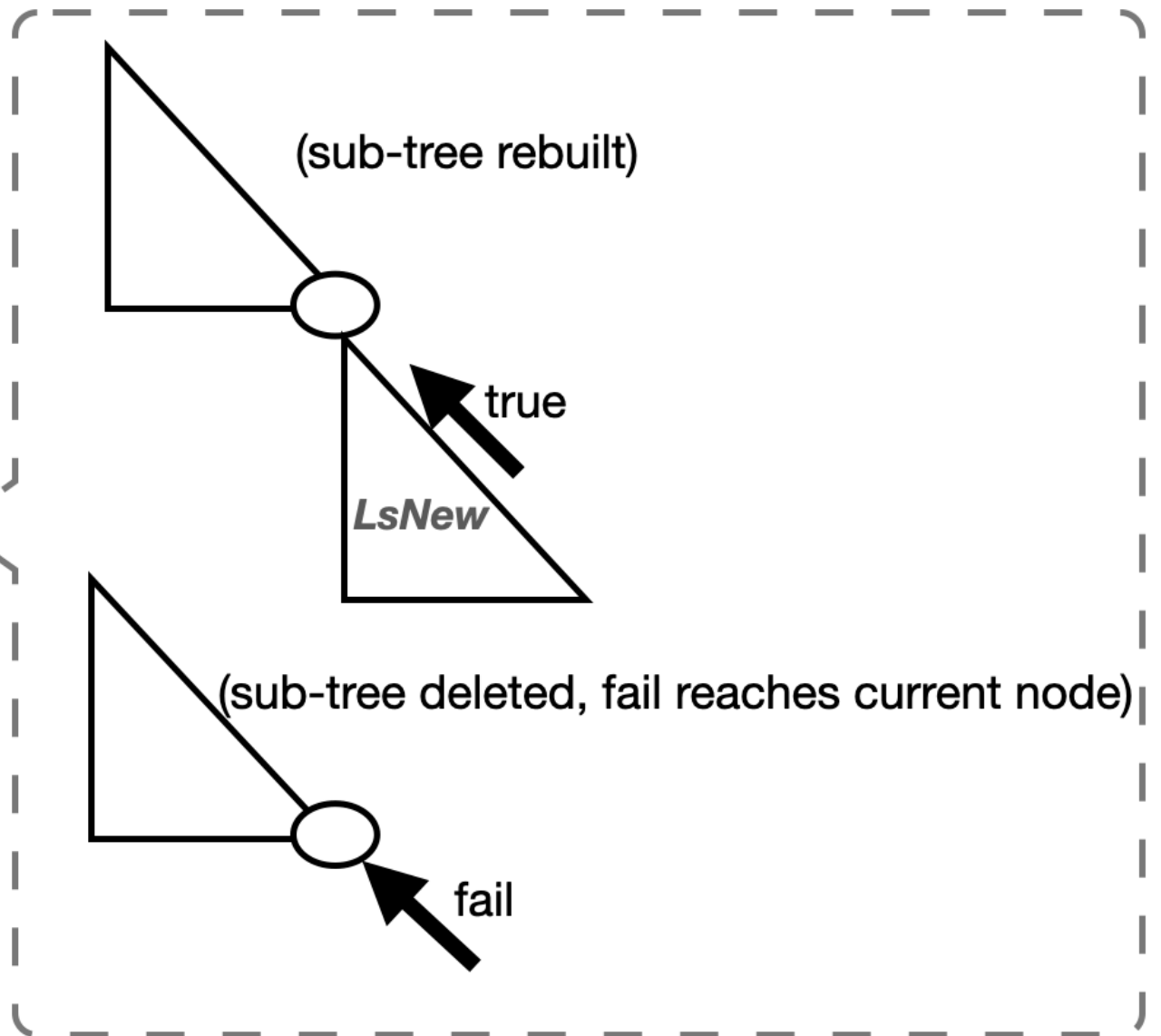
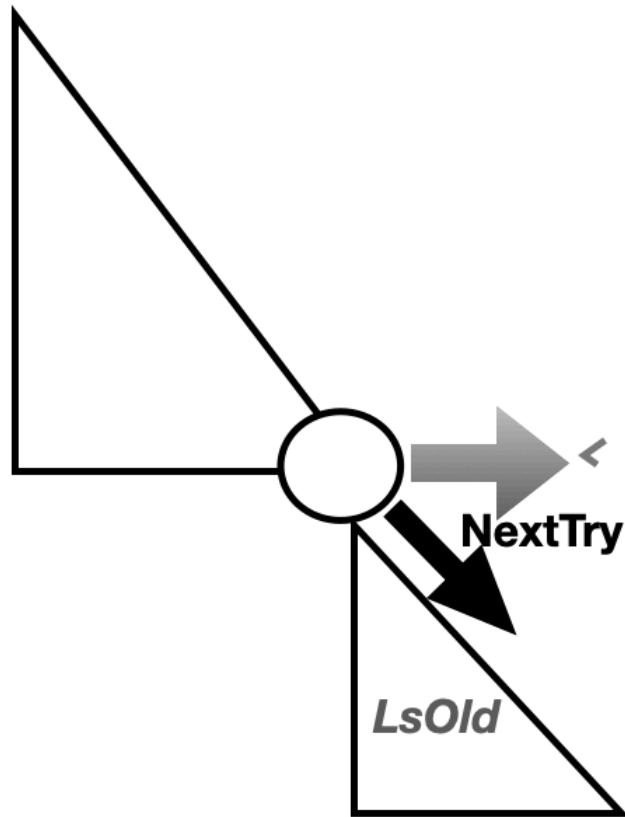
NextQ – how arguments evolve

NextQ - parameters



- the list of **actual parameters** of the new node is **inherited** from the list of **actual parameters** of the current node
- the list of **formal parameters** of the new node is **allocated** now (thus, the corresponding variables are indexed with the number of the node)

NextTry



Example 3rd

Check if an element is present in a list

How many arguments/why?

Predicate's signature is `member/2` (element, list)

```
member(X, L) :-  
    L = [H | T],  
    X = H.  
member(X, L) :-  
    L = [H | T],  
    X \= H,  
    member(X, T).  
member(X, L) :-  
    L = [],  
    fail.
```

Example 3rd contd.

```
member (X, L) :-  
    L = [H | T],           //list not empty  
    X = H.                 //searched element matches the head of the list  
  
member (X, L) :-  
    L = [H | T],           //list not empty  
    X \= H,                 //searched element does not match the head of the list  
    member (X, T).         // searched element found in the rest of the list  
  
member (X, L) :-  
    L = [],                //if reached the empty list  
    fail.                  // searched element not found in the list
```

With default unifications, the predicate becomes:

```
member (H, [H | T]) .      //default decomposition and unification  
member (X, [H | T]) :-    //default decomposition  
    X \= H,                //Is it necessary? Why/why not?  
    member (X, T).  
  
member (X, []) :-  
    fail.                  //Is it necessary? Why/why not?
```

Example 3rd contd.

```
member (H, [H | T]) . //default decomposition and unification
member (X, [H | T]) :- //default decomposition
    X \= H, //Is it necessary?
    member (X, T) . //not really due to the search rule (A1)
member (X, []) :- //Is it necessary?
    fail. //not really due to the default failure by absence.
```

Thus, the predicate becomes:

```
member (H, [H | T]) .
member (X, [H | T]) :-
    member (X, T) .
```

Qs:

- What happens if we reverse the order of clauses? Would it still be a correct predicate? Why/why not?
- How is better? Why?
- Trace a deterministic query
- Trace a nondeterministic query

Example 3rd contd.

```
member(H, [H|T]).  
member(X, [H|T]) :-  
    member(X, T).
```

```
member(H, [H|_]).  
member(X, [_|T]) :-  
    member(X, T).
```

Avoiding Variable used only once warning
_ = ANY variable (does NOT produce unification)

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #3, 2021
Cluj-Napoca

Computer Science

Agenda

- **Example 4 – length of a list**
 - **Introducing recursion types**
 - **Types of parameters**
- **Example 5 – reversing a list**
 - **Introducing pattern position**
 - **Meaning of arguments and measures**
- **Backtracking prevention – the CUT (!)**
- **Negation (as failure)**

Example 4 – how to recurse?

- Determine the length of a list
- Number of arguments?

```
list_length/2 (list, length)
```

```
list_length([], 0).           //length of empty list is 0
```

```
list_length([_|T], N):-  
    list_length(T, N1),       //length of tail calculated as N1  
    N is N1+1.               //add 1 to the length of tail
```

- Order of clauses? This way? Swapped? Which is correct? Which is better? Why? Is it important? Why?

(discuss indexation on the first argument. Oral explanation.)

- Meaning of partial result (N1) not quite relevant (is length of the tail)
- What if we need to keep the number of “consumed elements”?

List length – backward recursion

- Previous solution used ***backward*** recursion, meaning that the **result is built as recursion RETURNS**
- What if we build it while advancing?

```
list_length2/2 (list, length)
```

```
list_length2([],R).           // R has the final list length
```

```
list_length2([_|T],PR):-
```

```
    NPR is PR+1,              //add 1 to the partial result so far  
                                //i.e. length of the covered part
```

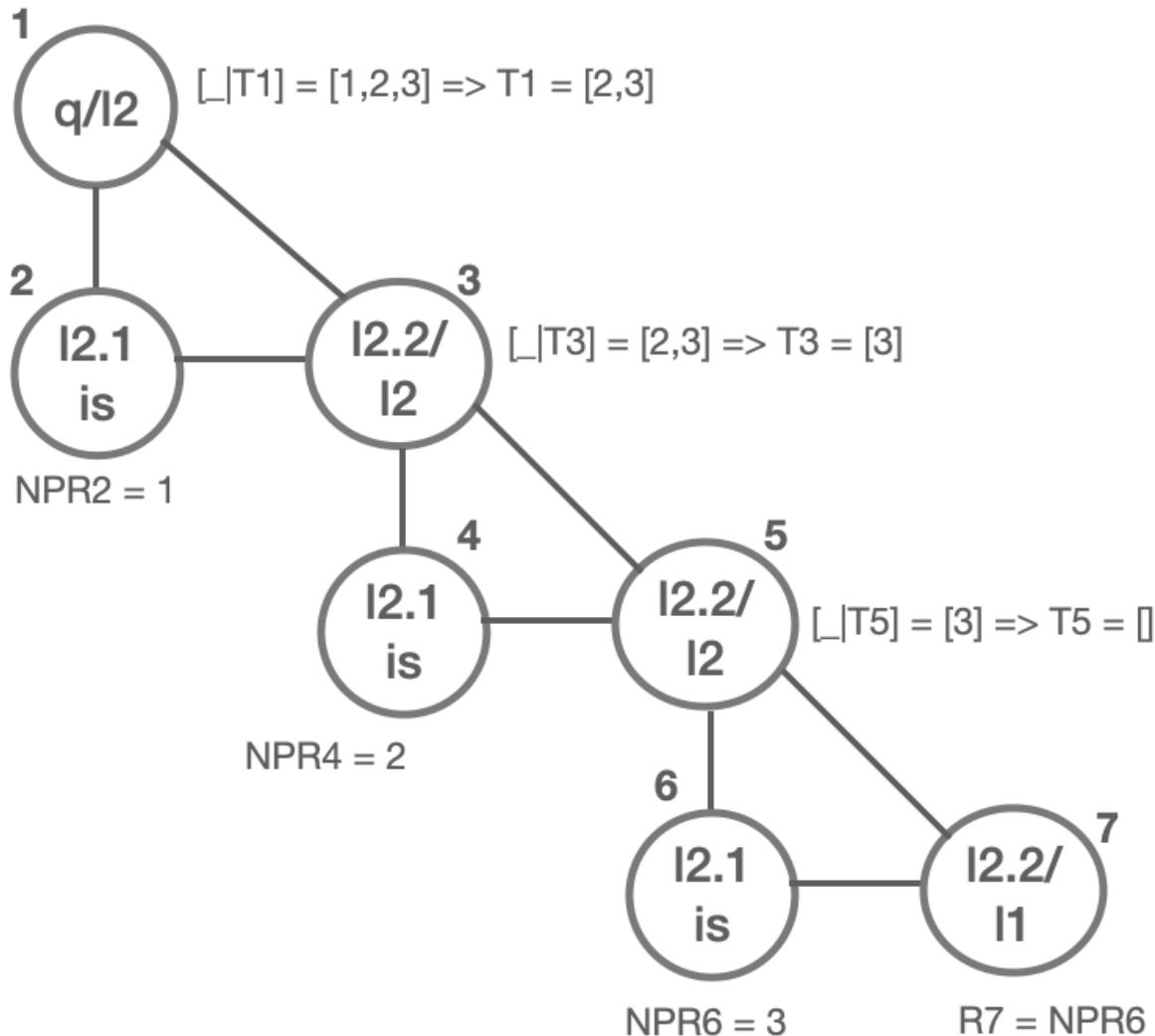
```
    list_length1(T,NPR).      //go count the rest of the elements
```

- what's the difference compared to the previous solution?
- N represents here what we *have covered*; before – what we've *yet to cover*.

list_length1 deduction tree

```

11 list_length1([],R).
12 list_length1([_|T],PR):-
    NPR is PR+1, 12.1
    list_length1(T,NPR). 12.2
q ?- list_length1([1,2,3], 0).
  
```



List length – forward recursion

```
list_length2/2 (list, length)
```

- Not enough – as result, although calculated, is NEVER reported

```
list_length2([],N).
```

```
list_length2([_|T],N):-
```

```
    N is N1+1,
```

```
    list_length2(T,N1).
```

- Upgrade it with 3rd argument = “steal” the result from the rightmost leaf

```
list_length3([],PR,FR):-FR=PR.
```

```
list_length3([_|T],PR,FR):-
```

```
    NPR is PR+1,
```

```
    list_length3(T,NPR,FR).
```

List length – forward recursion

- With **default** unification:

```
list_length3([], Rez, Rez) .  
list_length3([_ | T], PR, FR) :-  
    NPR is PR+1,  
    list_length3(T, NPR, FR) .
```

- How should we call it? Needs specific INITIALIZATION:

```
?- list_length3(InList, 0, Result) .
```

- To avoid mandatory initial call, use a wrapper:

```
list_length3(InList, Result) :-  
    list_length3(InList, 0, Result) .
```

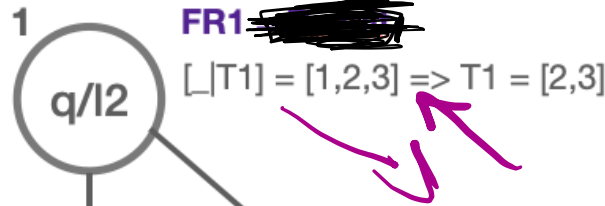
- Can have same name? why/why not?
- Discuss signature definition

list_length3 deduction tree

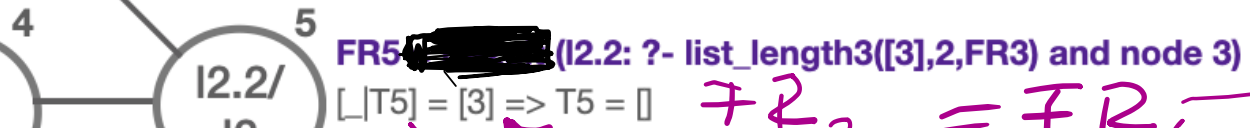
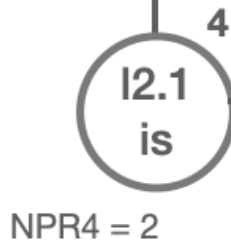
```

11 list_length3([], Rez, Rez).
12 list_length3([_|T], PR, FR) :-
    NPR is PR+1,      12.1
    list_length3(T, NPR, FR). 12.2
q ?- list_length3(InList, 0, Rezult).
  
```

Rezult = FR1 (1)

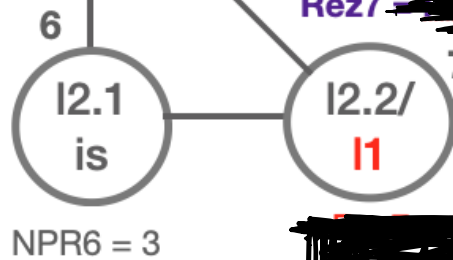


FR1 = FR3 (3)



FR3 = FR5 (5)

~~Rez7~~ (l2.2: ?- list_length3([],3,FR5) and node 5)



FR5 = Rez7 (7)

Rez7 = NPR6 = 3

Forward vs Backward recursion

Forward recursion

Backward recursion

+

- Potentially useful partial results
- Easy to parallelize

- No need for extra variable
- No need for specialized initial call

-

- Need for extra variable (final result)
- Need for specialized call (for accumulator initialization)

- Rarely useful intermediate results
- No improvement for parallel executions (execution of concurrent process(es) - delayed until recursion returns)

Computer Science

Reversing a list

- How many arguments?

- 2 = list + reversed list

```
reverse1/2 (list, reversed_list)
```

```
reverse1([], []). //empty input returns empty output
```

```
reverse1([H|T], R) :-
```

```
    reverse1(T, PR), //reverses the tail of list
```

```
    append(PR, [H], R). //concatenates the partial results
```

- Needs specialized call? Why/why not?
- What type of recursion is here?
 - *Backward*
- Efficiency? How is it calculated?
- Can we avoid 2 traversals through the list? How?

Reversing a list – forward recursion

- How many arguments?

list + partial_rev_list + reversed list

```
reverse2/3(list, part_rev_list, rev_list)
```

```
reverse2([], PR, PR). //when input gets empty the partial  
//result becomes the final one
```

```
reverse2([H|T], PR, R) :-
```

```
    NPR=[H|PR], //concatenates the partial results
```

```
    reverse2(T, NPR, R). //reverses the tail of list
```

- Wrapper

```
reverse(In, Out) :-
```

```
    reverse2(In, [], Out).
```

- Efficiency? Is it better?

Reversing a list – forward recursion

```
reverse2 ([], PR, PR) .  
reverse2 ([H|T], PR, R) :-  
    reverse2 (T, [H|PR], R) .
```

- Same as before just default unification what was explicit
- Let's change the order of arguments 2 and 3 (makes no difference in the functionality)

```
reverse2 ([], PR, PR) .  
reverse2 ([H|T], R, PR) :-  
    reverse2 (T, R, [H|PR]) .
```

- Just variables renaming:

```
reverse2 ([], L, L) .  
reverse2 ([H|T], L, R) :-  
    reverse2 (T, L, [H|R]) .
```

Comparative analysis

<pre>reverse([], L, L) . reverse([H T], L, R) :- reverse(T, L, [H R]) .</pre>	<pre>append([], L, L) . append([H T], L, [H R]) :- append(T, L, R) .</pre>
---	--

- What is the difference?
- Meaning of pattern's position
- Length of arguments analysis

Comparative analysis

Length of arguments analysis

```
reverse([], L, L) .           append([], L, L) .
reverse([H|T], L, R) :-      append([H|T], L, [H|R]) :-
    reverse(T, L, [H|R]) .    append(T, L, R) .
```

length_arg1+length_arg3=length_arg2 length_arg1+length_arg2=length_arg3

Init call:

$n+0=n$

At step i:

$(n-i)+i=n$

$(n-i-1)+(i+1)=n$ in the head
in the body

At the end:

$0+n=n$

Init call:

$n1+n2=n1+n2$

At step i:

$(n1-i)+n2=(n1-i)+n2$

$(n1-i-1)+n2=(n1-i-1)+n2$ in the head
in the body

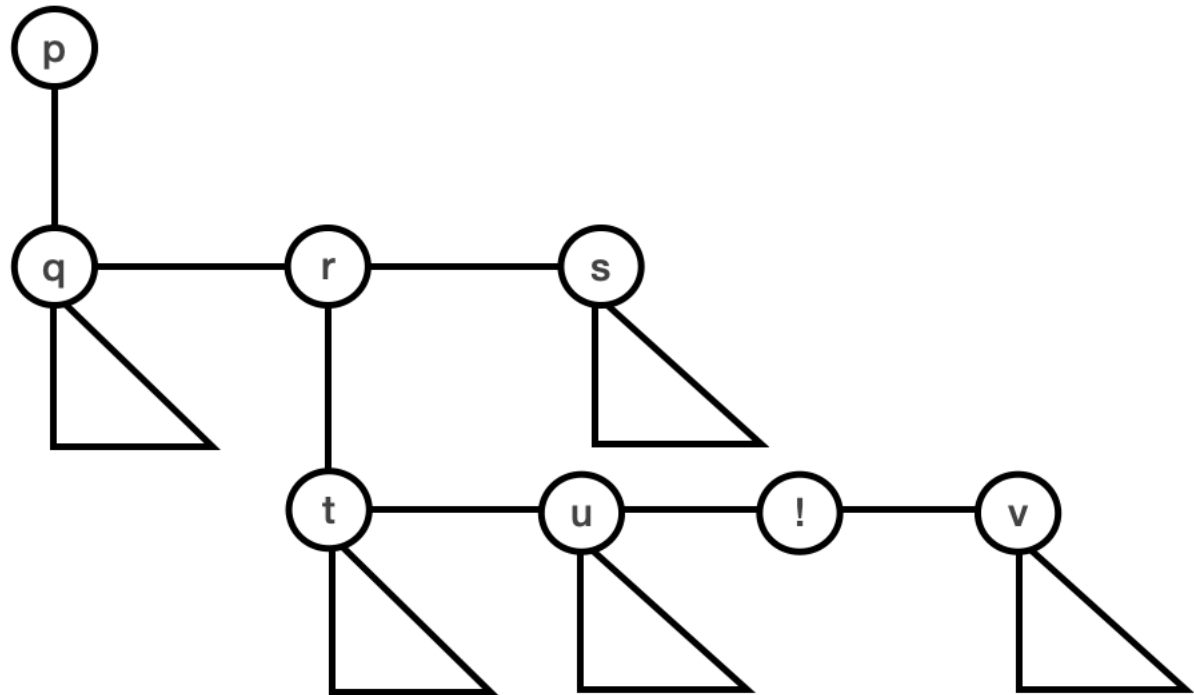
At the end:

$0+n2=n2$

Backtracking avoidance (The CUT!)

- Built-in predicate
- Always succeeds
- Never backtracks

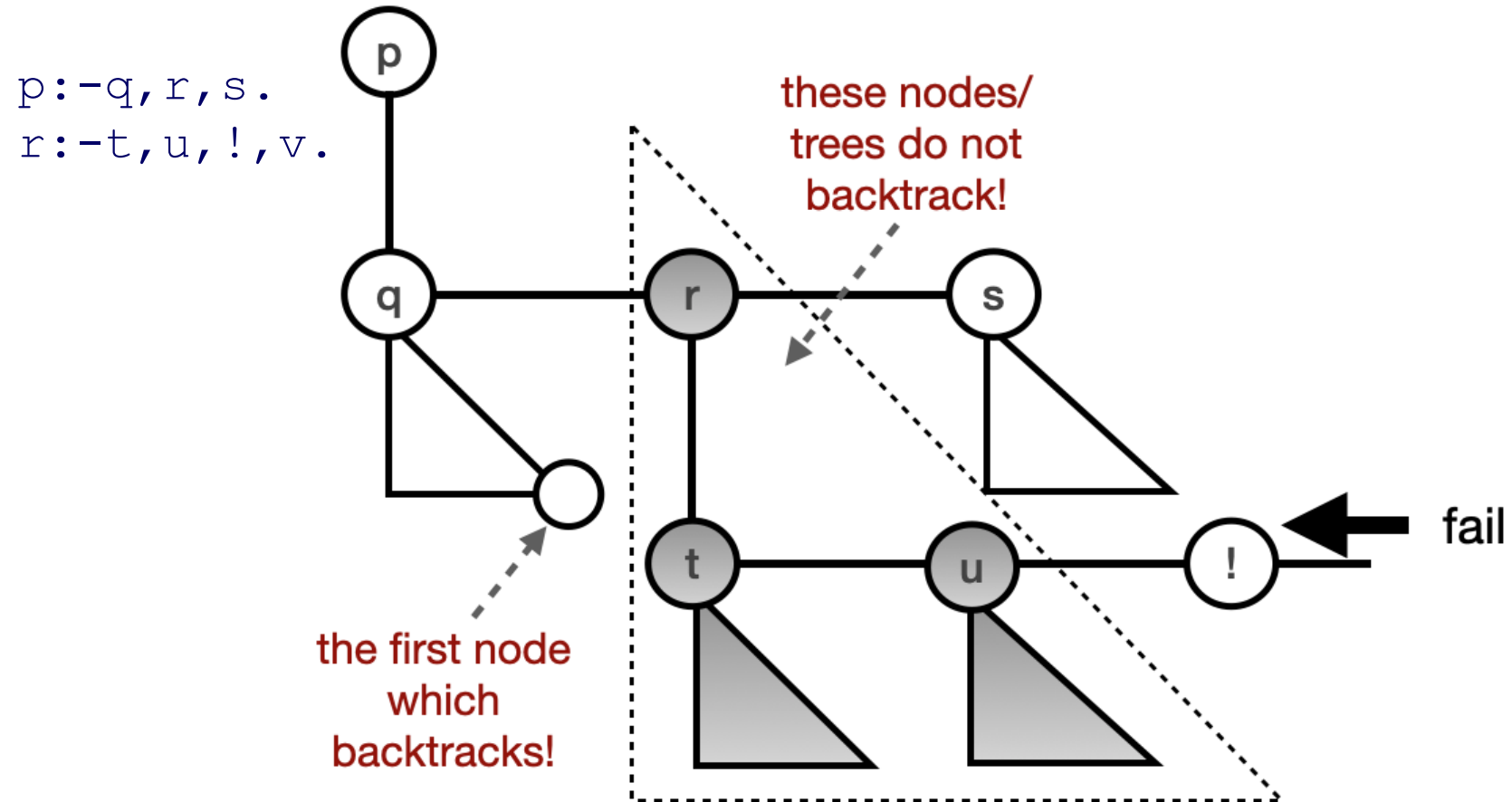
$p:-q, r, s.$
 $r:-t, u, !, v.$



The CUT (!) – effects

- ! Prevents from backtracking the following categories:
 - All left siblings
 - All subtrees of left sibling nodes
 - Parent node
- NO other node is affected:
 - Siblings on the right/their subtrees have the ability to backtrack
 - Left siblings DO backtrack BEFORE ! Is executed
 - Their subtrees also
- As everything, takes effect ONLY AFTER its (!) execution (before execution it does NOT actually exist)

The CUT (!)



The CUT (!)

(when getting to fail, node s and the tree rooted by it was also REMOVED)

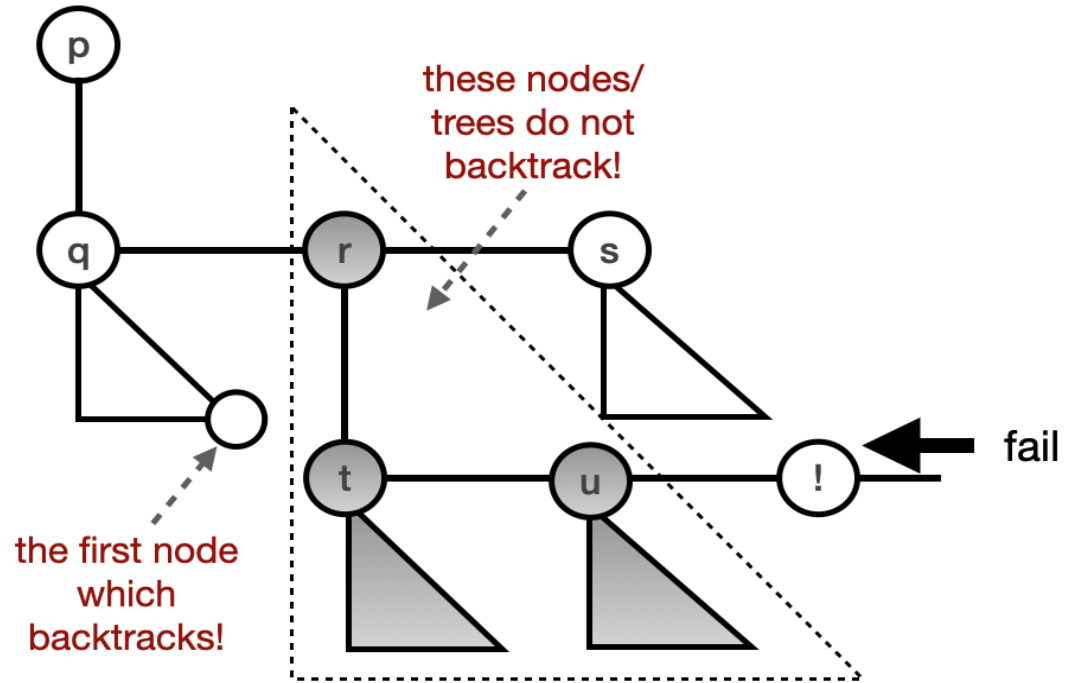
```
p:- q, r, s.
...
r:- t, u, !, v.
r:- something.
r:- something_else.
```

Presence of ! is as if we have:

```
p:-q, r, s.
r:-t, u, v.
r:- not(t, u),
    something.
r:- not(t, u),
    something_else.
```

...
JUST that:
- It executes FASTER

=> As if the clauses below the ! are removed for the current execution



Computer Science

Negation (as failure)

- Unfair behavior
- As it just assumes the inability to prove T means F
- Isn't it so? Why? Postponed discussion.

`not (P) :-`

`P, !, fail.`

`not (P) .`

- $P - \text{true} \Rightarrow \text{not}(P) - \text{false}$; OK.
- $P - \text{false} \Rightarrow \text{not}(P) - \text{true}$; OK
- $P - ?$ Don't know how it is. Why? Incomplete information, not known at the time.

How is `not(P)`? Let's run it:

P's execution fails (P is NOT yet T),
we enter the second clause
succeeds.

Negation (as failure)

`not (P) :-`

`P, !, fail.`

`not (P) .`

- Negation assumes negated is T just because we don't have information about P.
- Negation as failure (to check validity)

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #4, 2021

Cluj-Napoca

Agenda

- Forward vs backward recursion
 - sum
 - generics
 - Comparative analysis, Pros and cons
- append3
 - Forms
 - Efficiency analysis and justification
 - Nondeterministic call
- Delete from a list
 - One, just one, all, repeating the query

Forward vs backward recursion

- Forward vs backward - alternative approaches to handle data
- Data is split into partition
- Forward = processing takes place when the current item is first encountered, and the rest of the data (the other components than the item) are processed AFTER the current item is processed.
- Backward = starts by processing all but current item (the other components than the item) via recursive call(s) while the current item is processed just AFTER we return from recursion(s). NOTE: is returned from recursive call (and **NOT from backtracking!!!**).
- In case of lists [H|T]:
 - *forward* handles H first and next T (via recursive call),
 - *backward* starts by solving the problem on T (via recursive call), and just when returning from the call H is processed.

Forward sum

```
% sum_2/3
```

```
%sum_2(in_list, final_result, accumulator_part_result,).
```

```
sum_2([],PartialSum,PartialSum).    //final result, arg 2, copies the value of the partial result, arg3  
                                     //with default unification
```

```
sum_2([H|T],Sum,PartialSum):-
```

```
    NewPartialSum is PartialSum + H, //do process the current item
```

```
    sum_2(T,Sum,NewPartialSum).    //go ahead with the reminding structure
```

- List decomposed into [H|T]
 - Starts by processing (addition here) the current item (H here)
 - Continue with processing the rest of the structure (one recursive call here, as partition is H and T)
 - This implies the items are added in the sum forward (starting from the first one).
- ->->->...-> processing is performed in the order of items in the structure
- The accumulated result represents the sum of values from the beginning to the current item

Backward sum

```
% sum_1/2
```

```
%sum_1(in_list,sum_of_els_in_arg_1).
```

```
sum_1([],0).                                //result gets initialized. Empty input, null output
```

```
sum_1([H|T],Sum):-
```

```
    sum_1(T,TailSum),                        //call first the processing on the rest of the partition
```

```
    Sum is TailSum + H.                      //do process the current item
```

- List decomposed into [H|T]
 - Starts with processing T, same predicate, recursive call
 - When returned, use the obtained result (TailSum) to evaluate the overall result on the whole list (Sum)
 - This implies the items are added in the sum backwards (starting from the last one).
- ---... --> first go this way (all way to the end) doing nothing (just call)
- <- the processing occurs after returning from the call, one at a time
- <- in the opposite order, *last* item *first* processed, before last item second processed, ...
- <- third, second and first items being last processed in this specific order (first is last)
- The accumulated result represents the sum of values from the current item to the end

Forward vs backward sum

- Forward solution needs specific initial call => write a special predicate (a wrapper) to make it in a sound way (avoid the need of user knowing how to initialize the accumulator parameter) :

```
run_sum_2(List,Sum):-                //same partial result as in case of backward
    sum_2(List,Sum,0).                //nothing yet processed, null result. Same as
                                     //in stop condition for bwd.
```

- If the input list is [1,2,3,4,5,6,7] what is going to be the partial result (`PartialSum`) and (`TailSum`) respectively on forward and backward solutions?
- Try to **estimate before running** them.
- Follow the textbook to update the predicate with the necessary lines to print the values.

Computer Science

Forward generic

```
%forward_recursion/3
%forward_recursion(input argument, final result, partial result)

forward_recursion([],PartialResult,PartialResult). //final result, arg 2, copies the
//value of the partial result, arg3, with default unification
forward_recursion([H|T],Result,PartialResult):- //partition data, here split into H and T
    do(NewPartialResult,H,PartialResult), //start by processing the current item
//and thus updating the previous PartialResult to NewPartialResult via processing do.
    forward_recursion(T,Result,NewPartialResult) //process the rest of the structure
//with recursive call

%forward_recursion_call/2
%forward_recursion_call(in, out)
forward_recursion_call(Input,Output):-
    forward_recursion(Input,Output,InitialValueOfResult) //make the initialization with
//a separate predicate (wrapper) to avoid mandatory user initialization
```

Backward generic

```
%backward_recursion/2
```

```
%backward_recursion(input argument, output result)
```

```
backward_recursion([],InitialValue).    //empty input, make initialization backwards
```

```
backward_recursion([H|T],PartialResult):-
```

```
    backward_recursion(T,NewPartialResult), // starts with processing the rest of  
    //the structure; all partition but the current item.
```

```
    do(PartialResult,H,NewPartialResult). //process the current item
```

- No need for a specific initial call, hence, no wrapper predicate.

Forward vs backward pros and cons

	Forward	Backward
+	• Process "as we go" => structure is processed from front to end => intermediate results could be useful (is the result of the structure "so far"). • In concurrent processing that result is made available and another process using it can start immediately • Last call optimization = reusing the same stack area without the need of restoring it back	• Needs no initialization=>Needs no specific call => needs no wrapper => needs no additional argument
-	• Needs specific initial call => always make a wrapper to initialize the accumulator • Needs additional argument (partial result)	• Intermediate results are seldom useful • The result of the structure is known just at the end of the processing, so, concurrency is postponed on sync

Concatenate 3 lists

- Use what you ***have*** vs use what you ***know***
- We have the concatenation of 2 lists
- Use it twice.

```
append([],List,List).
```

```
append([Head|Tail],List,[Head|Rest]):-
```

```
    append(Tail,List,Rest).
```

- To put together L1, L2 and L3 do the following
 - $(L1+L2) + L3$
- OR
- $L1 + (L2+L3)$

Concatenate 3 lists: Use what you have 1

- $(L1+L2) + L3$; say $|L1|=n1$, $|L2|=n2$, $|L3|=n3$,
`append([], List, List).` //order of classes does not matter
`append([Head|Tail], List, [Head|Rest]):-` //due to the indexation
`append(Tail, List, Rest).` //on the first argument
- **Efficiency: $O(n)$** where n the length of the first parameter
`append3_1(L1, L2, L3, Result):`
`append(L1, L2, Intermediate),` //link L2 at the end of L1
`append(Intermediate, L3, Result).` //link L3 at the end of the
//intermediate result created before.
- **Efficiency:**
 - $O(n1)$ for the first call (links $L2$ at the end of $L1$)
 - $O(n1+n2)$ for the second call, length $Intermediate$ of is length of $L1$ and $L2$ (links $L3$ at the end of $Intermediate$)
 - Overall: $t(n)=2n1+n2$

Concatenate 3 lists: Use what you have 2

- $L1 + (L2 + L3)$; say $|L1|=n1$, $|L2|=n2$, $|L3|=n3$,

append3_2(L1,L2,L3,Result):

```

    append(L2,L3,Intermediate),           //link L3 at the end of L2
    append(L1,Intermediate,Result).       //link result created before
                                           //Intermediate at the end of the first list L1

```

- Efficiency:
 - $O(n2)$ for the first call, decomposes $L2$ (links $L3$ at the end of $L2$)
 - $O(n1)$ for the second call, decomposes $L1$ (links $Intermediate$ at the end of $L1$)
 - Overall: $t(n)=n1+n2$
- Observations:
 - **Second version better regardless the input!**
 - Order of calls matters (again!)
 - How can we use this in a standalone predicate?

Concatenate 3 lists: Use what you know

- What we know? Concatenation via decomposition of the first argument! Use it!

`append3_3([Head|Tail],List2,List3,[Head|Rest]):- //as long as the first arg`

`append3_3(Tail,List2,,List3,Rest). // nonempty, decompose it`

`append3_3([],[Head|Tail],List,[Head|Rest]):-//once first argument empty`

`append3_3([],Tail,List,Rest).//you are back on 2 list concatenation`

`append3_3([],[],List,List).`

- **Observations:**
 - Clauses 2 and 3 are disjoint from clause 1 (indexation on the first argument would treat them separately, i.e. cl1 one entrance, cl 2 and 3 another one). Therefore, it does NOT matter where clause 1 is placed
 - On the other hand, clause 3 should come AFTER clause 2 (as indexation on the second argument is NOT available

Concatenate 3 lists: Use what you know – contd.

```
append3_3([Head|Tail],List2,List3,[Head|Rest]):-
```

```
    append3_3(Tail,List2,_,List3,Rest).    // decomposes first list
```

```
append3_3([],[Head|Tail],List,[Head|Rest]):-
```

```
    append3_3([],Tail,List,Rest).          // decomposes second list
```

```
append3_3([],[],List,List).
```

- **Observations:**

- How is done? (resembles in behavior to version1)
 - Decomposes L_1 to go through it by adding each of its items in result (behaves as version 1, as in “link L_2 at the end of L_1 ”)
 - Decomposes L_2 to go through it by adding each of its items in result (links L_3 at the end of L_1 concatenated to L_2).
- Efficiency (resembles in performance to version2) $t(n)=n_1+n_2$
 - $O(n_1)$ first clause
 - $O(n_2)$ second clause

Concatenate 3 lists: Use what you know – contd.

- Behavior: resembles to version1 $(L1+L2) + L3$
- Efficiency: resembles to version2 $t(n)=n1+n2$
- How is possible? Behavior V1 and performance V2?
- Explain!
- Explain which of the 3 version is best?
- Which to use and why?
- Which should never be used? Why?

Computer Science

Concatenate 3 lists: Nondeterministic call

- Given append3 predicate and the call: `?-append3(X,Y,Z,[1,2]).`
- What is the meaning of the call?
 - Nondeterministic call
 - What are the lists x, y and z whose concatenation form list $[1,2]$.
- What is/are the result/s?

X	Y	Z
[]	[]	[1,2]
[]	[1]	[2]
[]	[1,2]	[]
[1]	[]	[2]
[1]	[2]	[]
[1,2]	[]	[]

Other results? Why not?

- Which order/why?
 - Depends on the implementation.
 - Identify (BEFORE running) the order of results in each of the 3 implementations
 - For append3 with append, the order of clauses on append is MANDATORY (fact first), otherwise it enters infinite loop with 3 free variables on the first call.

Delete from a list

- Given a list, remove one item from it.
- How many arguments/why?

%delete/3

%delete(item to remove, input list, output list)

delete(**H**,[**H**|T],T). //if the item to remove is head of input, just don't put it on output

delete(X,[**H**|T],[**H**|R]):- //otherwise, keep it on output and

delete(X,T,R). //remove from tail

- What are the results on call when the item occurs several times? Try to estimate and justify BEFORE running.

q1?- delete(3,[1,3,2,3,4,3],Output).

- What happens in case the item is not present in the list?

q2?- delete(5,[1,3,2,3,4,3],Output).

Delete from a list – contd.

```
delete(H,[H|T],T).
```

```
delete(X,[H|T],[H|R]):-
```

```
    delete(X,T,R).
```

- What are the results on call? Why?

```
| ?- delete(1,[1,2,1,3,1],R).
```

```
R = [2,1,3,1] ? ;
```

```
R = [1,2,3,1] ? ;
```

```
R = [1,2,1,3] ? ;
```

```
no.
```

- What about the call? Why?

```
| ?- delete(4,[1,2,1,3,1],R).
```

```
no.
```

- So, the meaning of the predicate is: **delete exactly one occurrence** of an item from the list.

Delete from a list - contd

```
delete(H,[H|T],T).    //if the item to remove is head of input, just don't put it on output
delete(X,[H|T],[H|R]):-    //otherwise, keep it on output and
    delete(X,T,R).    //remove from tail
delete(_,[],[]).    //when the empty list is reached, whatever element is assumed to
                    //be deleted, done, result empty
```

- What are the results on call when the item occurs several times? Try to estimate and justify **BEFORE running**.
- What happens in case the item is not present in the list?

Computer Science

Delete from a list - contd

`delete(X,[X|T],T).`

`delete(X,[H|T],[H|R]):-`

`delete(X,T,R).`

`delete(_,[],[]).`

- What are the results on call?

`| ?- delete(1,[1,2,1,3,1],R).`

`R = [2,1,3,1] ? ;`

`R = [1,2,3,1] ? ;`

`R = [1,2,1,3] ? ;`

//what's next? Why?

`R = [1,2,1,3,1] ? ;`

`no.`

- What about the call? Why?

`| ?- delete(4,[1,2,1,3,1],R).`

`R = [1,2,1,3,1] ? ;`

`no.`

- So, the meaning of the predicate is: **delete one occurrence** of an item from the list; if absent, do NOTHING (leave the list unchanged).

Delete from a list - contd

- What are the differences when the 2 implementations (without/with 3rd clause) are compared? Explain!
- What happens if the clause when item is found cuts the backtrack?
- Implementation without 3rd clause:

```
delete(H,[H|T],T):-!.
```

```
delete(X,[H|T],[H|R]):-
```

```
    delete(X,T,R).
```

- A cut in a clause is as if in all consequent clauses we add the negation of the conjunction to the left of the cut. Therefore, in a clause like:

$p:-q,r,!s.$

The cut implies a “default” negation of **q,r** (therefore (**not(q,r)**)) in all clauses after it.

- In the predicate `delete`, first clause contains **nothing** to the left of the cut. So, what does it negate?
- Does it make any sense to place that cut?

Delete from a list - contd

```
delete(H,[H|T],T):-!.
```

```
delete(X,[H|T],[H|R]):-
```

```
    delete(X,T,R).
```

//means

//therefore a

```
delete(X,[H|T],T):-X=H,!.
```

X<>H here

- In the implementation with 3rd clause:

```
delete(H,[H|T],T):-!.
```

```
delete(X,[H|T],[H|R]):-
```

```
    delete(X,T,R).
```

```
delete(_,[],[]).
```

- Answer the same queries for **both** implementations and estimate the output (initial queries and repetitions). Explain!

```
| ?- delete(1,[1,2,1,3,1],R1).
```

```
R1=...? ; ? WHY?
```

```
| ?- delete(4,[1,2,1,3,1],R2).
```

```
R2=...? ; ? WHY?
```

Delete all occurrences of an item from a list

Start from the first version of the predicate

`delete(H,[H|T],R):-` //if the item to remove is head of input, don't put it on output

`delete(H,T,R).` //but also remove other occurrences from tail

`delete(X,[H|T],[H|R]):-` //otherwise, keep it on output and

`delete(X,T,R).` //remove from tail

- Could it be just this?
- Justify!

Delete all occurrences of an item from a list

```
delete_all(H,[H|T],R):-//if the item to remove is head of input, don't put it on output
    delete_all(H,T,R). //but also remove other occurrences from tail
delete_all(X,[H|T],[H|R]):- //otherwise, keep it on output and
    delete_all(X,T,R). //remove from tail
delete_all(_,[],[]). //when the empty list is reached, done, result empty
```

?- delete_all(1,[1,2,1,3,1],R).

R = [2,3]

- What happens if we repeat the query? Why?
- How many answers? Which order? Explain!
- How can we obtain just the answer from above?

Delete all occurrences of an item from a list

```
delete_all(H,[H|T],R):-!,
    delete_all(H,T,R).
delete_all(X,[H|T],[H|R]):-
    delete_all(X,T,R).
delete_all(_,[],[]).
```

- A cut in a clause is as if in all consequent clauses we add the negation of the conjunction to the left of the cut. Therefore, in a clause like:
 $p:-q,r,! ,s.$
 The cut implies a “default” negation of q,r (therefore $\text{not}(q,r)$) in all clauses after it.
- In the predicate `delete_all`, first clause contains **nothing** to the left of the cut. So, what does it negate?
- Does it make any sense to place that cut?

Delete all occurrences of an item from a list

```
delete_all(H,[H|T],R):-!,  
    delete_all(H,T,R).
```

What does ! negates?

Is the default unification! The clause above is like:

```
delete_all(X,[H|T],R):-  
    X=H,!,  
    delete_all(X,T,R).
```

Therefore, the cut negates $X=H$, thus, in the next clauses, there is a default $X \neq H$.

Delete all occurrences of an item from a list

```
delete_all(H,[H|T],R):-!,
    delete_all(H,T,R).
delete_all(X,[H|T],[H|R]):-
    delete_all(X,T,R).
delete_all(_,[],[]).
?- delete_all(1,[1,2,1,3,1],R).
```

- What is the result? Why?
- What happens if we repeat the query? Why? How many answers? Which order? Explain!

Conclusion

- Order of clauses matters
- Order of calls matters
- Always estimate performance
- Meaning of
 - Repeating the query
 - The cut
- Questions?

References explained

1. **Lloyd**, J. W, Foundations of Logic Programming, Springer, 1984
2. **Clocksin**, William, **Mellish**, Christopher S., Programming in Prolog - Using the ISO Standard, Springer 1981
3. Leon S. **Sterling** and Ehud Y. **Shapiro**, The Art of Prolog Advanced Programming Techniques
4. Robert **Kowalski**, Logic for Problem Solving, 1979
5. Rodica **Potolea**, Programare Logica (vol 1), UTPres, 2007

TECHNICAL UNIVERSITY

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #5, 2021

Cluj-Napoca

Computer Science

Agenda

- Side effects
- Trees
 - representation
 - basic operations
- Tree traversal
 - pre, in, post orders
- Tree transformation into an ordered list
 - forward
 - backward
 - can we do better?

Side effects

- Allow for **persistent structure**
- Allow for **wider scope visibility** (modelling global variables; created in some predicate and visible elsewhere, without passing arguments)
- Allow for **metaprogramming**
- By using them, we exceed the pure Logic Programming (operations NOT undone on backtracking)

Side effects – contd.

- Built in predicates
- *assert/retract* for adding/extracting ONE clause of the predicate with the name provided as argument
- The argument (for both assert and retract) is a variable representing a dynamic predicate
- `assert(X) / retract(X)`
- The variable (X) must be ALREADY instantiated to the predicate intended to be updated

Side effects – contd.

- assert comes into 2 flavors:
 - asserta – adds in the beginning (on top) of the existing clauses for the predicate provided as argument
 - assertz – adds at the end (at bottom) of the existing clauses for the predicate provided as argument
 - It NEVER fails
 - on backtracking, the item (clause) is NOT removed. However, assert does not re-execute on backtracking (NO new clause is added).

Side effects – contd.

- retract:
 - removes the first item (clause of the predicate provided as argument) with whom it matches successfully
 - if no match (the matching fails), retract fails. So, as opposed to assert, it could fail
 - on backtracking, the item (clause removed) is NOT added back. On the contrary, another item is removed in case another successful match is possible, *or* retract fails

assert

- Assume we have some (3 in the ex.) clauses for a predicate $p()$ (arguments do not matter here).
- Here on the right, the clauses have numbers added to represent their index

$p():-\dots$

$p_1():-\dots$

$p():-\dots$

$p_2():-\dots$

$p():-\dots$

$p_3():-\dots$

asserta/z

- As a result of the execution of `asserta(p())`, the predicate becomes:

$p_{\text{new}}() :- \dots$

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

- As a result of the execution of `assertz(p())`, the predicate becomes:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

$p_{\text{new}}() :- \dots$

retract

- Assume the same scenario:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

- The execution of `retract(p())` would:
 - Start scanning the list of clauses of the `p`
 - Stop at the first clause where arguments (not mentioned here) match
 - Remove the given clause and report success
 - If no successful matching, the retract fails

retract – success on the first clause

- Assume the same scenario:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

- The execution of $\text{retract}(p())$ succeeds on matching with the **first** clause of p would lead to:

$p_2() :- \dots$

$p_3() :- \dots$

retract – success on the second clause

- Assume the same scenario:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

- The execution of $\text{retract}(p())$ succeeds on matching with the **second** clause of p would lead to:

$p_1() :- \dots$

$p_3() :- \dots$

retract – success on the third clause

- Assume the same scenario:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

- The execution of $\text{retract}(p())$ succeeds on matching with the **third** clause of p would lead to:

$p_1() :- \dots$

$p_2() :- \dots$

retract – fails

- Assume the same scenario:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

- The execution of `retract(p())` **does NOT succeed** trying to match `p` to ANY of the 3 clauses, the result would be:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

retract – on backtracking

- After the execution of retract, on backtracking, NO undo is performed (the removal stays). That clause is NEVER restore, unless an explicit request (assert) is made
- Moreover, on the attempt to REsatisfy the goal (due to backtracking), Prolog continues FROM THE PLACE it previously removed and tries to remove ANOTHER clause, and:
 - it does so if the match succeeds,
 - or else fails and the execution is resumed (backtracked) to the predicate on the left of retract

Computer Science

retract – on backtracking - contd

- Assume the same scenario where we have 5 clauses:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

$p_4() :- \dots$

$p_5() :- \dots$

- Assume the execution of $\text{retract}(p())$ succeeds on matching with the **second** clause of p , after execution the predicate contains:

$p_1() :- \dots$

$p_3() :- \dots$

$p_4() :- \dots$

$p_5() :- \dots$

retract – on backtracking - contd

- If now we backtrack again, and p_3 is matched, the predicate looks:

$p_1():-\dots$

$p_4():-\dots$

$p_5():-\dots$

- If we now backtrack again, and p_5 is matched the predicate looks:

$p_1():-\dots$

$p_4():-\dots$

- A new backtrack now would fail, as we reached the end of the list of clauses.

retract – on backtracking - contd

- Assume the same scenario where we have 5 clauses:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

$p_4() :- \dots$

$p_5() :- \dots$

- Assume the execution of $\text{retract}(p())$ cannot unify with **ANY** of the 5 clauses, the execution **fails** and p remains unchanged:

$p_1() :- \dots$

$p_2() :- \dots$

$p_3() :- \dots$

$p_4() :- \dots$

$p_5() :- \dots$

assert and retract

- Combining assert and retract
 - asserta + retract => LIFO
 - assertz + retract => FIFO

Trees

- Structure not defined; it is (used) as we define them
- Traditional (in use in literature) definition is either prefix or infix notation, with a letter as name for the structure: n or t.
- Therefore, the tree



- Is represented in one or the forms:

prefix not: `tree(n(7,n(2,nil,nil),n(9,nil,nil)))`.

infix not: `tree(n(n(nil,2,nil),7,n(nil,9,nil)))`.

- Existing trees are represented as facts (predicate tree) in the program

Trees –operations

- Same as in other languages
- Basic operations:
 - Traversal
 - Search
 - Insert
 - Delete

Trees – traversal

- 3 types: pre/in/post order

`inorder(nil).`

`inorder(n(Left,Key,Right)):-`

`inorder(Left),`

`do_something(Key),`

`inorder(Right).`

- The predicate has not outcome/output unless `do_something` is doing side effects. Why?
 - No arguments!
- The order of clauses doesn't matter. Why?
 - Due to indexation on the first argument
- The predicate `do_something` does the actual job
- It runs $O(n)$
 - in case `do_something` is having constant time ($O(1)$)
- Otherwise the time is calculated with the Master Theorem.
- pre/post orders are similar, just that `do_something` goes first/last.

Tree transformation into an ordered list

- Given a Binary Search Tree
 - BST; what is it? Why do we represent trees as search trees?
 - Benefits? Limitations?
 - Means of overcoming limitation?
- Generate the ordered list of the keys in the tree
- Approaches: divide et impera
 - divide – already done (the tree structure is a partition already)
 - impera – combine the results obtained on the subsets of the partition
 - Depending on how efficient we combine them we get the efficiency of the solution
- Approaches considered:
 - Forward and backward recursion
 - Pre/in/post order (which one is available in the context)

Computer Science

Tree 2 list forward

- By the approach, besides the output argument, we need a supplementary argument, some partial result to accumulate the results "so far"
- The arguments needed are: input tree, output list, accumulator (list type)
- Order of arguments: input has to come first to benefit from the first argument indexation
- The other arguments are in any order

Tree 2 list forward – code

- By the approach, besides the output argument, we need a supplementary argument, some partial result to accumulate the results “so far”
- The arguments needed are: input tree, output list, accumulator (list type)
- Order of arguments: input has to come first to benefit from the first argument indexation
- The other arguments are in any order; we’ll consider accumulator as second argument
- The meaning of the accumulator = the elements from the tree we already covered placed in an ordered list

Tree 2 list forward – code

```
% sort_1/3
% sort_1(in_tree,partial_result_list,final_result_list)

sort_1(nil,L,L) .
sort_1(n(Left,Key,Right),Lin,Lout) :-
    sort_1(Left,Lin,Llout),
    append(Llout,[Key],Lrin),
    sort_1(Right,Lrin,Lout).
```

Computer Science

Tree 2 list forward – code

```
sort_1(nil,L,L).  
sort_1(n(Left,Key,Right),Lin,Lout):-  
    sort_1(Left,Lin,Llout),  
    append(Llout,[Key],Lrin),  
    sort_1(Right,Lrin,Lout).
```

Clauses:

- When we reached the empty tree, the accumulator instantiate the result (default unification) as always in forward recursion.
- For the nonempty tree, the forward approach solves the parts of the partition in the standard order (first to last flow)
 - Solve the problem on the left subtree; the accumulator (content of the tree already covered, Lin) is passed as partial result. The result computed by this call will add all the keys in the left subtree to that accumulator and passed as accumulator (Llout) to the next processing task; sort_1(Left,Lin,Llout),
 - Solve the problem on the next part in the partition = Key; append(Llout,[Key],Lrin),
 - Solve the problem on the right subtree; the accumulator (content of the tree already covered, Lrin) is passed as partial result. The result computed by this call will add all the keys in the right subtree to that accumulator and passed as final result = on the whole tree); sort_1(Right,Lrin,Lout).

Tree 2 list forward – code analysis

Needs specific initial call:

```
sort_1(nil, L, L).  
sort_1(n(Left, Key, Right), Lin, Lout) :-  
    sort_1(Left, Lin, Llout),  
    append(Llout, [Key], Lrin),  
    sort_1(Right, Lrin, Lout).
```

```
sort_1(Tree, Result) :-  
    sort_1(Tree, [], Result).
```

- Needs additional argument (accumulator)
- Evaluation:
- 2 recursive calls (a=2)
- 2 proportional parts (assuming tree is balanced; why is it a reasonable assumption?) (b=2)
 - Outside recursive call linear time (append) (c=1)
 - Hence, **$O(n \lg n)$** – supralinear time, but a small quantity over n
- Please come back to this code AFTER we study difference lists

Tree 2 list backward – code

- Needs no additional argument
- The arguments needed are: input tree, output list,
- Order of arguments: input has to come first to benefit from the first argument indexation

```
% sort_2/2
% sort_2(in_tree, final_result_list)
sort_2(nil, []).
sort_2(n(Left,Key,Right),Lout):-
    sort_2(Left,Llout),
    append(Llout,[Key],Lintout),
    sort_2(Right,Lrout),
    append(Lintout, Lrout,Lout).
```

Tree 2 list backward – code

```
sort_2(nil, []).  
sort_2(n(Left, Key, Right), Lout):-  
    sort_2(Left, Llout),  
    append(Llout, [Key], Lintout),  
    sort_2(Right, Lrout),  
    append(Lintout, Lrout, Lout).
```

Clauses:

- When we reached the empty tree, the result is empty.
- For the nonempty tree, the backward approach solves:
 - left subtree; the keys from left subtree generate the list from left out (Llout),
 - on the next part in the partition = Key to be added at the end of the Llout to create Lintout; `append(Llout, [Key], Lintout)`,
 - right subtree; the keys from right subtree generate the list from right out (Lrout),
 - finally, the results of parts are concatenated (`append(Lintout, Lrout, Lout)`).

Tree 2 list backward– code analysis

```
sort_2(nil, []).  
sort_2(n(Left, Key, Right), Lout) :-  
    sort_2(Left, Llout),  
    append(Llout, [Key], Lintout),  
    sort_2(Right, Lrout),  
    append(Lintout, Lrout, Lout).
```

- 2 recursive calls ($a=2$)
- 2 proportional parts (assuming tree is balanced) ($b=2$)
- Outside recursive call linear time (append) ($c=1$)
- Hence, **$O(n \lg n)$** – supralinear time, but a small quantity over n
- Compared to the previous strategy?
- Time is larger (2 append calls instead of 1)

Tree 2 list backward– better?

- What can be done to improve?
- 2 append calls, both passing (almost) through the same list
- The first append go over the whole first argument (large list) in order to add just one element (Key) at the end.
- Better?
- What if somehow we rephrase:
 - Add at the end of the list
 - With add at the beginning of a list. How?
 - Instead of adding Key at the end of L1out
 - Add it at the beginning of Lrout
 - But is NOT available at this moment
 - Change order of processing

Tree 2 list backward – code

```
sort_3(nil, []).  
sort_3(n(Left, Key, Right), Lout) :-  
    sort_3(Left, Llout),  
    sort_3(Right, Lrout),  
    append(Llout, [Key| Lrout], Lout).
```

Clauses:

- When we reached the empty tree, the result is empty.
- For the nonempty tree, the backward approach solves:
 - left subtree; the keys from left subtree generate the list from left out (Llout),
 - right subtree; the keys from right subtree generate the list from right out (Lrout),
 - finally, the results of parts are concatenated with the unit element in between; just that between means now before the result on the right (append(Llout, Lrout, Lout)).

Tree 2 list backward – code

```
sort_3(nil, []).  
sort_3(n(Left, Key, Right), Lout) :-  
    sort_3(Right, Lrout),  
    Lint=[Key| Lrout],  
    sort_3(Left, Llout),  
    append(Llout, Lint, Lout).
```

Pure backward recursion code

Tree 2 list backward– code analysis

```
sort_3(nil, []).  
sort_3(n(Left, Key, Right), Lout) :-  
    sort_3(Left, Llout),  
    sort_3(Right, Lrout),  
    append(Llout, [Key| Lrout], Lout).
```

- 2 recursive calls ($a=2$)
- 2 proportional parts (assuming tree is balanced) ($b=2$)
- Outside recursive call linear time (append) ($c=1$)
- Hence, **$O(n \lg n)$**
- Better (multiplicative constant) than the previous backward as just one append
- Better than forward – needs no additional parameter
- Can we do even better? Answer – we can! Check after difference lists

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #6, 2021

Cluj-Napoca

Computer Science

Agenda

- Trees (BST and regular trees)
 - insert
 - search
 - delete
- Incomplete structures
 - meaning/utility
 - lists
 - Trees – next lecture

Insert in BST

```
%insert_bst/3
```

```
%insert_bst(in_tree, key_2_ins,out_tree).
```

Q: is the order of arguments relevant ? Why/why not?

```
insert_bst(nil, Key,t(nil,Key,nil)):-!.
```

```
% insert_bst(Tin, Key,Rez):-Tin=nil,! ,Rez= t(nil,Key,nil).
```

```
insert_bst(t(Left,Key,Right), Key, t(Left,Key,Right)):-!,  
    write("already in tree").
```

```
insert_bst(t(Left,Key1,Right),Key,t(NewLeft,Key1,Right)):-  
    Key<Key1,! ,  
    insert_bst(Left,Key,NewLeft).
```

```
insert_bst(t(Left,Key1,Right),Key,t(Left,Key1,NewRight)):-  
    insert_bst(Right,Key,NewRigth).
```

Qs: **1.** Where is inserted (position)? ALWAYS LEAF! NO EXCEPTION!

2. What does ! in clause 1 negate? In clause 2?

Time estimate: a=1, b=2 (if balanced), c=0 =>O(lgn)

Q: Estimate best/worst case (situation) and time

Insert in regular (not search) tree

```
%insert/3
%insert(in_tree, key_2_ins,out_tree).

insert(nil, Key,t(nil,Key,nil)):-!.
%insert(Tin, Key,Tout):-Tin=nil,!, Tout=t(nil,Key,nil).
insert(t(Left,Key,Right),Key,t(Left,Key,Right)):-!,
    write("already in tree").
%insert(Tin,Key,Tout):-Tin=t(Left,Key,Right),!,
    %write("already in tree"),Tout= t(Left,Key,Right)).
insert(t(Left,Key1,Right),Key,t(NewLeft,Key,Right)):-
    insert(Left, Key,NewLeft).
insert (t(Left,Key1,Right), Key,t(Left,Key,NewRight)):-
    insert(Right,Key,NewRigth).
```

Qs: 1. Where is inserted (position)? Specifically!

Oral discussion on OR nondeterminism for parallel/concurrent processing.

Search in BST

Assume the node contains a structure of type `a(key,value)`

The tree is BST based on key

```
Ex: tree(n(
    (n(nil,a(5, john), nil)
    a(7, dan),
    n(nil,a(9, peter), nil)
    )
    ).
```

Represents a tree with 7 and 2 leaves (7 and 9).

```
      a(7, dan)
     /      \
a(5, john)  a(9, peter)
```

If stored in knowledge base, processing is with:

```
?-tree(T), process(T,...).
```

Search in BST

(search based on Key)

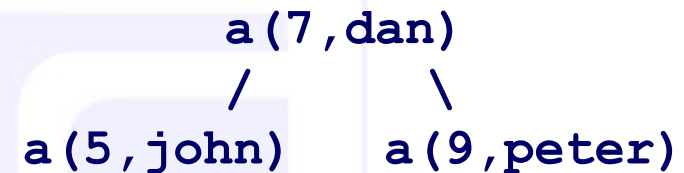
```
%is_in_stree/2
%is_in_stree(key_searched,tree)

is_in_stree(SN,n(_,Node,_):-
    eqch(SN,Node),!
    SN=Node.
is_in_stree(SN,n(Left,Node,_):-
    ord(SN,Node),!
    is_in_stree(SN,Left).
is_in_stree(SN,n(_,_,Right):-
    is_in_stree(SN,Right).

eqch(a(SK,_),(a(Key,_)):-
    nonvar(SK),
    SK=Key,!..
ord(a(SK,_),a(Key,_)):-
    nonvar(SK),
    SK<Key,!..
```

Search in BST (search by Key) – contd.

```
eqch(a(SK,_), (a(Key,_)) :-  
    nonvar(SK),  
    SK=Key, !.
```



Predicate to check if the *key* we are *searching* for has the **same** value as the *key* in the *current node* in the tree.

Qs: what is the meaning of `nonvar(SK)` and why is needed? The `!` Is not mandatory! Why?

```
ord(a(SK,_), a(Key,_)) :-  
    nonvar(SK),  
    SK<Key, !.
```

Predicate to check if the *key* we are *searching* for has a **smaller** value than the *key* in the *current node* in the tree.

Q: same questions.

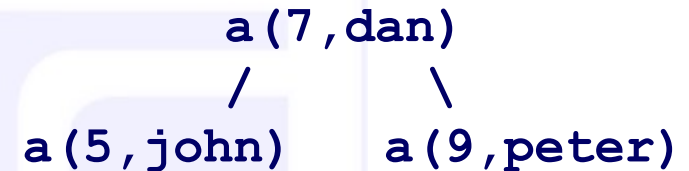
```
?-tree(T), is_in_tree(a(9,X), T).
```

Yes, T=..., X=...?

Search in regular tree (search by Value)

```
%is_in_tree/2
%is_in_tree(key_searched,tree)

is_in_tree(SN,n(_,SN,_)).
is_in_tree(SN,n(Left,_,_):-
    is_in_tree(SN,Left).
is_in_tree(SN,n(_,_,Right):-
    is_in_tree(SN,Right).
```



```
?-tree(T),is_in_tree(a(R,john),T).
Yes, T=...,R=5.
```

%T would be the tree read, R the key assoc. to the value (name) provided

```
?-tree(T),is_in_tree(a(9,X),T).
```

X=...?

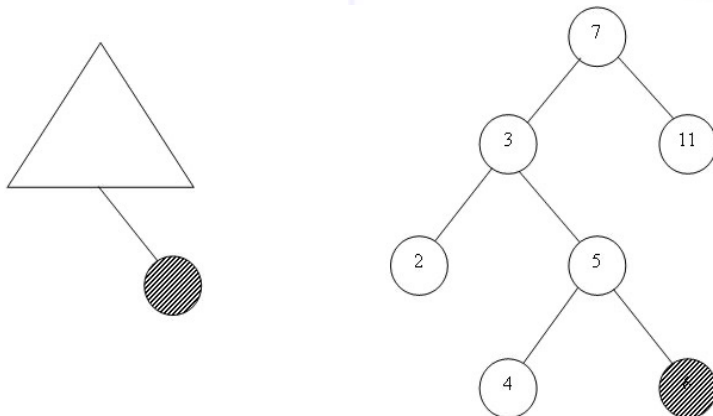
% can we ask this way? What's the answer? What's the meaning?

Delete from BST

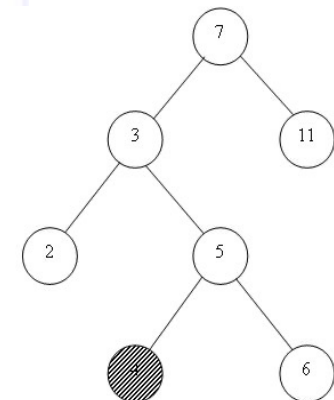
- Search for the node where the key resides + remove the node
- Delete a key = remove the node containing the key
- Cases to consider:
 - Leaf – just remove the node
 - One child node – shortcut the node (link the child to its grandparent)
 - Two children ($\Leftrightarrow$ delete the root after the search stage) different story

Figs. For leaves removals

Case 1



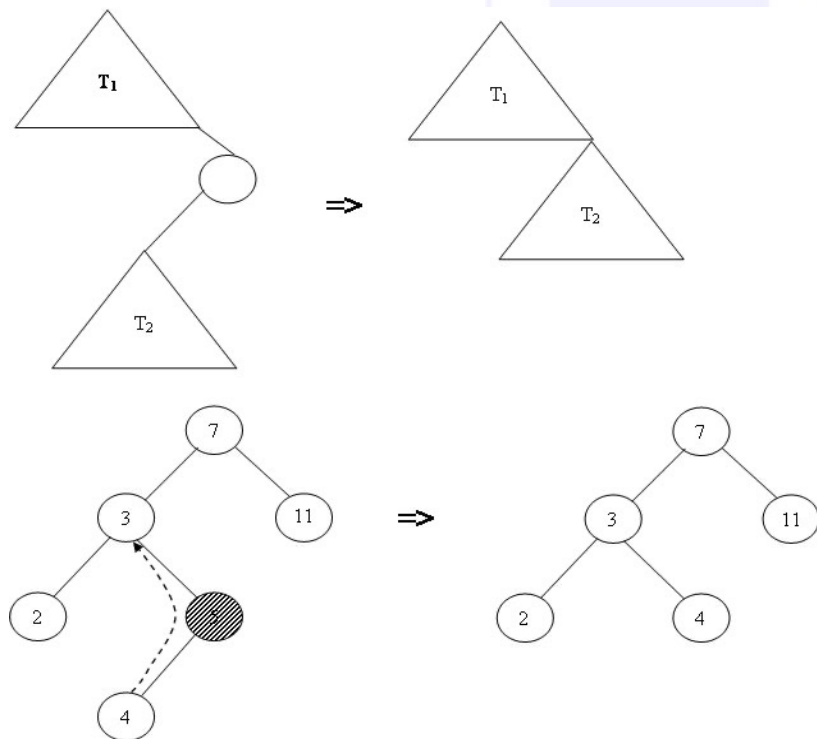
Case 2



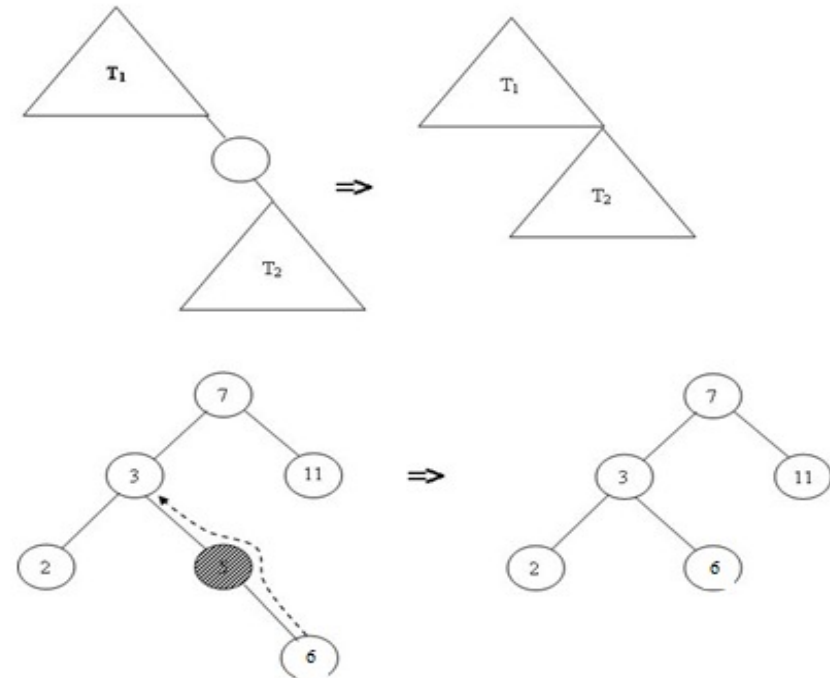
Delete from BST – contd.

Removal of a right child node with just one subtree

Case 3



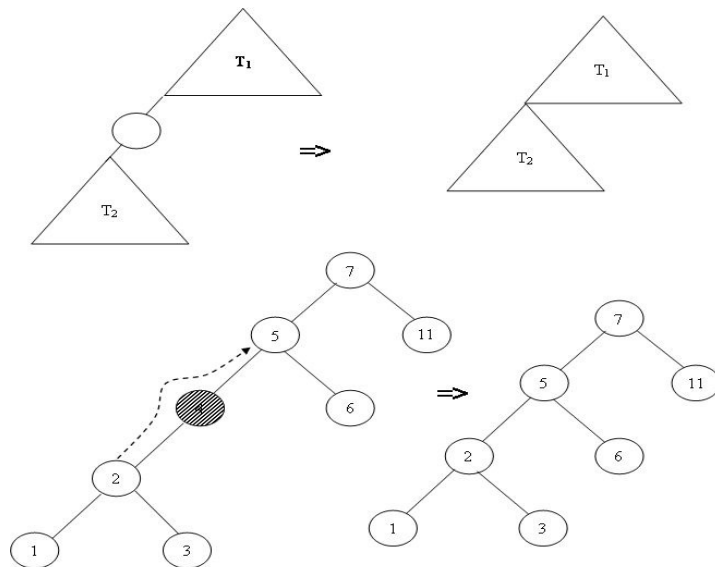
Case 4



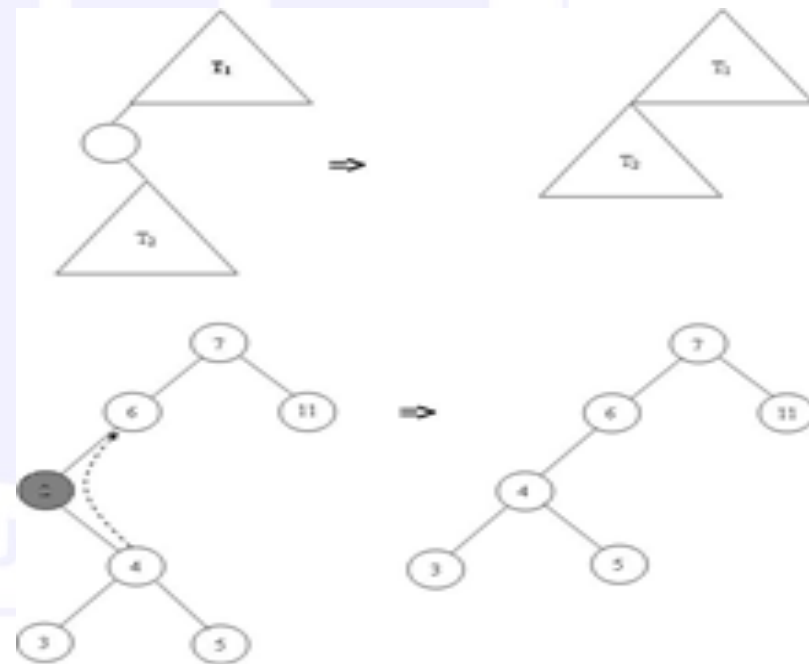
Delete from BST – contd.

Removal of a left child node with just one subtree

Case 5



Case 6

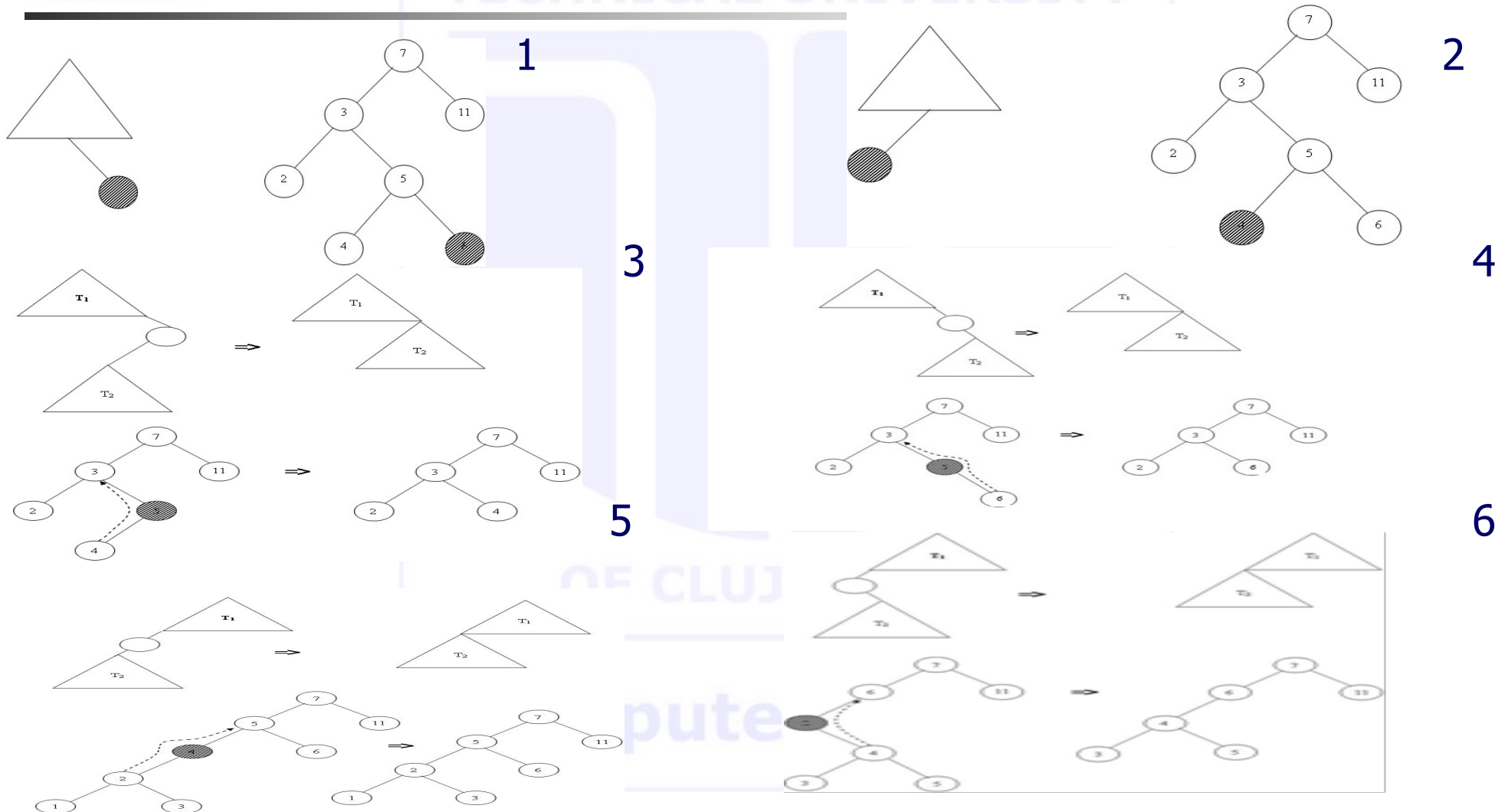


- How about root? Postpone for the moment.
- Let's reach this point with code.

Delete from BST - code

```
% delete_bst/3
% delete_bst(in_tree, key_to_del,out_tree)
% search key; find the node to contain the key
delete_bst(t(Left,Key1,Right),Key,t(NewLeft,Key1,Right)):-
    Key<Key1,!,
    delete_bst(Left,Key,NewLeft).
delete_bst(t(Left,Key1,Right),Key,t(Left,Key1,NewRight)):-
    Key>Key1,!,
    delete_bst(Right,Key,NewRight).
% found key; let's solve the easy cases discussed before
delete_bst(nil,Key,nil):-!,
    write(Key),write('not in tree').
delete_bst(t(nil,Key,nil),Key,nil):-!.
delete_bst(t(nil,Key,Right),Key,Right):-!.
delete_bst(t(Left,Key,nil),Key,Left):-!.
% "difficult" case postponed
```

Delete cases



Delete from BST - code

```
% delete_bst/3
% delete_bst(in_tree, key_to_del,out_tree)
delete_bst(t(Left,Key1,Right), Key,t(NewLeft,Key1,Right)):-
    Key<Key1,!,
    delete_bst(Left, Key,NewLeft).
delete_bst(t(Left,Key1,Right), Key,t(Left,Key1,NewRight)):-
    Key>Key1,!,
    delete_bst(Right, Key,NewRight).
delete_bst(nil,Key,nil):-!,
    write(Key),write(' nu exista in arbore').
delete_bst(t(nil,Key,nil), Key,nil):-!.
delete_bst(t(nil,Key,Right), Key,Right):-!.
delete_bst(t(Left,Key,nil), Key,Left):-!.
```

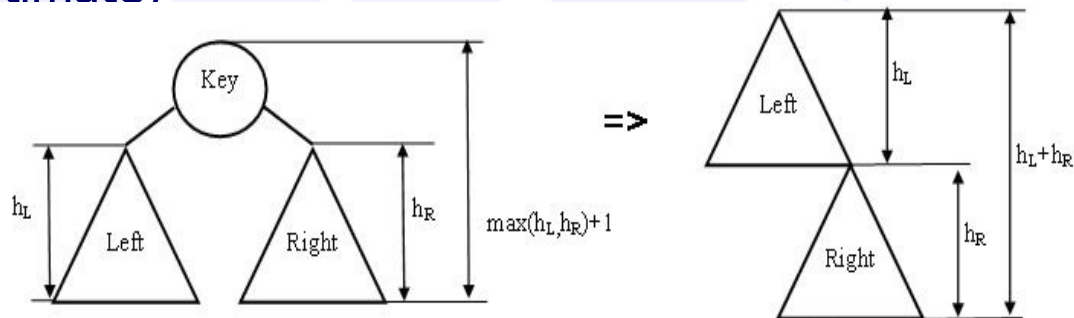
Qs:

- What cases did we cover so far?
- Which clause covers which case? Explain!
- Do we need all clauses so far? Why/why not? Justify

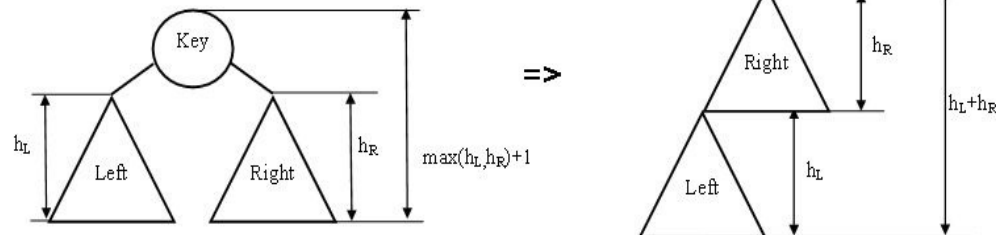
Delete from BST – code – contd.

- We reached the node containing the key to delete
- Has 2 children = is the root of a subtree => problem is remove the root of the tree
- How to...? Think! We have alternatives:
- Either link the subtrees:
 - Easy to apply (time estimate)
 - Disadvantage?

Sol1



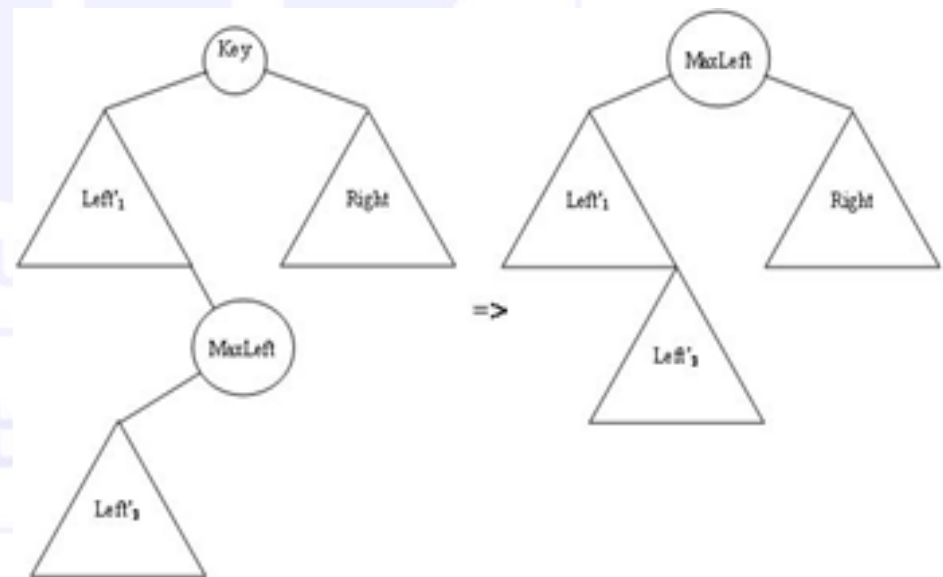
Sol2



Delete from BST – code – contd.

- Or replace the problem to solve with a problem you know to solve!
- Remove **INSTEAD** the node you want to remove (is the root, remember this) with another one. How:
 - Find an easy to remove node
 - Place it instead of the root (is possible?)
 - To be possible, that node should be a node in inorder:
 - Either before the key in the root
 - Or after the key in the root
 - So replace with
 - Predecessor
 - Successor
 - !
 - Are those nodes with the desired property?
 - Justify

Sol3



Delete from BST – code – contd.

```
delete_bst(t(Left,Key,Right), Key,t(NewLeft,MaxLeft,Right)):-!,
    deleteMaxFromTree(Left,MaxLeft,NewLeft).
```

```
% deleteMaxFromTree/3
```

```
% deleteMaxFromTree(in_tree,max_key,out_tree).
```

- The predicate takes a tree as input `in_tree`, finds the max key `max_key` (the rightmost node) and removes it from the tree, returning the tree without that key `out_tree`.

```
deleteMaxFromTree(t(Left,Key,Right),MaxKey,t(Left,Key,NewRight)):-
    deleteMaxFromTree(Right,MaxKey,NewRight).
```

```
deleteMaxFromTree(t(Left,MaxKey,nil),MaxKey,Left):-!.
```

- Clause 1 says: as long as the right subtree is not empty, go to the right
- Clause 2 says: if right is `nil`, you found the max key `MaxKey` and `Left` is out tree.
- Do we ever reach second clause? Infinite loop. Clauses not in correct order! Let's change it!

Delete from BST – code – contd.

```
deleteMaxFromTree (t (Left, MaxKey, nil) , MaxKey, Left) :- !.  
deleteMaxFromTree (t (Left, Key, Right) , MaxKey, t (Left, Key, NewRight) ) :-  
    deleteMaxFromTree (Right, MaxKey, NewRight) .
```

- Now is highly inefficient! We always try the non-useful clause first! Too much!
- Do we really need to have the fact first?
- No, is good to go second (JUSTIFY!).

```
deleteMaxFromTree (t (Left, Key, Right) , MaxKey, t (Left, Key, NewRight) ) :-  
    deleteMaxFromTree (Right, MaxKey, NewRight) .  
deleteMaxFromTree (t (Left, MaxKey, nil) , MaxKey, Left) :- !.
```

Computer Science

Delete from BST – complete code

```
delete_bst(t(Left,Key1,Right),Key,t(NewLeft,Key1,Right)):-
    Key<Key1,!,
    delete_bst(Left,Key,NewLeft).
delete_bst(t(Left,Key1,Right),Key,t(Left,Key1,NewRight)):-
    Key>Key1,!,
    delete_bst(Right,Key,NewRight).
delete_bst(Key,nil,nil):-!,
    write(Key),write('not in tree').
delete_bst(t(nil,Key,Right),Key,Right):-!.
delete_bst(t(Left,Key,nil),Key,Left):-!.
delete_bst(t(Left,Key,Right),Key,t(NewLeft,MaxLeft,Right)):-!,
    deleteMaxFromTree(Left,MaxLeft,NewLeft).

deleteMaxFromTree(t(Left,Key,Right),MaxKey,t(Left,Key,NewRight)):-
    deleteMaxFromTree(Right,MaxKey,NewRight).
deleteMaxFromTree(t(Left,MaxKey,nil),MaxKey,Left):-!.
```

Delete from BST – code analysis

- Search phase: $O(h)$
- Delete (no matter the solution chosen): $O(h)$
- Overall: $2h$
- $O(h)$, constant 2. Is it so?
- It is just $O(h)$, constant 1. JUSTIFY!
- $h=?$ Make a general analysis per types of trees.

Incomplete structures

Lists

- Structures ended in variables = instead of the specific empty structure it ends in a variable.
- This IS a list ended in variable: $L_1 = [1, 2, 3 | \mathbf{A}]$
 - Tail is VARIABLE
 - Can you specify its length?
- This IS NOT a list ended in variable $L_2 = [1, 2, 3, \mathbf{B}]$
 - Last element variable, can be anything (including a list, such as $[5, 6]$). BUT in case it unifies a variable, the structure would be a heterogeneous one: $L_2 = [1, 2, 3, [5, 6]]$

Incomplete Lists

- Basic operations

- Search
- Insert

```
member(H, [H|_]) .
```

```
member(H, [_|T]) :-
```

```
    member(H, T) .
```

```
?- L=[1,2,3|_], member(2, L) .
```

```
Yes, L=[1,2,3|_]
```

```
?- L=[1,2,3|_], member(4, L) .
```

```
Yes, L=[1,2,3,4|_]
```

- Bad behavior. Why? How can we fix the issue?

Incomplete Lists - search

```
member_IL(H, [H|_]) :- !.      %item found, stop. For good!
member_IL(H, [_|T]) :-        %not found,
    member_IL(H, T).           % go search in tail
member_IL(_, L) :-
    var(L), !,                 %reached final free variable
    fail.                      %stop with failure.

?- L=[1,2,3|_], member_IL(2, L).
Yes, L=[1,2,3|_]

?- L=[1,2,3|_], member_IL(4, L).
Yes, L=[1,2,3,4|_]
```

- Again same bad behavior. How come we didn't fix the issue?

Incomplete Lists – search – contd.

- NEVER reach clause #3 in the prev. code? Why?

```
member_IL(_, L) :-
    var(L), !,                %this order: check, cut, fail
    fail.
member_IL(H, [H|_]) :- !.    %cut mandatory
member_IL(H, [_|T]) :-
    member_IL(H, T).
?- L=[1, 2, 3|_], member_IL(2, L).
Yes, L=[1, 2, 3|_].          %exe stops on clause 2
?- L=[1, 2, 3|_], member_IL(4, L).
No.                          %exe stops on clause 1
```

- **Stop conditions** for **incomplete** structures are (i) with **cut** (!) and (ii) **ALWAYS come first**. Otherwise, it is as if they are missing!
 - Starts with failure condition(s). It MUST be explicit (NEVER default).
 - Continues with the successful stop condition(s).

Incomplete Lists – insert

```
insert_IL(X,L):-  
    var(L),           % reached the final free variable?  
    !,                % stop backtracking  
    L=[X|_].          % update the list with the new added item  
insert_IL(H,[H|_]):-!. %item found, and don't allow backtracking  
insert_IL(H,[_|T]):-  
    insert_IL(H,T).
```

```
q1:?- L=[1,2,3|_], insert_IL(4,L).
```

```
Yes, L=[1,2,3,4|_].
```

```
q2:?- L=[1,2,3|_], insert_IL(3,L).
```

```
Yes, L=[1,2,3|_].
```

- q1: pure insert behavior (stop on clause 1)
- q2: member behavior (stop on clause 2)

Incomplete Lists – insert – contd.

- We don't really need clause 1. Clause 2 covers it (default).

```
insert_IL(H, [H|_]) :- !.  
insert_IL(H, [_|T]) :-  
    insert_IL(H, T).
```

q1 : ?- L=[1,2,3|_], insert_IL(4,L).

Yes, L=[1,2,3,4|_].

Has pure insert behavior. Clause behaves as:

```
insert_IL(X,L) :-  
    var(L), !, L=[X|_].
```

q2 : ?- L=[1,2,3|_], insert_IL(3,L).

Yes, L=[1,2,3|_].

Has member. Clause behaves as:

```
member(X, [H|_]) :- X=H, !.
```

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #7, 2021

Cluj-Napoca

Computer Science

Agenda

- Incomplete structures
 - Review – meaning, usage, behavior
 - Incomplete Trees
- Difference lists

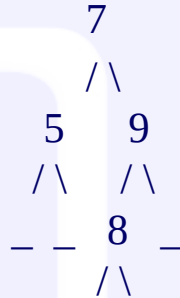
Computer Science

Incomplete structures - review

- Incomplete structures (lists, trees)
 - Replace the empty structure at the end
 - with a variable
- Used as I/O structure (saves time and space)
- Change the behavior of regular predicates
- *Failure* conditions NEED to be:
 - ALL Explicit (no default failure in the absence of the clause)
 - Come FIRST (otherwise, is as if it does not exist)
- *Success* conditions should be:
 - All explicit (as before)
 - Start with the condition for the variable at the end which comes right after the failure condition(s)

Search in BST

This is an Incomplete BST
(ended in variable)



```

tree(t(7, t(5,_,_),t(9,t(8,_,_),_))).
search(Key,t(Key,_,_)):-!,
    write("found").
search(Key, t(Key1,Left,_)):-
    Key<Key1,!,
    search(Key,Left).
search(Key,t(_,_,Right)):-
    search(Key,Right).
  
```

- The search predicate Works well for a **regular** BST
- What behavior would it have in case of an incomplete tree?

Search in BST

- It has **dual** significance in the search process:
 - if Key **present**, reports so
 - else **inserts** it as leaf (is absent does NOT fail)

```
tree(t(7, t(5,_,_),t(9,t(8,_,_),_))).
search(Key,t(Key,_,_)):-!,
    write("found").
search(Key, t(Key1,Left,_)):-
    Key<Key1,!,
    search(Key,Left).
search(Key,t(_,_,Right)):-
    search(Key,Right).
```

q1: ?-tree(T),search_ins(9,T).

q2: ?-tree(T),search_ins(6,T).

q1 behaves as search (= **search**_ins)

q2 behaves as insert (= search_**ins**)

Search in BST

```
q1: ?-tree(T),search_ins(9,T).  
search_ins(Key,t(Key,_,_)):-!,  
    write("found OR inserted").
```

q1 behaves as search (= **search_ins**)

The meaning of the clause is:

```
search_ins(Key,T):-  
    nonvar(T),T=t(Key1,_,_),Key1=Key,!,  
    write("found OR inserted").
```

```
q2: ?-tree(T),search_ins(6,T).  
search_ins(Key,t(Key,_,_)):-!,  
    write("found OR inserted").
```

q2 behaves as search (= **search_ins**)

The meaning of the clause is:

```
search_ins(Key,T):-  
    var(T),!,T=t(Key,_,_),  
    write("found OR inserted").
```

PURE Search in BST

(search based on Key)

```
search_IT(Key, T) :-  
    var(T), !,  
    fail.  
search_IT(Key, t(Key, _, _)) :- !,  
    write("found").  
search_IT(Key, t(Key1, Left, _)) :-  
    Key < Key1, !,  
    search_IT(Key, Left).  
search_IT(Key, t(_, _, Right)) :-  
    search_IT(Key, Right).
```

(would fail in case absent from tree)

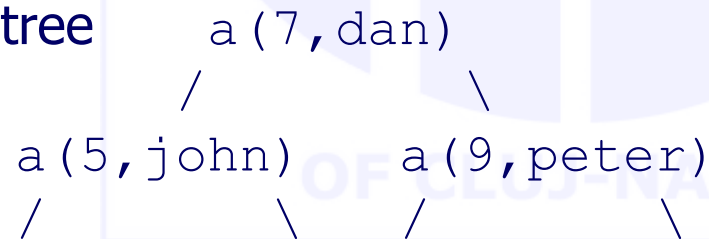
Search in BST

Assume the composite structure of type $a(\text{key}, \text{value})$

The incomplete BST

```
Ex: tree(n(
    (n(_, a(5, john), _)
    a(7, dan),
    n(_, a(9, peter), _)
    )
    ).
```

Represents the tree



Search in BST (search based on Key)

```
search_ins(Key,n(_,a(Key,Info),_),Info):-!.      a(7,dan)
search_ins(Key,n(Left,a(Key1,_),_),Info):-      /      \
      Key<Key1,!                                a(5,john) a(9,peter)
      search_ins(Key,Left,Info).                /      \
search_ins(Key,n(_,_,Right),Info):-              -      -      -      -
      search_ins(Key,Right).
```

q1: ?- tree(T), **search_ins**(9,T,R) .
Yes, R=peter

Computer Science

Search in BST (search based on Key)

```
search_ins(Key,n(_,a(Key,Info),_),Info):-!.      a(7,dan)
search_ins(Key,n(Left,a(Key1,_),_),Info):-      /      \
      Key<Key1,!                                a(5,john) a(9,peter)
      search_ins(Key,Left,Info).                /      \      /      \
search_ins(Key,n(_,_,Right),Info):-              -      -      -      -
      search_ins(Key,Right).
```

q1: ?- tree(T), **search_ins**(9,T,R) .
Yes, R=peter

The meaning of the first clause is:

```
search_ins(Key,n(_,a(Key1,Info1),_),Info):-
      Key=Key1,!,
      Info=Info1.
```

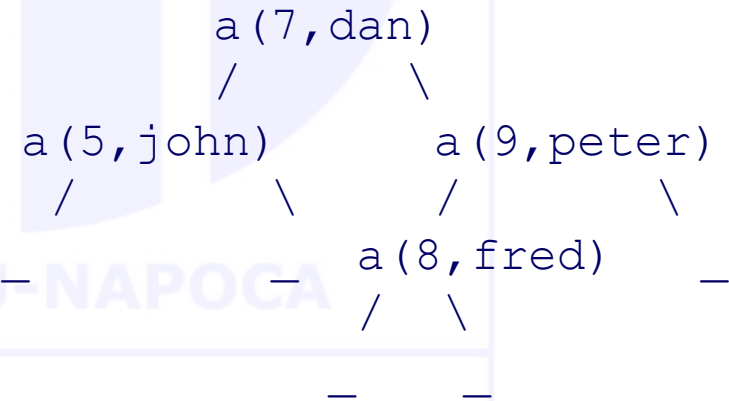
Computer Science

Search in BST (search based on Key)

```
search_ins(Key,n(_,a(Key,Info),_),Info):-!.      a(7,dan)
search_ins(Key,n(Left,a(Key1,_),_),Info):-      /      \
      Key<Key1,!                                a(5,john) a(9,peter)
      search_ins(Key,Left,Info).                /      \      /      \
search_ins(Key,n(_,_,Right),Info):-             -      -      -      -
      search_ins(Key,Right).
```

q2: ?- tree(T),search_ins(8,T,fred).

Yes, T increases



Computer Science

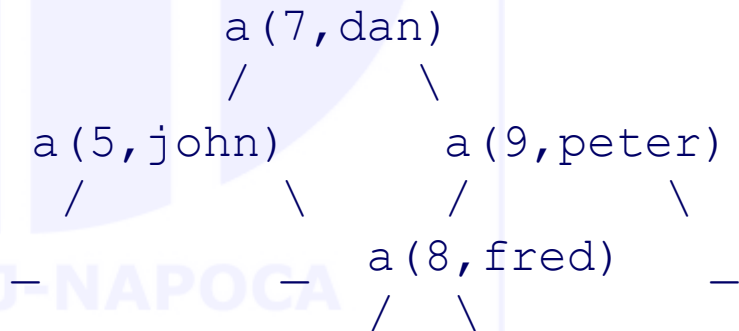
Search in BST (search based on Key)

```
search_ins(Key,n(_,a(Key,Info),_),Info):-!.      a(7,dan)
search_ins(Key,n(Left,a(Key1,_),_),Info):-      /      \
      Key<Key1,!                                a(5, john)  a(9, peter)
      search_ins(Key,Left,Info).                  /      \      /      \
search_ins(Key,n(_,_,Right),Info):-              -      -      -      -
      search_ins(Key,Right).
```

q2: ?- tree(T),search_ins(8,T,fred).

Yes, T increases

Here the meaning of the first clause is:

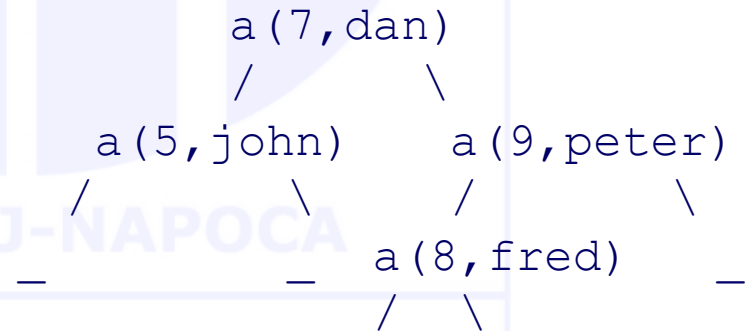


```
search_ins(Key,T,Info):-
      var(T),!
      T=n(_,a(Key,Info),_).
```

Search in BST (search based on Key)

```
search_ins(Key,n(_,a(Key,Info),_),Info):-!.      a(7,dan)
search_ins(Key,n(Left,a(Key1,_),_),Info):-      /      \
      Key<Key1,!                                a(5,john) a(9,peter)
      search_ins(Key,Left,Info).                /      \      /      \
search_ins(Key,n(_,_,Right),Info):-              -      -      -      -
      search_ins(Key,Right).
```

q3: ?- tree(T),search_ins(5,T,maria).

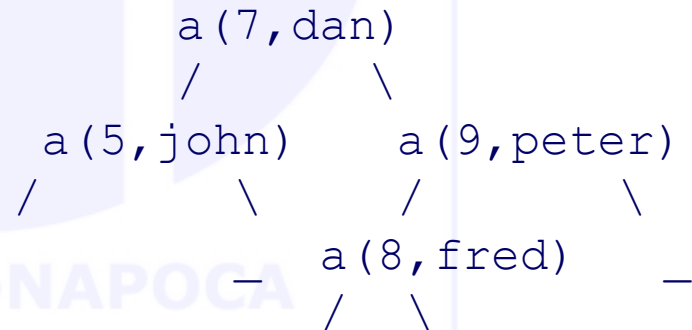


Does it fail?
If Yes, How?
If Not, Why?

Search in BST (search based on Key)

```
search_ins(Key,n(_,a(Key,Info),_),Info):-!.      a(7,dan)
search_ins(Key,n(Left,a(Key1,_),_),Info):-      /      \
      Key<Key1,!                                a(5,john) a(9,peter)
      search_ins(Key,Left,Info).                  /      \      /      \
search_ins(Key,n(_,_,Right),Info):-              -      -      -      -
      search_ins(Key,Right).
```

q3: ?- tree(T),search_ins(5,T,maria).



If we want it to fail, the first clause should be (but to have the other meanings, it should be added first)

```
search_ins(Key,n(_,a(Key1,Info1),_),Info):-
      Key=Key1,! ,Info=/=Info1, fail.
```

Search in BST

(PURE search by Key without insert)

```
search(_, T, _) :-
    var(T), !,          //key not in the tree
    fail.

search(Key, n(_, a(Key1, Info1), _), Info) :-
    Key=Key1, !,        //key in the tree
    Info=/=Info1, fail. //but with a different info

search(Key, n(_, a(Key, Info), _), Info) :- !. //key in the tree with same info

search_ins(Key, n(Left, a(Key1, _), _), Info) :-
    Key<Key1, !
    search_ins(Key, Left, Info).

search_ins(Key, n(_, _, Right), Info) :-
    search_ins(Key, Right).
```

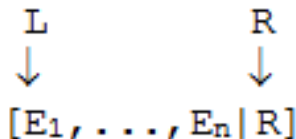
Clause #3 is reached only if (i) incomplete tree is reached (clause 1), (ii) key not found (clause 2)

Clause #3 can be interpreted as:

```
search(Key, n(_, a(Key1, Info1), _), Info) :- Key=Key1, Info=Info1, !.
```

Difference Lists

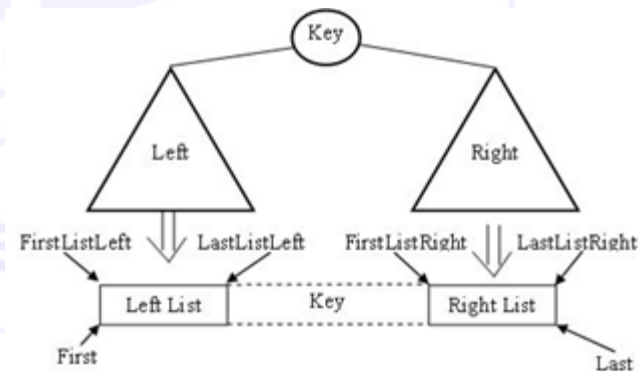
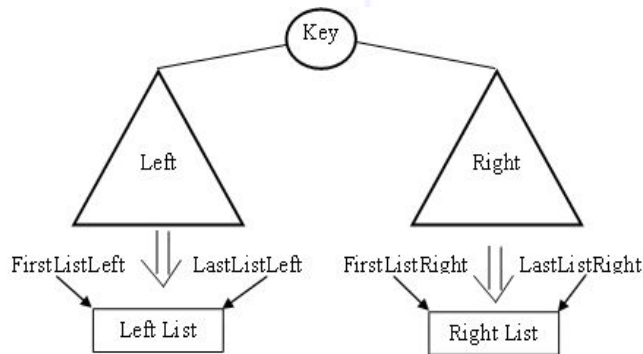
- Another type of “irregular” structures
- Allow access to both beginning and END of lists 😊
- Named as lists with First and Last element (that is, besides the variable =“pointer” at the beginning of the structure, we have a variable to “point” at the end of the list). Similar to imperative languages where we have lists with pointers in the beginning and end of the list
- Allow for increased performance
- Difference list:



- D , denoted as $L \setminus R$ (sometimes, in code, it is preferred to be represented as 2 arguments, L and R) is:
 - $D = L \setminus R = [E_1, \dots, E_n]$
 - L “points” at the beginning, R at the “end”

Tree to list (remember solutions in Lecture 5)

```
%generate_ordered_list/3
%generate_ordered_list(tree, begin_diff_list, end_diff_list)
generate_ordered_list(nil,First,Last):-First=Last.
generate_ordered_list (t(Left,Key,Right),First,Last):-
    generate_ordered_list(Left,FirstListLeft,LastListLeft),
    generate_ordered_list(Right,FirstListRight,LastListRight),
    First= FirstListLeft,
    LastListLeft=[Key|FirstListRight],
    Last= LastListRight.
```



Tree 2 list forward – code (Lecture #5)

```
% sort_1/3
```

```
% sort_1(in_tree,partial_result_list,final_result_list)
```

```
sort_1(nil,L,L) .
```

```
sort_1(n(Left,Key,Right),Lin,Lout) :-  
    sort_1(Left,Lin,Llout),  
    append(Llout,[Key],Lrin),  
    sort_1(Right,Lrin,Lout) .
```

```
sort_1(Tree,Result) :-  
    sort_1(Tree,[],Result) .
```

Tree to list

(same solution – check Lecture #5)

- Needs specialized call

```
generate_ordered_list(ITree, OList) :-
    generate_ordered_list(ITree, OList, []).
```

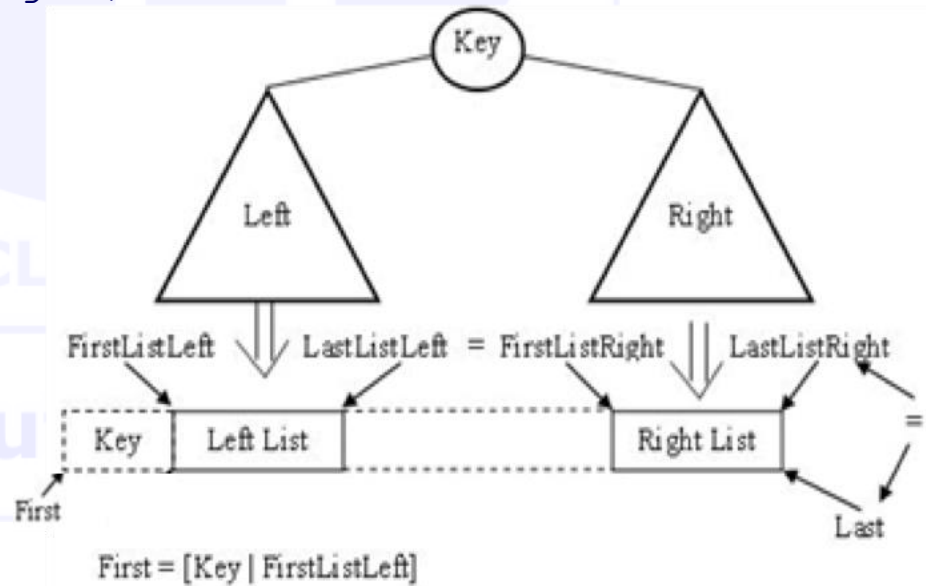
- Replace explicit unifications with implicit ones.

```
generate_ordered_list(nil, L, L).
generate_ordered_list(t(Left, Key, Right), FirstListLeft, LastListRight) :-
    generate_ordered_list(Left, FirstListLeft, [Key | Intermediate]),
    generate_ordered_list(Right, Intermediate, LastListRight).
```

- Same as before, yet the utilization of the same name for variables (FirstListLeft) instead of making explicit unifications (First=FirstListLeft and the other 2).

Pre-order

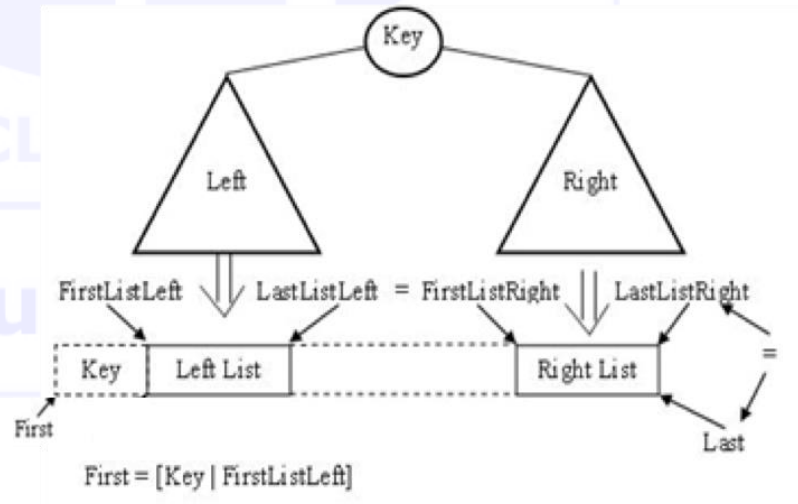
```
generate_preorder_list(nil, First, Last):-First=Last.
generate_preorder_list(t(Left,Key,Right),First,Last):-
    generate_preorder_list(Left,FirstListLeft,LastListLeft),
    generate_preorder_list(Right,FirstListRight,LastListRight),
    First= [Key|FirstListLeft],
    LastListLeft=FirstListRight,
    Last= LastListRight.
```



Pre-order

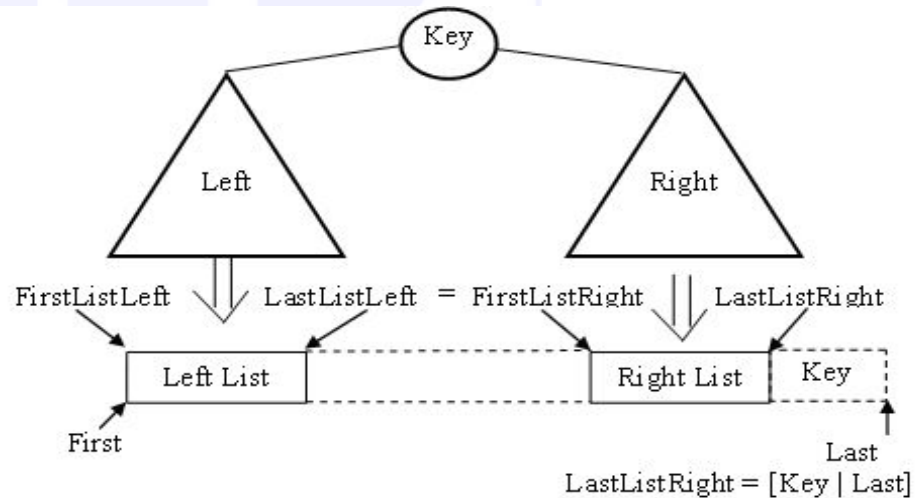
```
generate_preorder_list(nil, L, L) .
generate_preorder_list(t(Left, Key, Right), [Key | FirstListLeft], LastListRight) :
    generate_preorder_list(Left, FirstListLeft, Interm),
    generate_preorder_list(Right, Interm, LastListRight) .

generate_preorder_list(ITree, OList) :-
    generate_preorder_list(ITree, OList, []).
```



Post-order

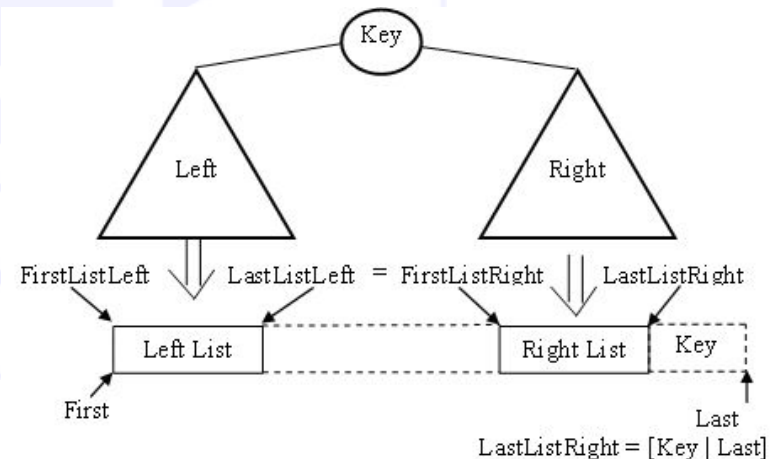
```
generate_postorder_list(nil, First, Last):-First=Last.
generate_postorder_list(t(Left,Key,Right),First,Last):-
    generate_postorder_list(Left,FirstListLeft,LastListLeft),
    generate_postorder_list(Right,FirstListRight,LastListRight),
    First= FirstListLeft,
    LastListLeft=FirstListRight,
    LastListRight=[Key|Last].
```



Post-order

```
generate_postorder_list(nil,L,L).
generate_postorder_list(t(Left,Key,Right),FirstListLeft,LastListRight):-
    generate_postorder_list(Left,FirstListLeft,Interm),
    generate_postorder_list(Right,Interm,[Key|LastListRight]).

generate_postorder_list(ITree,OList):-
    generate_postorder_list(ITree,OList,[]).
```



Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #8, 2021

Cluj-Napoca

Computer Science

Agenda

- Incomplete Trees – review
 - search to fail when key not in tree
 - search to fail when $\langle \text{key}, \text{value} \rangle$ given, key is in tree, but with a different value
- Difference lists
 - Review
 - Quicksort
 - queues
 - Flattening lists
- (other) Built in predicates

Search/insert in Incomplete BST

- The regular from a BST in an Incomplete BST has dual behavior:
 - Returns found in case in tree
 - Inserts it at the bottom level in case not found

- If want to fail if key not found? Add an explicit **first** clause as:

```
search(_,T):-  
    var(T),!,  
    fail.
```

- What other situations? Nodes contain <key,value>
- search to fail if key exists yet with a different value!

```
search(Key,n(_,a(Key1,Info1),_),Info):-  
    Key=Key1,!,  
    Info/=Info1,!,  
    fail.
```

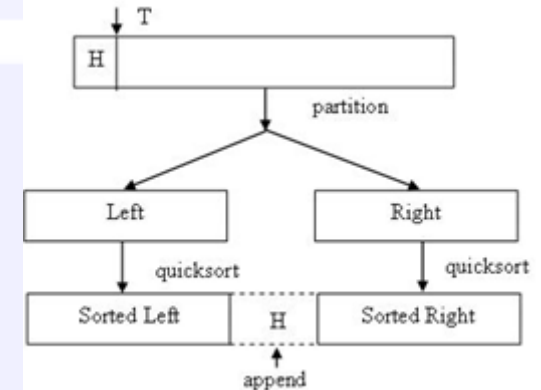
Difference lists

- Difference lists
 - Pair of arguments (First,Last) or First\Last, pointing to the first and respectively AFTER the last element of the list
 - Allow to speed up execution (avoid second traversal of list)
 - Needs special initial call => make the warm up call by setting Last on call to []
 - What if instead of [] we have some content?
 - Stop condition: empty list as a difference; therefore, the difference between the same variable (L,L).

Quicksort

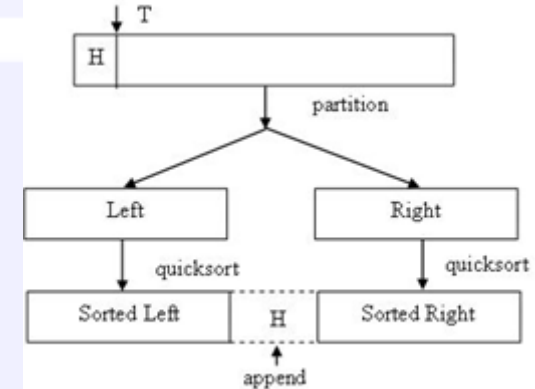
```
partition(X, [H|T], [H|Left], Right) :-
    H < X, !,
    partition(X, T, Left, Right).
partition(X, [H|T], Left, [H|Right]) :-
    partition(X, T, Left, Right).
partition(_, [], [], []).
```

```
quicksort([H|T], Result) :-
    partition(H, T, Left, Right),
    quicksort(Left, SortedLeft),
    quicksort(Right, SortedRight),
    append(SortedLeft, [H|SortedRight], Result).
quicksort([], []).
```



Quicksort - analysis

- A divide et impera strategy
- Cost analysis per cases:
 - $O(n \lg n)$ in best and average case (why?)
 - $O(n^2)$ in worst (which?)
- Multiplicative constant larger than in imperative implementations! Why?
- How can be avoided?



```

quicksort([H|T], Result) :-
    partition(H, T, Left, Right),
    quicksort(Left, SortedLeft),
    quicksort(Right, SortedRight),
    append(SortedLeft, [H|SortedRight], Result) .//how can
                                                    //we skip it?

quicksort([], []).
    
```

Quicksort – with DL

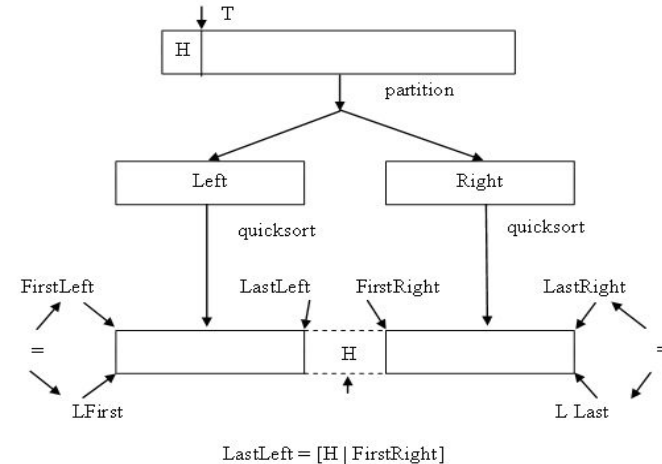
%qsort_LD1/3

%qsort_LD1(in_list,first_olist,last_olist)

```
qsort_LD1 ([H|T],LFirst,LLast):-
    partition(H,T,Left,Right),
    qsort_LD1(Left,FirstLeft,LastLeft),
    qsort_LD1(Right,FirstRight,LastRight),
    LFirst=FirstLeft,
    LLast=LastRight,
    LastLeft=[H|FirstRight].
```

```
qsort_LD1 ([],L,L).
```

Q: partition should not return difference lists? How should we do that? Oral discussion.



Quicksort – with DL (default unifications)

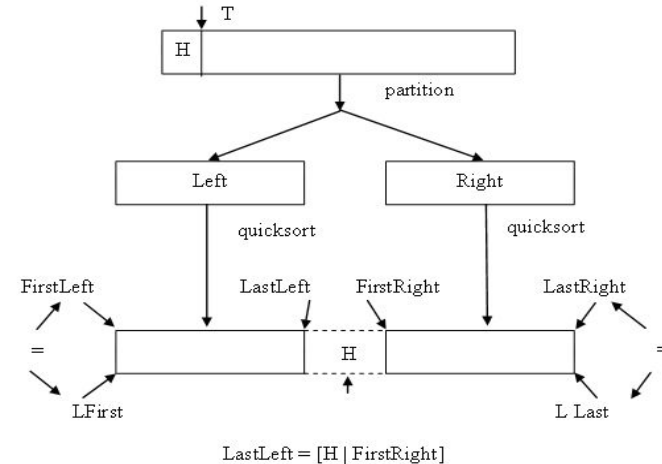
```
%qsort_LD2/3
```

```
%qsort_LD2(in_list,first_olist,last_olist)
```

```
qsort_LD2([H|T],LFirst,LLast):-  
    partition(H,T,Left,Right),  
    qsort_LD2(Left,LFirst,[H|Right]),  
    qsort_LD2(Right,Right,LLast).  
qsort_LD2([],List,List).
```

Q needs special initial call, to initialize last to empty:

```
quicksort2(List,Result):-  
    qsort_LD2(List,Result,[]).
```



Queues with D

(section 15.4 in The art of Prolog – Shapiro)

```
%enqueue/5
%enqueue (el2EnQ, firstQBefore, lastQBefore,
          firstQAfter, lastQAfter)

enqueue (Eel, First, [Eel | Last], Fist, Last) .
```

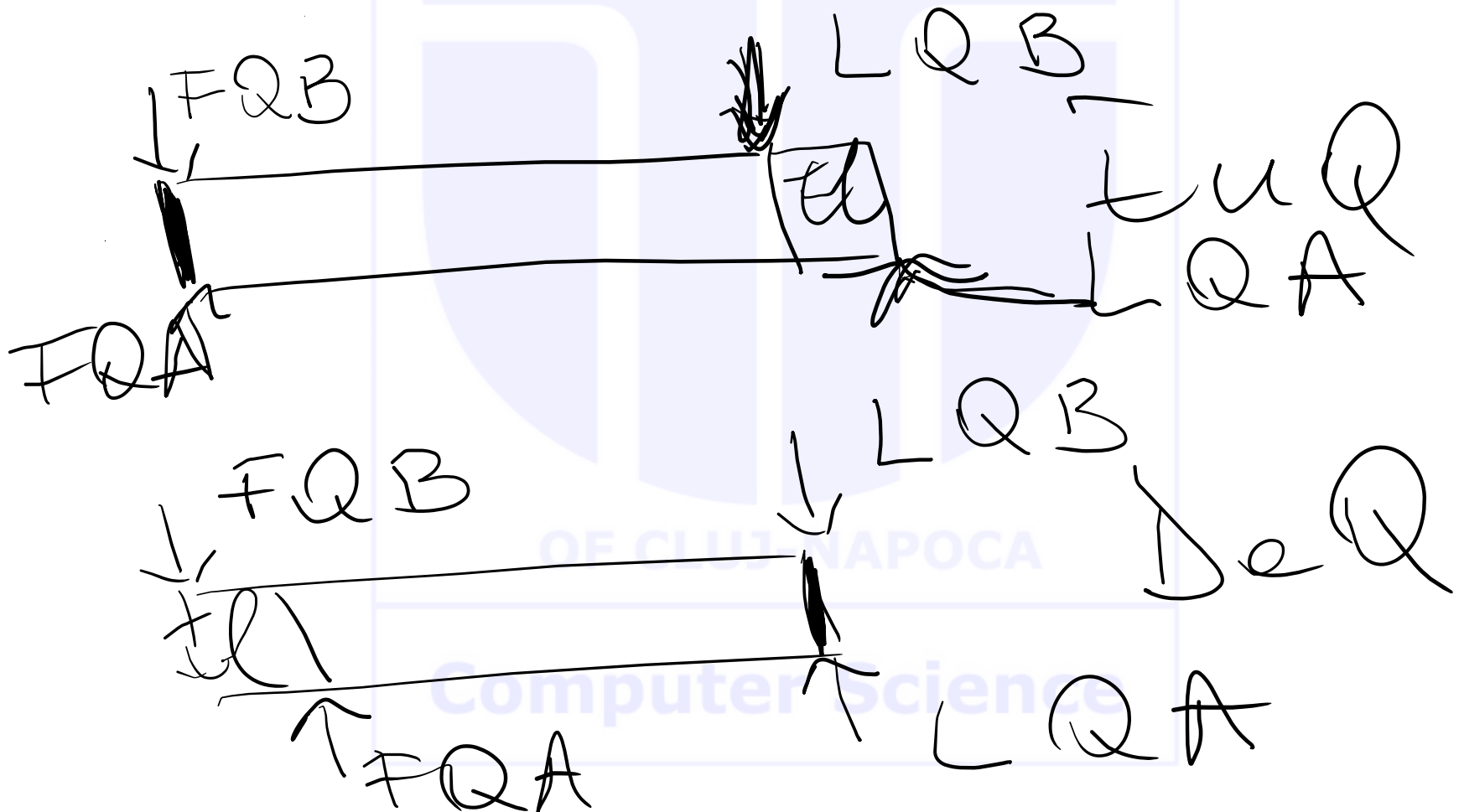
Means:

```
enqueue (Eel, FQB, LQB, FQA, LQA) :-
    FQA=FQB//after adding in the Q, it starts in the same place
    LQB=[H|LQA], //before adding, the list ends IN FRONT
    H= Eel.      //of the item to be added
```

How to query it?

```
?-enqueue (Item, FB, LB, FA, LA) .
```

In the usual employment FB and LB are the First/Last element in the Queue before the call (known arguments), and FA and LA the First/Last element in the Queue after the call (unknown arguments at call time).



Queues with D

(section 15.4 in The art of Prolog – Shapiro)

```
%dequeue/5
%dequeue (el2DeQ,    firstQBefore, lastQBefore,
                firstQAfter, lastQAfter)

dequeue (Eel, [Eel | First], Last, Fist, Last) .
```

Means:

```
dequeue (Eel, FQB, LQB, FQA, LQA) :-
    FQB=[H|FI], //before removing from the Q, the Q has
    Eel=H,      //extracted element is the front of the Q
    FI=FQA,     //some items which is removed, followed
                // by the reminding = FirstQueueAfter
    LQA=LQB.    //after removing, the list ends where it ended
                //as removal is from front
```

How to query it?

```
?-dequeue (Item, FB, LB, FA, LA) .
```

In the default meaning, Item gets instantiated at the end of the execution. What happens if we ask to dequeue some specific element (say 7), yet, it is NOT the first in the queue? That is, we specify Item.

Types of lists

- Regular (ended in [])
- Ended in variable
- Difference lists
- Write specific predicates to make transformations among each of the 3 known types of lists (section 7.3 of the textbook):
 - **Regular to ended in variable**
 - Regular to difference
 - Ended in variable to regular
 - **Ended in variable to difference**
 - Difference to regular
 - Difference to ended in variable

Deep lists

- What is a deep list?

$L = [1, 1, [2, 2, 2, [3, 3, [4], 3], 2, 2, [3, [4, [5, 5, 5]]], 2]]$

- Levels? How do we know?

- Flatten deep lists. Meaning?

$L = [1, 1, 2, 2, 2, 3, 3, 4, 3, 2, 2, 3, 4, 5, 5, 5, 2]$

- How, without knowing structure/#of levels?

Flatten Deep lists

```
%deep2flat/2
%deep2flat(deep_list, flat_list).

deep2flat([], []) :- !.
deep2flat(H, [H]) :- atomic(H), !.
deep2flat([H|T], [H|FlattenedTail]) :-
    atomic(H), !,
    deep2flat(T, FlattenedTail).
deep2flat([H|T], FlattenedList) :-
    deep2flat(H, FlattenedHead),
    deep2flat(T, FlattenedTail),
    append(FlattenedHead, FlattenedTail, FlattenedList).
```

```
L=[1,1,[2,2,2,[3,3,[4],3],2,2,[3,[4,[5,5,5]]],2]]
```

Flatten Deep lists – with difference lists - explicit

```
%deep2flat_dl/3
%deep2flat_dl(deep_list,first_flat_list, last_flat_list).

deep2flat_dl([],First,Last):-!,First=Last.
deep2flat_dl(H,First,Last):-  atomic(H),!,  //[H]
                                First=[H|Last].

deep2flat_dl([H|T],First,Last):-
    atomic(H),!,
    First=[H|FirstT],
    deep2flat_dl(T, FirstT,LastT),
    Last=LastT.

deep2flat_dl([H|T],First,Last):-
    deep2flat_dl(H,FirstH,LastH),
    deep2flat_dl(T,FirstT,LastT),
    First=FirstH,
    LastH= FirstT,
    Last=LastT.
```

Flatten Deep lists – with difference lists - default

```
%deep2flat_dl/2
%deep2flat_dl(deep_list,first_flat_list, last_flat_list).

deep2flat_dl([],Last,Last):-!.
deep2flat_dl(H,[H|Last],Last):-atomic(H),!.
deep2flat_dl([H|T],[H|FlattenedTail],Last):-
    atomic(H),!,
    deep2flat_dl(T,FlattenedTail,Last).
deep2flat_dl([H|T],FlattenedList,Last):-
    deep2flat_dl(H,FlattenedList,Int),
    deep2flat_dl(T,Int,Last).

flatten(Deep,Flat):-
    deep2flat_dl(Deep,Flat,[]).
```

Flatten Deep lists – homework

- Try to flatten a deep list taking just elements with various properties:
 - Heads of lists ([1,2,3,4,3,4,5])
 - Atomic elements on last positions of lists ([4,3,5,2])
 - All atomic elements from lists at **odd**/even levels ([2,2,2,4,2,2,4,2])
 - Heads of lists at odd/**even** levels ([1,3,3,5])
 - Atomic elements on odd/even positions on any list
 - ...imagine 😊
 - (section 8.3 of the textbook)

$L = [1, 1, [2, 2, 2, [3, 3, [4], 3], 2, 2, [3, [4, [5, 5, 5]]], 2]$

Creating/accessing structures

Built-in predicates

- `functor(T, F, N)`
 - means T is a structure named F and arity N
 - 2 functionalities:
 - **if** `nonvar(T)` **then** F matches the functor of T
N matches the arity of T
 - **if** `var(T)` **and** `nonvar(F)` **and** `nonvar(N)`
then T becomes a structure
with name F and N arguments

functor(T, F, N)

if nonvar(T) **then** F matches the functor of T
N matches the arity of T

?-functor(a+b, F, N) .

Yes, F=+, N=2

?-functor([a,b,c], F, N) .

Yes, F=., N=2 (why 2? Which ones?)

?-functor([a,b,c], F, 3) .

Fails. Why?

?-functor(a, F, N) .

Yes, F=a, N=0

?-functor(f(a,b), F, N) .

Yes, F=f, N=2 .

$\text{functor}(T, F, N)$

if $\text{var}(T)$ **and** $\text{nonvar}(F)$ **and** $\text{nonvar}(N)$
then T becomes a structure with name F and N arguments

?-functor($F, f, 3$) .

Yes, $F=f(A, B, C)$.

?-functor($F, +, 2$) .

Yes, $F=A+B$

Creating/accessing structures

Built-in predicates

- `arg(N, T, A)`
 - means the N^{th} argument of `T` is `A`
 - `N` and (partially)`T` must be instantiated

```
?-arg(2, f(a,b), X) .
```

```
Yes, X=b.
```

```
?-arg(1, a+b+c, X) .
```

```
Yes, X=a
```

```
?-arg(3, a+b+c, Y) .
```

```
No. WHY?
```

```
?-arg(2, a+b+c, Z) .
```

```
Yes, Z=b+c
```

```
?-arg(1, [a,b,c], X) .
```

```
Yes, X=a
```

```
?-arg(2, [a,b,c], Y) .
```

```
Yes, Y=[b,c]
```

Creating/accessing structures

Built-in predicates

- $X = . . L$ INFIX predicate (a.k.a. UNIV)
 - means list L contains as first element the functor of X, followed by arguments of X
 - 2 functionalities:
 - **if** var(X) **then** L is used to construct the appropriate structure of X; the head of L must be an atom, and becomes the functor of X
 - **if** nonvar(X) **then** functor of X matches the first element (head) of L, while the arguments of X matches with the tail of the list (left to right arguments of X, left to right in tail of L)

$X = \dots L$

if var(X) **then** L is used to construct the appropriate structure of X; the head of L must be an atom, and becomes the functor of X

?-X=..[a,b,c,d].

Yes, X=a(b,c,d)

?-X=..[member,a,[b,c]].

Yes, X=member(a,[b,c]).

$X = \dots L$

if nonvar(X) **then** functor of X matches the first element (head) of L, while the arguments of X matches with the tail of the list (left to right arguments of X, left to right in tail of L)

?-f(a,b,c)=..X.

Can we ask this way (with X on the left)?

Yes, X=[f,a,b,c].

?-append([H|T],L,[H|R])=..X.

Yes, X=[append,[H|T],L,[H|R]].

?-(a+b)=..L.

Yes, L=[+,a,b].

?-(a+b+c)=..L.

Yes, L=[+,a,b+c].

?-[a,b,c,d]=..[H|T].

Can we ask this way (with list on the left)?

Yes, H=., T=[a,[b,c,d]]

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #9, 2021

Cluj-Napoca

Computer Science

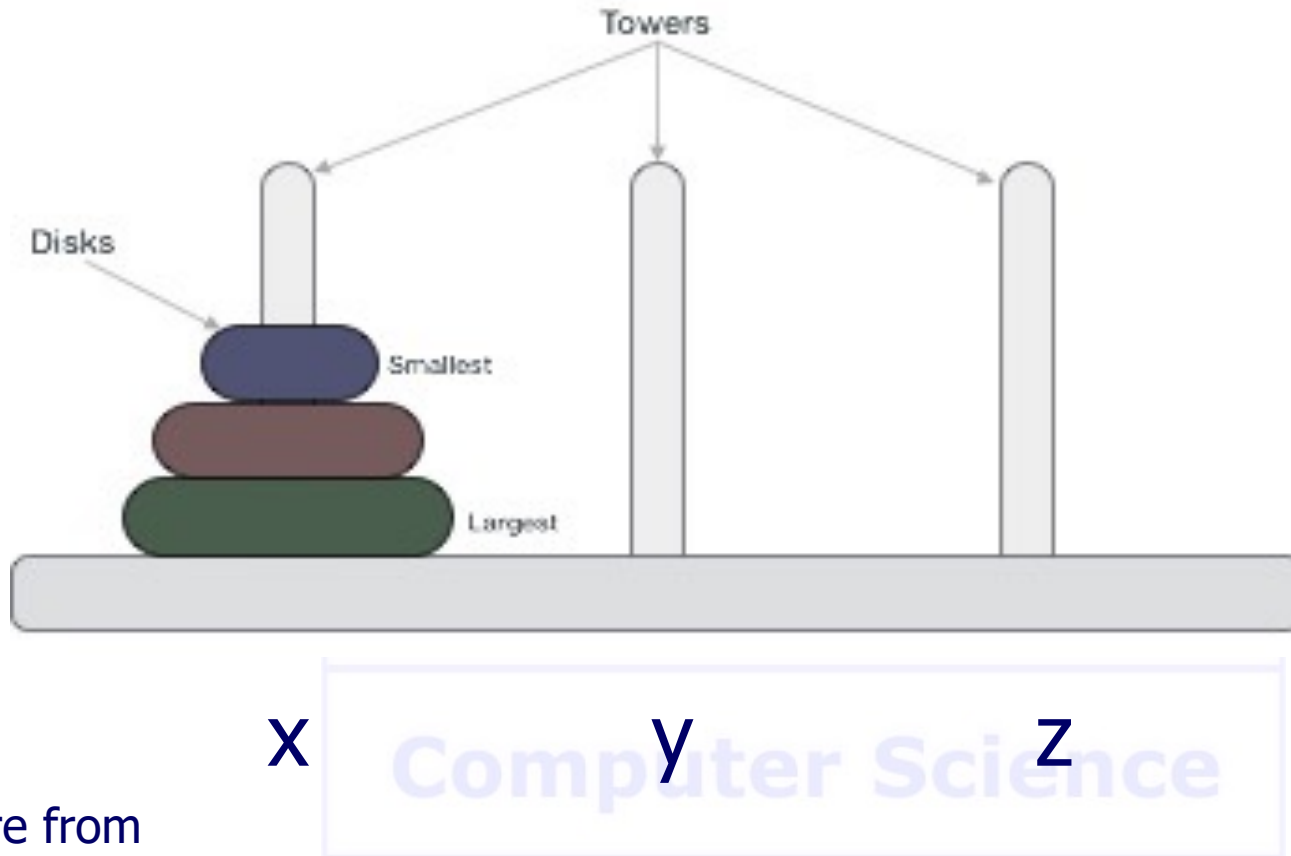
Agenda

- Built in predicates – review
 - `functor` and `arg`
 - `=..`
- Utilization on complex tasks
 - Towers of Hanoi
 - Substructure change
- Quiz 3 analysis

Built in predicates

- functor and arg
- = ..

Towers of Hanoi – the problem



Picture from

https://www.tutorialspoint.com/data_structures_algorithms/tower_of_hanoi.htm

Towers of Hanoi - formulation

- Given n disks and 3 poles
- Move the disks from X to Z
- At any time the constraints are:
 - One at a time
 - Just the top disk is accessible
 - on any pole, any disk has the diameter smaller than the diameter of the disk on top of which it stays

Towers of Hanoi - rephrasing

Start with n disks on X and end with n disks on Z

(n,Y)
 $X \rightarrow Z$ means $X \xrightarrow{(n-1,Z)} Y$
 $X \rightarrow Z$
 $(n-1,X)$
 $Y \rightarrow Z$

BUT this is almost Prolog code:

```

move_discs(0,_,_,_):-!.
move_discs(N,X,Y,Z):- N1 is N-1,
    move_discs(N1,X,Z,Y),
    move_disc(X, Z),
    move_discs(N1,Y,X,Z) .
    
```

Towers of Hanoi - realization

- Disc representation:

x(1).

⋮

x(n-1).

x(n). //n is NOT a variable, is a value.

- How is it realized?

```
init_pole(_, []) :- !.
```

```
init_pole(P, [N|L]) :-
```

```
    D = .. [P, N], //P name of pole, as x in example; N size
```

```
    asserta(D), //put on top
```

```
    init_pole(P, L).
```

- Where L is a list of integers in decreasing order

```
init_list(1, [1]) :- !.
```

```
init_list(N, [N|Rest]) :- N1 is N-1,
```

```
    init_list(N1, Rest).
```

Towers of Hanoi – moving one disk

- Move one disc from Input Pole to Output Pole:

```
move_disc(IP,OP) :-
```

```
    DI=..[IP,Size],      //IP name of input pole, Size diam of disk
```

```
    retract(DI),!,       //extract from top
```

```
    DO=..[OP,Size],      //OP name of output pole, Size diam of disk
```

```
    asserta(DO).         //put on top
```

- If the predicate base has:

```
x(3).
```

```
x(4).          y(6).
```

```
x(5).          y(7).
```

And call `move_disc(x,y)`. After the call:

```
    y(3).
```

```
x(4).          y(6).
```

```
x(5).          y(7).
```

Towers of Hanoi – complete code

```
move_discs(0,_,_,_):-!.
move_discs(N,X,Y,Z):- N1 is N-1,
    move_discs(N1,X,Z,Y),
    move_disc(X, Z),
    move_discs(N1,Y,X,Z).

move_disc(IP,OP):-
    DI=..[IP,Size],
    retract(DI),!,
    DO=..[OP,Size],
    asserta(DO).

init_pole(_,[]):-!.
init_pole(P,[N|L]):-
    D=..[P,N],
    asserta(D),
    init_pole(P,L).
```

Towers of Hanoi – complete code

```
move_discs(0,_,_,_):-!.  
move_discs(N,X,Y,Z):- N1 is N-1,  
    move_discs(N1,X,Z,Y),  
    move_disc(X, Z),  
    move_discs(N1,Y,X,Z).
```

```
move_disc(IP,OP):-  
    DI=..[IP,Size],  
    retract(DI),!,  
    DO=..[OP,Size],  
    asserta(DO).
```

```
init_pole(_,[]):-!.  
init_pole(P,[N|L]):-  
    D=..[P,N],  
    asserta(D),  
    init_pole(P,L).
```

// INITIAL call

```
hanoi(N):-init_list(N,List),  
    init_pole(input,List),  
    move_discs(N,input,int,out).
```

Substituting expressions

- Given a (complex) expression, it is requested to replace a given subexpression with another one.
- `subst (OldExpr, NewExpr, OldSubExpr, NewSubExpr) .`
- Example: in a large arithmetic expression replace $(7-2*x)$ wherever it occurs with another one, say $(2*y+3/z)$
- 2 solutions
 - `=..`
 - functor + arg

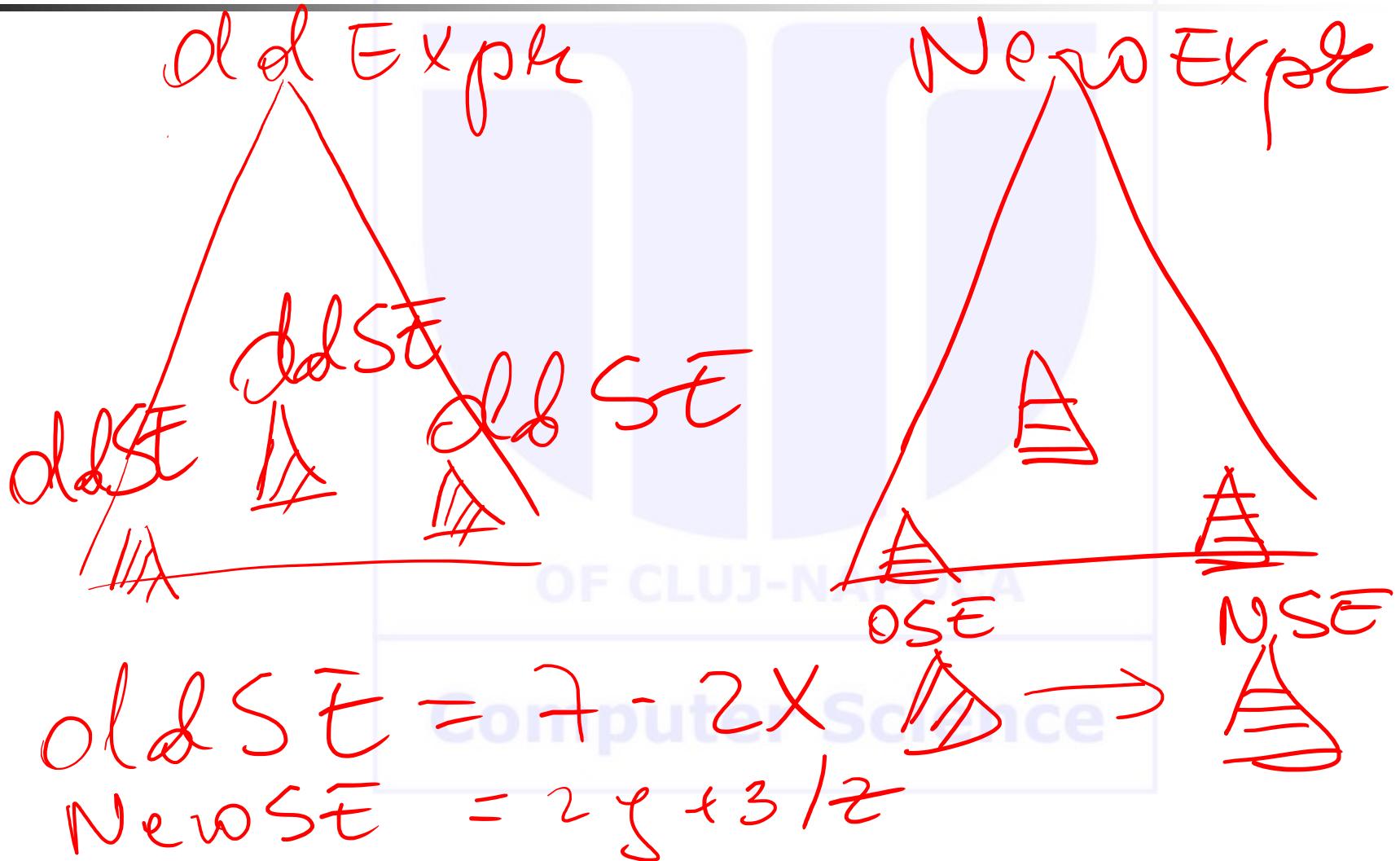
```
//subst/4
```

```
//subst (OldExpr, NewExpr, OldSubExpr, NewSubExpr) .
```

```
//subst_args/4
```

```
//subst_args (ListOldExpr, ListNewExpr, OldSubExpr, NewSubExpr) .
```

Picturing the arguments ...



Substituting expressions =..

```
subst (Old, New, Old, New) :- !. //in case the whole expression is represented
                                //by only the sub-expression to be replaced,
                                //the old one is replaced by the new one

subst (Val, Val, _, _) :-      //in case the expression is an atomic value
    atomic(Val), !.           //it remains unchanged

subst (Val, NewVal, OldSubExpr, NewSubExpr) :-
    Val = .. [F|Args], //extract from the old expression its functor and the list
                        //of arguments
    subst_args (Args, NewArgs, OldSubExpr, NewSubExpr),
    //take the old list of arguments and process it with substitution
    NewVal = .. [F|NewArgs]. //create the new expression with the same functor
                             // and the new list of args, updated by substitutions

subst_args ([], [], _, _).

subst_args ([Arg|Args], [NArg|NArgs], Old, New) :-
    subst (Arg, NArg, Old, New), //as Arg is an expression, go back to the
                                //other predicate
    subst_args (Args, NArgs, Old, New). //continue with the rest of
                                        //arguments, tail of list
```

Substituting expressions =.. Just code, comments removed

```
subst (Old, New, Old, New) :- !.  
subst (Val, Val, _, _) :-  
    atomic (Val), !.  
subst (Val, NewVal, OldSubExpr, NewSubExpr) :-  
    Val = .. [F | Args],  
    subst_args (Args, NewArgs, OldSubExpr, NewSubExpr),  
    NewVal = .. [F | NewArgs].  
  
subst_args ([], [], _, _).  
subst_args ([Arg | Args], [NArg | NArgs], Old, New) :-  
    subst (Arg, NArg, Old, New),  
    subst_args (Args, NArgs, Old, New).
```

Qs:

1. Order of processing the structure?
2. Where the execution ends?

Substituting expressions functor+arg

```

subst (Old, New, Old, New) :- !.           //same as before
subst (Val, Val, _, _) :-                  //same as before
    atomic (Val), !.
subst (Val, NewVal, OldSubExpr, NewSubExpr) :-
    functor (Val, F, N), //extract from the old expression Val its functor F
                          // and the number of arguments N
    functor (NewVal, F, N), //create a new expression NewVal from the
                          // known functor F and N arguments, free variables at the time
    subst_args (N, Val, NewVal, OldSubExpr, NewSubExpr) .
                          //for each of the arguments, 1, 2, ..., N, process it one at a time
subst_args (0, _, _, _, _) :- !. //arg 0 needs no change
subst_args (N, Val, NewVal, Old, New) :-
    arg (N, Val, OldArg), //extract the Nth arg of the old expression
    arg (N, NewVal, NewArg), //set the Nth arg of the new expression; a var. at the time
    subst (OldArg, NewArg, Old, New), //as OldArg is an expression, go back
                                      //to the other predicate and make the same processing
    N1 is N-1, //process next the previous argument
    subst_args (N1, Val, NewVal, Old, New) . //continue with the rest of args

```

Substituting expressions functor+arg

Just code, comments removed

```
subst (Old, New, Old, New) :- !.
subst (Val, Val, _, _) :-
    atomic (Val), !.
subst (Val, NewVal, OldSubExpr, NewSubExpr) :-
    functor (Val, F, N),
    functor (NewVal, F, N),
    subst_args (N, Val, NewVal, OldSubExpr, NewSubExpr) .

subst_args (0, _, _, _, _) :- !.
subst_args (N, Val, NewVal, Old, New) :-
    arg (N, Val, OldArg),
    arg (N, NewVal, NewArg),
    subst (OldArg, NewArg, Old, New),
    N1 is N-1,
    subst_args (N1, Val, NewVal, Old, New) .
```

Qs:

1. Order of processing the structure?
2. Where the execution ends?

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #10, 2021

Cluj-Napoca

Computer Science

Agenda

- Graphs
 - Representations
 - Search for a path
 - One way
 - V1
 - V2
 - V3 – nonmonotonic reasoning!
 - Best way
 - All ways
 - Restricted way

Graphs - Representations

- Traditional representations:
 - Adjacency matrix – not efficient here
 - Adjacency lists – 2 way of doing this
- Neighbor list:
 - Knowledge base with facts of nodes, list of neighbors
- Adjacency list
 - Knowledge base with edge pairs

Graphs - Representations

- Neighbor list – Knowledge base with facts of nodes, list of neighbors

```
//neighb/2
```

```
//neighb(Vertex,List_of_neighb) .
```

```
neighb(a,[b,c]) .
```

- Directed graphs? What does it change?
- Isolated nodes?

```
neighb(z,[]) .
```

Graphs - Representations

- Adjacency list - Knowledge base with edge pairs

```
//edge/2
```

```
//edge(vertex1,vertex2) .
```

```
edge(a,b) .
```

```
edge(a,c) .
```

- UNDirected graphs? Add a predicate to ask searching both ways.

```
//is_edge/2
```

```
//is_edge(vertex1,vertex2) .
```

```
is_edge(X,Y) :-
```

```
    edge(X,Y) ; //check one way
```

```
    edge(Y,X) . //and the other way
```

- Isolated nodes?

```
edge(z,nil) . //not a built-in way; just assume an edge to some dummy vertex
```

Search for a path in a graph

- Example from Clocksin & Mellish book
- Finding the way out from the labyrinth
- Labyrinth =
 - a set of rooms
 - With doors linking them
 - Unique entrance
 - One objective = way out = reach a given room
 - You **know** you reach it just **AFTER** you entered the room! (that is, you cannot define the problem in terms of "go to room x" as you don't know what x would be beforehand. You have this ability just after you reach there).

Search for a path in a graph

Labyrinth representation

- Is a graph
 - Rooms = vertices
 - Doors = edges
- Is it directed? Why/why not?
- Enter the labirynth from room a.

```
is_door(a,b) .
```

```
is_door(b,c) .
```

```
is_door(b,e) .
```

```
is_door(c,d) .
```

```
is_door(d,e) .
```

```
is_door(e,f) .
```

```
is_door(e,g) .
```

```
is_objective (g) .
```

Search for a path in a graph

Labyrinth way out

- Search the way out = check every possible root. When deadlock, backtrack. Prolog advantage: backtrack is there for free.

- Graph undirected => add necessary predicate

```
is_pass(X,Y):-
```

```
    is_door(X,Y);
```

```
    is_door(Y,X).
```

```
//search/3
```

```
//seach(start,objective,path).
```

```
search(X,Y,Way):-
```

```
    try(X,Y,[X],Way), //try a path from X to Y with the partial path  
                        //containing just the starting vertex at this point
```

```
    is_objective(Y),!. //why not start with this?
```

Call it with:

```
?-seach (a,X,Way_Out). //is X safe here? Not Y?
```

Search for a path in a graph

Labyrinth way out – main predicate (v1)

```
//try/4  
//try(from_vertex,to_vertex,partial_path,final_path)  
try(X,X,L,L) .  
try(X,Y,Thread,Way):-  
    is_pass(X,Z) ,  
    not(member(Z,Thread)) ,  
    try(Z,Y,[Z|Thread],Way) .
```

- Order of clauses? Why?
- `is_pass(X,Z)`? Why not `is_door(X,Y)`?
- Pattern composition `[Z|Thread]` in the recursive call? Is it correct? Shouldn't it be in the head of the rule? Why?
- Make the analysis of the calling tree. When does try stop? When does try eventually close?
- What answer would we get to the initial query? Discussion on `Way_Out`.
- `?-seach(a,X,Way_Out)`.
- Ask ANY question you have at this point. Is important to understand NOW!

Search for a path in a graph

Labyrinth way out – main predicate (v2)

```
//try/4  
//try(from_vertex,to_vertex,partial_path,final_path_ordered)  
try(X,X,L,[X]).  
try(X,Y,Thread,[X|L]):-  
    is_pass(X,Z),  
    not(member(Z,Thread)),  
    try(Z,Y,[Z|Thread],L).
```

- Initial call?
- Why do we need both `Thread` and `L`? Do we really need both?
- Just use the 4th arg? With `not(member(Z,L))`?
- Is it possible? What `Thread` and `L` do contain? Make the difference
Understand and NEVER make confusions. Learn on the meaning of pattern composition on:
 - The head of the rule
 - Recursive call

Search for a path in a graph

Labyrinth way out – main predicate (v3)

```
//try/3
//try(from_vertex,to_vertex,path).
try(X,X,[X]).
try(X,Y,[X|L]):-
    is_pass(X,Z),
    accept(Z), //can Z be part of the thread
    try(Z,Y,L).
//accept/1
//accept(vertex).
accept(X):-
    seen(X),!,
    fail.
accept(X):-
    assert(seen(X)).
accept(X):-
    retract(seen(X)),!
    fail.
```

Search for a path in a graph

Labyrinth way out – main predicate (v3)

- THIS is the nonmonotonic reasoning part
- Is the predicate which states if and WHEN a vertex makes part of the solution
- Reasoning is nonmonotonic (part of default logic) as the assumptions are not nomoton
 - During the reasoning,
 - If an assumption does NOT already take part of the reasoning
 - And does NOT contradict ANY other assumption
 - Is added to the knowledge base
 - If at a latter point
 - If the reasoning cannot be closed (completed), chance is made to attempt some reasoning WITHOUT that piece of knowledge
 - Therefore, quantity of knowledge is not monotonic

accept (X) :-

seen (X) , ! , //is the contradiction part! The ONLY contradiction is that the vertex is
fail. //ALREADY in the solution. If there, don't loop; fail to backtrack!

accept (X) :-

assert (seen (X)) . //no contradiction, add it in the solution

accept (X) :-

retract (seen (X)) , ! //cannot conclude with X in solution, remove and
fail. //backtrack to try WITHOUT it!

Labyrinth way out

v3 / v2 comparative analysis

```
Code v3                                     //comments v2
accept (X) :-
    seen (X) , ! ,                          //if member (Z, Thread) succeeds, then
    fail.                                   //not (member (Z, Thread) ) fails and execution in BOTH versions
                                           //backtrack to a different neighbor of the current vertex

accept (X) :-
    assert (seen (X) ) . //not (member (Z, Thread) ) succeeds, hence Z could be
                           //added to the current attempted solution

accept (X) :-
    retract (seen (X) ) , ! , //is there any point in v2 where we do this?
    fail.                       //if so, where?
                                //if not, are solutions similar?

//v3 comments
//although X was in the solution
//at some point since there is no final way
//decide to remove it from path
```

- Q on nonmonotonic reasoning. We'll see it soon again!

Graphs – Find the Best way

```

a_way(V,V,Thread,Th_length):-
    is_objective(V),!
    retract(best(_,_)),!
    asserta(best(Thread,Th_length),
    fail. // cut above. So, where does fail backtracks us? Mistake?

a_way(V1,V2, Thread,Th_length):-
    best(BThread,BTh_length),
    Th_length1 is Th_length +1
    Th_length1<BTh_length,
    is_pass(V1,V3),
    not(member(V3,Thread)),
    a_way(V3,V2, [V3|Thread],Th_length1).

best_way(V1,V2,Thread):-
    assert(best([],1000)),
    a_way(V1,V2,[V1],1).

best_way(_,_,Thread):-
    retract(best(Ttherad,_)).
  
```

Graphs – Find all ways

- Assume neighbor representation:

```
neighb(b, [a, c, e]) .
```

```
⋮
```

```
//a_way/3
```

```
//a_way(first_vertex, last_vertex, path) .
```

```
a_way(V, V, [V]) .
```

```
a_way(V1, V2, [V1 | Rest]) :-
```

```
    neighb(V1, L) ,
```

```
    ways(L, V2, Rest) .
```

```
//ways/3
```

```
//ways(list_of_vertices_to_start_from, last_vertex, path) .
```

```
ways([V1 | _], V2, Way) :-
```

```
    a_way(V1, V2, Way) .
```

```
ways([_ | Rest], V2, Way) :-
```

```
    ways(Rest, V2, Way) .
```

- Q: How are loops avoided? How/when does this work?

Graphs – Find restricted way

- Assume back the edge representation.

```
is_way_obj(NX,NY,Way):-
```

```
    nonvar(NX), //meaning? Why needed?
```

```
    try(NX,NY,[NX],Way), //any try from v1, v2, v3
```

```
    is_objective(NY) .//why here the test? Not before try? Would be better?
```

```
is_restricted_way(NX,NY,Restrictions,Way):-
```

```
    is_way_obj(NX,NY,Way),
```

```
    is_in_order(Restrictions,Way) .//how else could we do?
```

```
is_in_ord([NX|TX],[NX|TY]):-!,//what is this cut cutting?
```

```
    is_in_ord(TX,TY).
```

```
is_in_ord([NX|TX],[_|TY]):-!,
```

```
    is_in_ord([NX|TX],TY).
```

```
is_in_ord([],_).
```

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #11, 2021

Cluj-Napoca

Computer Science

Agenda

- More built in predicates (and relationship to graphs)
 - findall and more
- Constructs and relationship with graphs
 - For all is true
 - Application – same graph
- Search for a path in graphs
 - Dfs
 - Bfs (and other bfs – next time)

Built-in predicates

their utility in graph searching

- `findall` – selects all items (and only those items) with a certain property from a knowledgebase
- Knowing the predicate to use it appropriately
- Knowing HOW is implemented
 - to utilize the strategy in graph problems

findall

(example from Clocksin and Mellish)

- Suppose we have a knowledge base of some drinks lovers:

```
//likes/2
```

```
//likes(person,drink_person_likes).
```

```
likes(bill,wine).
```

```
likes(dick,beer).
```

```
likes(harry,beer).
```

```
likes(john,beer).
```

```
likes(peter,wine).
```

```
likes(tom,beer).
```

```
?-findall(X,likes(X,beer),L).
```

```
Yes, L=[dick,harry,john,tom].
```

findall – implementation

```
//findall/3
//findall(collected_var, collector_predicate, collector_var).

findall(X,G,_):-                // uses an auxiliary knowledge base (akb)
    asserta(found(end)),        // marker for the bottom of the akb
    G,                          // = call(G), G is a predicate; its exe instantiates
                                // some argument X.
    asserta(found(X)),          // puts G's instantiated argument on top of akb
    fail.                       // request for backtrack to G.
findall(_,_,L):-                // reach here when G fails on backtrack
    collect_found([],L).         // starts collecting from the akb, initializing the partial
                                // result to empty list.

collect_found(P,L):-
    get_next(X),!,              // asks for one element in akb; X hidden; known AFTER call
    collect_found([X|P],L).      // adds it in front and go recurse
collect_found(L,L).

get_next(X):-
    retract(found(X)),!,         // extracts from top of the akb
    X=/_end.                     // succeeds if a genuine data; fails when get to end.
```

findall – implementation

```
//findall/3
//findall(collected_var, collector_predicate, collector_var).

findall(X,G,_):-
    asserta(found(end)),
    G,
    asserta(found(X)),
    fail.
findall(_,_,L):-
    collect_found([],L).

collect_found(P,L):-
    get_next(X),!,
    collect_found([X|P],L).
collect_found(L,L).

get_next(X):-
    retract(found(X)),!,
    X/=end.
```

findall – execution

```
findall(X,G,_):-  
    asserta(found(end)),  
    G,  
    asserta(found(X)),  
    fail.  
findall(_,_,L):-  
    collect_found([],L).
```

```
collect_found(P,L):-  
    get_next(X),!,  
    collect_found([X|P],L).
```

```
collect_found(L,L).
```

```
get_next(X):-  
    retract(found(X)),!,  
    X/=end.
```

//initial kb

```
likes(bill,wine).  
likes(dick,beer).  
likes(harry,beer).  
likes(john,beer).  
likes(peter,wine).  
likes(tom,beer).
```

//akb

```
found(tom).  
found(john).  
found(harry).  
found(dick).  
found(end).
```

created bottom-up↑ collected top-down↓

findall – related

- **General call:** `?-findall(X,G,L)`
 where X is a variable
 G a predicate and X occurs in G
 L would collect all X's so that G's execution succeeds on X
- Creates the list of X's that satisfy G, in the order of occurrence in the original knowledge base
- Associate predicates:
- `setof(X,G,S)`
- Creates the set of terms (standard order, without duplicates; represented also as list) of X's that satisfy G. If none, fails.
- Differences to `findall`:
 - Order: standard vs as found in the knowledge base
 - Duplicates: no vs found in the knowledge base
 - No term: fails vs empty list
- `bagof(X,G,S)`
 - Same as `setof` BUT list NOT ordered + may have duplicates
 - So same as `findall` but fails if no term matches

`findall(X,G,L):-bagof(X,G,L),!.`

`findall(_,_,[])`

For all is true technique

- Construct to check if `for_all Predicate1
is_true Predicate2`
- Aims to identify/verify whether in all cases when P1 is true, P2 is true as well. Thus, checks $P1 \Rightarrow P2$.
- Template looks like: `for_all_is_true(X,Y):-`

```

X,
not(Y) , !,
fail.

```

```

for_all_is_true(_,_) .

```

```

for_all_is_true(X,Y):-

```

```

X,           //calls X, a predicate, whose execution may instantiate hidden args
not(Y) , !,  //calls Y in the same context. If succeeds, its negation fails,
             //and backtracks to X who is executed again, with a new instance of args
fail.        //the first context of X where Y fails, not(Y) succeeds, and ! is
             //irreversible crossed and the predicate fails overall.

```

```

for_all_is_true(_,_) . //if we reached here, in all contexts when X holds true, Y
                        //succeeds as well, therefore,  $X \Rightarrow Y$ 

```

For all is true in graph context

- Given graphs G1 and G2 with neighbor and edge representations respectively, do they represent the same graph?

G1

neighbor(Node, List_of_Neighbors)

- G1 and G2 same means $G1 \Leftrightarrow G2$ which is $(G1 \Rightarrow G2) + (G2 \Rightarrow G1)$

for_all edges_in_G1

is_true edge_in_G2

G2

edge(Node1, Node2)

$(G2 \Rightarrow G1)$

for_all edges_in_G2

is_true edge_in_G1

- Define the edges in the 2 representations:

An edge in G1:

```
is_edge_1(X, Y) :- //to find all edges in G1, call is nondeterminist (X,Y, free)
    neighbor(X, L), //finds first pair first (and next on backtrack), instantiates both X and L.
    member(Y, L) .//nondeterm call on member; instantiate Y, one at a time, eventually all.
```

An edge in G2:

```
is_edge_2(X, Y) :-
    edge(X, Y) ;
    edge(Y, X) .
```

Same graph?

- $(G1 \Rightarrow G2)$ $(G2 \Rightarrow G1)$
`for_all edges_in_G1` `for_all edges_in_G2`
`is_true edge_in_G2` `is_true edge_in_G1`
- Denote the `for_all_is_true` as `eq1` and `eq2` depending on the implication $(G1 \Rightarrow G2)$ respectively $(G2 \Rightarrow G1)$ we verify

`eq1:-`

```
is_edge_1(X,Y),           //for each edge in the first representation
not(is_edge_2(X,Y)),!,    //there is one edge in the other graph
fail.                     //otherwise we reach here, hence, fail.
```

`eq1. //when we get here,  $G1 \Rightarrow G2$  shown`

`eq2:-`

```
is_edge_2(X,Y),
not(is_edge_1(X,Y)),!,
fail.
```

`eq2.`

`same_graph:-eq1,eq2.`

Change representation of a graph

- Given graph **G1** a weighted, edge representation and we need to get **G2** with neighbor representation

G1

`edge(a, b, 15) .`

`gen_graph2:-`

```
    is_edge(X, _, _),
    not(neighbor(X, L)),
    findall(Z, succ(X, Z), L),
    assertz(neighbor(X, L)),
    fail.
```

`gen_graph2.`

`succ(X, Z) :-`

```
    is_edge(X, Y, W),
    Z=p(Y, W) .
```

G2

`neighbor(a, [p(b, 15), ...]) .`

- Note:**

`findall(Z, succ(X, Z), L)` same as `findall(p(Y, W), is_edge(X, Y, W), L)` .

- Given **G2**, generate **G1**. Homework.

Depth first search

- Assume undirected, unweighted graph, edge representation
- 2 stage process: (1) make the trail placing in a queue the vertices in order of visitation (being in the additional knowledge base `vert` (hence a dynamic predicate) is as if we have the vertex colored grey). (2) when done, collect them.

```
df_search(X,_) :-
```

```
    assertz(vert(X)) , //place start/current node in Q
```

```
    is_edge(X,Y) , //take first/next neighbor of current
```

```
    not(vert(Y)) , //if already visited, backtrack to take another; if not, continue from Y
```

```
    df_serach(Y,_) .
```

```
df_search(_,L) :-
```

```
    assertz(vert(end)) ,
```

```
    collect([],L) .//similar to collect in findall, but on vert akb.
```

- It is covering just the connected component of the starting vertex (is similar to `dfs_visit` in Cormen). Need a wrapper to re-run it from all connected components.

Breadth first search

- Assume undirected, unweighted graph, edge representation
- queue made by the akb named `node` (hence dynamic predicate)

```
bf_search(X,_) :-
```

```
    assertz(node(X)) , //add at the end of the Q the current node
```

```
    node(Y) , //reads from front of Q (first time is first=X=ONLY one in Q) current Y (gets Y instantiated)
```

```
    is_edge(Y,Z) , //take first/next neighbor of current Y (gets Z instantiated); if none, backtrack and take next from Q
```

```
    not(node(Z)) , //if Z already visited, backtrack to take another;
```

```
    assertz(node(Z)) , // if not, put Z in Q and
```

```
    fail. // backtrack anyway to another neighbor of Y
```

```
bf_search(_,L) :-
```

```
    assertz(node(end)) ,
```

```
    collect([],L) . //similar to collect in findall, but on node akb.
```

- There is an issue with this implementation for some languages/versions (due to an optimization). Oral explanation. Fix it! Homework.

Logic Programming

Rodica Potolea
Camelia Lemnaru

Lecture #12, 2021

Cluj-Napoca

Computer Science

Agenda

- Graphs isomorphism
 - with metaprogramming
 - With nonmonotonic reasoning
 - Various implementations (compare&contrast)
- Hamiltonian cycle
 - What is it
 - Class of the problem
 - How (not to) solve it

Graphs isomorphism problem statement

- $G1=(V1,E1)$
- $G2=(V2,E2)$
- $G1$ iso $G2$ iff
 - i) $|V1|=|V2|$
 - ii) $\exists \varphi : V1 \rightarrow V2$ a bijection so that
 - iii) $\forall x, y \in V1 [(x, y) \in E1 \Leftrightarrow (\varphi(x), \varphi(y)) \in E2]$

Ex: graph1([n(a,[b,c,d]),
 n(b,[a,c]),
 n(c,[a,b,d]),
 n(d,[a,c]),
]).

graph2([n(1,[2,4]),
 n(2,[1,3,4]),
 n(3,[2,4]),
 n(4,[1,2,3])]).

- Are the 2 graphs isomorphs?

Graphs isomorphism approach

```
?-graph1 (G1) , graph2 (G2) , iso_graph (G1, G2) .
iso_graph (L1, L2) :- eq_perm (L1, L2, eq_neighb) .

eq_perm ([H1 | T1] , L2, EQ) :- //is the perm predicate
    delete (H2, L2, T2) ,      //with an equivalence predicate
    P = .. [EQ, H1, H2] ,      //which is created here
    call (P) ,                 //and called here
    eq_perm (T1, T2, EQ) .
eq_perm ([], [], _) .

eq_neighb (n (N1, L1) , n (N2, L2)) :- //2 neighb pairs are equivalent
    eq_node (N1, N2) ,         //if the nodes are equivalent
    eq_perm (L1, L2, eq_node) . //and the neighb lists are
                                //equivalent

delete (H, [H | T] , T) .
delete (X, [H | T] , [H | R]) :-
    delete (X, T, R) .
```

Nodes equivalence

(implements φ function)

V1 – default logic with side effects

- p a dynamic predicate, used for the nonmonotonic reasoning
- implements φ function (to form p as akb)

```
eq_node(N1,N2):- // if nodes N1 and N2 already form a pair in the akb
    p(N1,N2). // the evaluation continues
eq_node(N1,_):- // if N1 forms a pair with some OTHER node
    p(N1,_),!, // we get an inconsistency, so should NOT allow
    fail. // (N1,N2) form a pair, so, fail to backtrack
eq_node(_,N2):- // symmetric on N2
    p(_,N2),!,
    fail.
eq_node(N1,N2):- // if you reach here, there is no inconsistency
    asserta(p(N1,N2)).//hence pair N1, N2 is a legitimate one.
eq_node(N1,N2):- //if at a later moment, although pair N1, N2 is a
    retract(p(N1,N2)),!, //legitimate one, the reasoning cannot
    fail. //conclude, so remove it from the akb, and fail to
    //backtrack and resume execution WITHOUT the pair in the
```

Nodes equivalence

(implements φ function)

V2 – with arguments (lists)

```
?-graph1 (G1) , graph2 (G2) , iso_graph (G1, G2) .
//instead of the akb p in v1, here we store the pairs in the arg list
// arg 4 = partial list, empty initially
//arg 5 = final list; a copy of the partial list at the end of the execution
```

```
iso_graph (L1, L2) :-eq_perm (L1, L2, eq_neighb, [], Lout) .
eq_perm ([H1 | T1], L2, EQ, LI, LO) :-//same as in v1
    delete (H2, L2, T2) ,
    P=.. [EQ, H1, H2, LI, Lint] , //just that with args
    call (P) ,
    eq_perm (T1, T2, EQ, Lint, LO) .
eq_perm ([], [], _, L, L) .
```

```
eq_neighb (n (n1, L1) , n (N2, L2) , LI, LO) :-
    eq_node (N1, N2, LI, Lint) ,
    eq_perm (L1, L2, eq_node, Lint, LO) .
```

Nodes equivalence

(implements φ function)

V2 – with arguments (lists) – contd.

- p is a pair of nodes in the list argument
- list evolves nonmonotonic (during reasoning) to allow for pairs addition/removal
- models φ function

```
eq_node (N1, N2, LI, LI) :-  
    member (p (N1, N2), LI), !.  
eq_node (N1, N2, LI, [p (N1, N2) | LI]) :-  
    not (member (p (N1, _), LI)),  
    not (member (p (_, N2), LI)).
```

- Those 2 clauses do the same as those 5 in v1
- How? Find 1to1 correspondence

Nodes equivalence

(φ function v2 VS v1)

```
eq_node (N1, N2, LI, LI) :-  
    member (p (N1, N2), LI), !.
```

```
eq_node (N1, N2, LI, [p (N1, N2) | LI]) :-  
  
    not (member (p (N1, _), LI)),  
  
    not (member (p (_, N2), LI)).
```

```
eq_node (N1, N2) :-  
    p (N1, N2) .  
eq_node (N1, _) :-  
    p (N1, _), !,  
    fail.  
eq_node (_, N2) :-  
    p (_, N2), !,  
    fail.  
eq_node (N1, N2) :-  
    asserta (p (N1, N2)) .  
eq_node (N1, N2) :-  
    retract (p (N1, N2)), !,  
    fail.
```

Nodes equivalence

(implements φ function)

V3 – with arguments (incomplete lists)

- Same as v2 just that lists are incomplete
- Just the equivalence changes (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node(N1,N2,[p(N1,N2)|_]) :- !.  
eq_node(N1,_,[p(N1,_)|_]) :-  
    !,fail.  
eq_node(_,N2,[p(_,N2)|_]) :-  
    !,fail.  
eq_node(N1,N2,[_|T]) :-  
    eq_node(N1,N2,T).
```

- Compare v1 with v3. Why is 1 clause missing?
- Compare v2 with w3.

Nodes equivalence

(ϕ function v3 VS v1)

```
eq_node (N1, N2, [p (N1, N2) | _]) :- !.  
eq_node (N1, _, [p (N1, _) | _]) :-  
    !, fail.  
eq_node (_, N2, [p (_, N2) | _]) :-  
    !, fail.  
eq_node (N1, N2, [_ | T]) :-  
    eq_node (N1, N2, T) .
```

```
eq_node (N1, N2) :-  
    p (N1, N2) .  
eq_node (N1, _) :-  
    p (N1, _), !,  
    fail.  
eq_node (_, N2) :-  
    p (_, N2), !,  
    fail.  
eq_node (N1, N2) :-  
    asserta (p (N1, N2)) .  
eq_node (N1, N2) :-  
    retract (p (N1, N2)), !,  
    fail.
```

Nodes equivalence

(φ function v3 VS v2)

```
eq_node (N1,N2,[p(N1,N2)|_]) :- !.  
eq_node (N1,_,[p(N1,_)|_]) :-  
    !,fail.  
eq_node (_,N2,[p(_,N2)|_]) :-  
    !,fail.  
eq_node (N1,N2,[_|T]) :-  
    eq_node (N1,N2,T).
```

```
eq_node (N1,N2,LI,LI) :-  
    member (p(N1,N2),LI), !.  
eq_node (N1,N2,LI,[p(N1,N2)|LI]) :-  
    not (member (p(N1,_) , LI)),  
    not (member (p (_,N2) , LI)).
```

Hamiltonian cycle

- Consider the undirected weighted graph:

edge (a, b, 7) .

edge (a, c, 1) .

edge (a, d, 8) .

edge (b, c, 2) .

edge (b, f, 5) .

edge (b, e, 10) .

edge (c, d, 3) .

edge (d, f, 4) .

edge (d, e, 8) .

edge (f, e, 3) .

is_edge (X, Y, W) :-

edge (X, Y, W) ;

edge (Y, X, W) .

Hamiltonian cycle (NP complete/NPhard problem)

```
hamilton(N,X,Cycle,Cost):-  
    N1 is N-1,  
    try(N1,X,X,[X],Cycle,0,Cost).  
  
?-hamilton(6,a,Cycle,Cost).  
  
try(N,X,Y,Pway,Cycle,Pcost,Cost):-  
    is_edge(Y,Z,W),  
    not(member(Z,Pway)),  
    N1 is N-1, N1>0,  
    NewPcost is Pcost+W,  
    try(N1,X,Z,[Z|Pway],Cycle,NewPcost,Cost).  
try(0,X,Y,Pway,[X|Pway],Pcost,Cost):-  
    is_edge(Y,X,W),  
    Cost is Pcost+W.
```

Hamiltonian cycle (NP complete/NP-hard problem)

Should not be addressed straight away unless the space is VERY small
(how small? Size ~ 20 ? TBD orally)

